

Package ‘simStateSpace’

March 25, 2026

Title Simulate Data from State Space Models

Version 1.2.16

Description Provides a streamlined and user-friendly framework for simulating data in state space models, particularly when the number of subjects/units (n) exceeds one, a scenario commonly encountered in social and behavioral sciences. This package was designed to generate data for the simulations performed in Pesigan, Russell, and Chow (2025) <[doi:10.1037/met0000779](https://doi.org/10.1037/met0000779)>.

URL <https://github.com/jeksterslab/simStateSpace>,
<https://jeksterslab.github.io/simStateSpace/>

BugReports <https://github.com/jeksterslab/simStateSpace/issues>

License GPL (>= 3)

Encoding UTF-8

Roxygen list(markdown = TRUE)

Depends R (>= 4.1.0)

LinkingTo Rcpp (>= 1.0.12), RcppArmadillo (>= 15.0.2-2)

Imports Rcpp (>= 1.0.12), stats

Suggests knitr, rmarkdown, testthat, expm, dynr

SystemRequirements GNU GSL (>= 2.5)

Config/roxygen2/version 7.3.3.9000

NeedsCompilation yes

Author Ivan Jacob Agaloos Pesigan [aut, cre, cph] (ORCID:
<<https://orcid.org/0000-0003-4818-8420>>),
Michael A. Russell [ctb] (ORCID:
<<https://orcid.org/0000-0002-3956-604X>>),
Sy-Miin Chow [ctb] (ORCID: <<https://orcid.org/0000-0003-1938-027X>>)

Maintainer Ivan Jacob Agaloos Pesigan <r.jeksterslab@gmail.com>

Contents

as.data.frame.simstatespace	3
as.matrix.simstatespace	5
LinSDE2SSM	8
LinSDECovEta	10
LinSDECovY	12
LinSDEInterceptEta	13
LinSDEInterceptY	14
LinSDEMeanEta	16
LinSDEMeanY	17
plot.simstatespace	18
print.simstatespace	21
ProjectToHurwitz	23
ProjectToStability	25
SimAlphaN	26
SimBetaN	27
SimBetaN2	29
SimBetaNCovariate	30
SimCovDiagN	32
SimCovN	33
SimIotaN	34
SimMuN	35
SimMVN	36
SimNuN	37
SimPhiN	38
SimPhiN2	40
SimPhiNCovariate	41
SimSSMFixed	43
SimSSMIVary	48
SimSSMLinGrowth	52
SimSSMLinGrowthIVary	56
SimSSMLinSDEFixed	60
SimSSMLinSDEIVary	65
SimSSMOUFixed	70
SimSSMOUIVary	76
SimSSMVARFixed	81
SimSSMVARIVary	84
SpectralAbscissa	88
SpectralRadius	89
SSMCovEta	90
SSMCovY	91
SSMInterceptEta	93
SSMInterceptY	94
SSMMeanEta	95
SSMMeanY	96
TestPhi	98
TestPhiHurwitz	99

TestStability	100
TestStationarity	101

Index	103
-------	-----

as.data.frame.simstatespace
<i>Coerce an Object of Class simstatespace to a Data Frame</i>

Description

Coerce an Object of Class simstatespace to a Data Frame

Usage

```
## S3 method for class 'simstatespace'
as.data.frame(
  x,
  row.names = NULL,
  optional = FALSE,
  eta = FALSE,
  long = TRUE,
  burnin = 0,
  reset_time = TRUE,
  ...
)
```

Arguments

x	Object of class simstatespace.
row.names	NULL or character vector giving the row names for the data frame. Missing values are not allowed.
optional	Logical. If TRUE, setting row names and converting column names is optional.
eta	Logical. If eta = TRUE, include eta. If eta = FALSE, exclude eta.
long	Logical. If long = TRUE, use long format. If long = FALSE, use wide format.
burnin	Positive integer. Initial data points to discard. Default is zero.
reset_time	Logical. Reset the time index after burnin.
...	Additional arguments.

Author(s)

Ivan Jacob Agaloos Pesigan

Examples

```

# prepare parameters
set.seed(42)
## number of individuals
n <- 5
## time points
time <- 50
## dynamic structure
p <- 3
mu0 <- rep(x = 0, times = p)
sigma0 <- diag(p)
sigma0_l <- t(chol(sigma0))
alpha <- rep(x = 0, times = p)
beta <- 0.50 * diag(p)
psi <- diag(p)
psi_l <- t(chol(psi))
## measurement model
k <- 3
nu <- rep(x = 0, times = k)
lambda <- diag(k)
theta <- 0.50 * diag(k)
theta_l <- t(chol(theta))
## covariates
j <- 2
x <- lapply(
  X = seq_len(n),
  FUN = function(i) {
    matrix(
      data = stats::rnorm(n = time * j),
      nrow = j,
      ncol = time
    )
  }
)
gamma <- diag(x = 0.10, nrow = p, ncol = j)
kappa <- diag(x = 0.10, nrow = k, ncol = j)

# Type 0
ssm <- SimSSMFixed(
  n = n,
  time = time,
  mu0 = mu0,
  sigma0_l = sigma0_l,
  alpha = alpha,
  beta = beta,
  psi_l = psi_l,
  nu = nu,
  lambda = lambda,
  theta_l = theta_l,
  type = 0
)

```

```
head(as.data.frame(ssm))
head(as.data.frame(ssm, long = FALSE))

# Type 1
ssm <- SimSSMFixed(
  n = n,
  time = time,
  mu0 = mu0,
  sigma0_l = sigma0_l,
  alpha = alpha,
  beta = beta,
  psi_l = psi_l,
  nu = nu,
  lambda = lambda,
  theta_l = theta_l,
  type = 1,
  x = x,
  gamma = gamma
)

head(as.data.frame(ssm))
head(as.data.frame(ssm, long = FALSE))

# Type 2
ssm <- SimSSMFixed(
  n = n,
  time = time,
  mu0 = mu0,
  sigma0_l = sigma0_l,
  alpha = alpha,
  beta = beta,
  psi_l = psi_l,
  nu = nu,
  lambda = lambda,
  theta_l = theta_l,
  type = 2,
  x = x,
  gamma = gamma,
  kappa = kappa
)

head(as.data.frame(ssm))
head(as.data.frame(ssm, long = FALSE))
```

Description

Coerce an Object of Class `simstatespace` to a Matrix

Usage

```
## S3 method for class 'simstatespace'
as.matrix(x, eta = FALSE, long = TRUE, burnin = 0, reset_time = TRUE, ...)
```

Arguments

<code>x</code>	Object of class <code>simstatespace</code> .
<code>eta</code>	Logical. If <code>eta = TRUE</code> , include <code>eta</code> . If <code>eta = FALSE</code> , exclude <code>eta</code> .
<code>long</code>	Logical. If <code>long = TRUE</code> , use long format. If <code>long = FALSE</code> , use wide format.
<code>burnin</code>	Positive integer. Initial data points to discard. Default is zero.
<code>reset_time</code>	Logical. Reset the time index after burnin.
<code>...</code>	Additional arguments.

Author(s)

Ivan Jacob Agaloos Pesigan

Examples

```
# prepare parameters
set.seed(42)
## number of individuals
n <- 5
## time points
time <- 50
## dynamic structure
p <- 3
mu0 <- rep(x = 0, times = p)
sigma0 <- diag(p)
sigma0_l <- t(chol(sigma0))
alpha <- rep(x = 0, times = p)
beta <- 0.50 * diag(p)
psi <- diag(p)
psi_l <- t(chol(psi))
## measurement model
k <- 3
nu <- rep(x = 0, times = k)
lambda <- diag(k)
theta <- 0.50 * diag(k)
theta_l <- t(chol(theta))
## covariates
j <- 2
x <- lapply(
  X = seq_len(n),
  FUN = function(i) {
    matrix(
```

```

        data = stats::rnorm(n = time * j),
        nrow = j,
        ncol = time
    )
}
)
gamma <- diag(x = 0.10, nrow = p, ncol = j)
kappa <- diag(x = 0.10, nrow = k, ncol = j)

# Type 0
ssm <- SimSSMFixed(
  n = n,
  time = time,
  mu0 = mu0,
  sigma0_l = sigma0_l,
  alpha = alpha,
  beta = beta,
  psi_l = psi_l,
  nu = nu,
  lambda = lambda,
  theta_l = theta_l,
  type = 0
)

head(as.matrix(ssm))
head(as.matrix(ssm, long = FALSE))

# Type 1
ssm <- SimSSMFixed(
  n = n,
  time = time,
  mu0 = mu0,
  sigma0_l = sigma0_l,
  alpha = alpha,
  beta = beta,
  psi_l = psi_l,
  nu = nu,
  lambda = lambda,
  theta_l = theta_l,
  type = 1,
  x = x,
  gamma = gamma
)

head(as.matrix(ssm))
head(as.matrix(ssm, long = FALSE))

# Type 2
ssm <- SimSSMFixed(
  n = n,
  time = time,
  mu0 = mu0,
  sigma0_l = sigma0_l,

```

```

alpha = alpha,
beta = beta,
psi_l = psi_l,
nu = nu,
lambda = lambda,
theta_l = theta_l,
type = 2,
x = x,
gamma = gamma,
kappa = kappa
)

head(as.matrix(ssm))
head(as.matrix(ssm, long = FALSE))

```

LinSDE2SSM

Convert Parameters from the Linear Stochastic Differential Equation Model to State Space Model Parameterization

Description

This function converts parameters from the linear stochastic differential equation model to state space model parameterization.

Usage

```
LinSDE2SSM(iota, phi, sigma_l, delta_t)
```

Arguments

iota	Numeric vector. An unobserved term that is constant over time (ι).
phi	Numeric matrix. The drift matrix which represents the rate of change of the solution in the absence of any random fluctuations (Φ).
sigma_l	Numeric matrix. Cholesky factorization ($\text{t}(\text{chol}(\text{sigma}))$) of the covariance matrix of volatility or randomness in the process (Σ).
delta_t	Numeric. Time interval (Δ_t).

Details

Let the linear stochastic equation model be given by

$$d\boldsymbol{\eta}_{i,t} = (\boldsymbol{\iota} + \boldsymbol{\Phi}\boldsymbol{\eta}_{i,t}) dt + \boldsymbol{\Sigma}^{\frac{1}{2}} d\mathbf{W}_{i,t}$$

for individual i and time t . The discrete-time state space model given below represents the discrete-time solution for the linear stochastic differential equation.

$$\boldsymbol{\eta}_{i,t_{l_i}} = \boldsymbol{\alpha}_{\Delta t_{l_i}} + \boldsymbol{\beta}_{\Delta t_{l_i}} \boldsymbol{\eta}_{i,t_{l_i}-1} + \boldsymbol{\zeta}_{i,t_{l_i}}, \quad \text{with} \quad \boldsymbol{\zeta}_{i,t_{l_i}} \sim \mathcal{N}(\mathbf{0}, \boldsymbol{\Psi}_{\Delta t_{l_i}})$$

with

$$\beta_{\Delta t_{l_i}} = \exp(\Delta t \Phi),$$

$$\alpha_{\Delta t_{l_i}} = \Phi^{-1}(\beta - \mathbf{I}_p) \iota, \quad \text{and}$$

$$\text{vec}(\Psi_{\Delta t_{l_i}}) = [(\Phi \otimes \mathbf{I}_p) + (\mathbf{I}_p \otimes \Phi)] [\exp((\Phi \otimes \mathbf{I}_p) + (\mathbf{I}_p \otimes \Phi) \Delta t) - \mathbf{I}_{p \times p}] \text{vec}(\Sigma)$$

where t denotes continuous-time processes that can be defined by any arbitrary time point, t_{l_i} the l^{th} observed measurement occasion for individual i , p the number of latent variables and Δt the time interval.

Value

Returns a list of state space parameters:

- alpha: Numeric vector. Vector of constant values for the dynamic model (α).
- beta: Numeric matrix. Transition matrix relating the values of the latent variables from the previous time point to the current time point (β).
- psi_1: Numeric matrix. Cholesky factorization ($\text{t}(\text{chol}(\text{psi}))$) of the process noise covariance matrix Ψ .

Author(s)

Ivan Jacob Agaloos Pesigan

References

Harvey, A. C. (1990). Forecasting, structural time series models and the Kalman filter. Cambridge University Press. doi:10.1017/cbo9781107049994

See Also

Other Simulation of State Space Models Data Functions: [LinSDECovEta\(\)](#), [LinSDECovY\(\)](#), [LinSDEInterceptEta\(\)](#), [LinSDEInterceptY\(\)](#), [LinSDEMeanEta\(\)](#), [LinSDEMeanY\(\)](#), [ProjectToHurwitz\(\)](#), [ProjectToStability\(\)](#), [SSMCovEta\(\)](#), [SSMCovY\(\)](#), [SSMInterceptEta\(\)](#), [SSMInterceptY\(\)](#), [SSMMeanEta\(\)](#), [SSMMeanY\(\)](#), [SimAlphaN\(\)](#), [SimBetaN\(\)](#), [SimBetaN2\(\)](#), [SimBetaNCovariate\(\)](#), [SimCovDiagN\(\)](#), [SimCovN\(\)](#), [SimIotaN\(\)](#), [SimMVN\(\)](#), [SimMuN\(\)](#), [SimNuN\(\)](#), [SimPhiN\(\)](#), [SimPhiN2\(\)](#), [SimPhiNCovariate\(\)](#), [SimSSMFixed\(\)](#), [SimSSMIVary\(\)](#), [SimSSMLinGrowth\(\)](#), [SimSSMLinGrowthIVary\(\)](#), [SimSSMLinSDEFixed\(\)](#), [SimSSMLinSDEIVary\(\)](#), [SimSSMOUFixed\(\)](#), [SimSSMOUIVary\(\)](#), [SimSSMVARFixed\(\)](#), [SimSSMVARIVary\(\)](#), [SpectralRadius\(\)](#), [TestPhi\(\)](#), [TestPhiHurwitz\(\)](#), [TestStability\(\)](#), [TestStationarity\(\)](#)

Examples

```

p <- 2
iota <- c(0.317, 0.230)
phi <- matrix(
  data = c(
    -0.10,
    0.05,
    0.05,
    -0.10
  ),
  nrow = p
)
sigma <- matrix(
  data = c(
    2.79,
    0.06,
    0.06,
    3.27
  ),
  nrow = p
)
sigma_l <- t(chol(sigma))
delta_t <- 0.10

LinSDE2SSM(
  iota = iota,
  phi = phi,
  sigma_l = sigma_l,
  delta_t = delta_t
)

```

LinSDECovEta

Steady-State Covariance Matrix for the Latent Variables in the Linear Stochastic Differential Equation Model

Description

The steady-state covariance matrix for the latent variables in the linear stochastic differential equation model $\text{Cov}(\boldsymbol{\eta})$ is the solution to the Sylvester equation, i.e.

$$\mathbf{A}\mathbf{X} + \mathbf{X}\mathbf{B} + \mathbf{C} = \mathbf{0},$$

where $\mathbf{X}$ is unknown, $\mathbf{A} = \boldsymbol{\Phi}$, $\mathbf{B} = \boldsymbol{\Phi}'$, and $\mathbf{C} = \boldsymbol{\Sigma}$ where $\boldsymbol{\Phi}$ is the drift matrix and $\boldsymbol{\Sigma}$ is the covariance matrix of volatility or randomness in the process.

Usage

```
LinSDECovEta(phi, sigma)
```

Arguments

phi	Numeric matrix. The drift matrix which represents the rate of change of the solution in the absence of any random fluctuations (Φ).
sigma	Numeric matrix. The covariance matrix of volatility or randomness in the process (Σ).

Author(s)

Ivan Jacob Agaloos Pesigan

See Also

Other Simulation of State Space Models Data Functions: [LinSDE2SSM\(\)](#), [LinSDECovY\(\)](#), [LinSDEInterceptEta\(\)](#), [LinSDEInterceptY\(\)](#), [LinSDEMeanEta\(\)](#), [LinSDEMeanY\(\)](#), [ProjectToHurwitz\(\)](#), [ProjectToStability\(\)](#), [SSMCovEta\(\)](#), [SSMCovY\(\)](#), [SSMInterceptEta\(\)](#), [SSMInterceptY\(\)](#), [SSMMeanEta\(\)](#), [SSMMeanY\(\)](#), [SimAlphaN\(\)](#), [SimBetaN\(\)](#), [SimBetaN2\(\)](#), [SimBetaNCovariate\(\)](#), [SimCovDiagN\(\)](#), [SimCovN\(\)](#), [SimIotaN\(\)](#), [SimMVN\(\)](#), [SimMuN\(\)](#), [SimNuN\(\)](#), [SimPhiN\(\)](#), [SimPhiN2\(\)](#), [SimPhiNCovariate\(\)](#), [SimSSMFixed\(\)](#), [SimSSMIVary\(\)](#), [SimSSMLinGrowth\(\)](#), [SimSSMLinGrowthIVary\(\)](#), [SimSSMLinSDEFixed\(\)](#), [SimSSMLinSDEIVary\(\)](#), [SimSSMOUFixed\(\)](#), [SimSSMOUIVary\(\)](#), [SimSSMVARFixed\(\)](#), [SimSSMVARIVary\(\)](#), [SpectralRadius\(\)](#), [TestPhi\(\)](#), [TestPhiHurwitz\(\)](#), [TestStability\(\)](#), [TestStationarity\(\)](#)

Examples

```
phi <- matrix(
  data = c(
    -0.357, 0.771, -0.450,
    0.0, -0.511, 0.729,
    0.0, 0.0, -0.693
  ),
  nrow = 3
)
sigma <- matrix(
  data = c(
    0.24455556, 0.02201587, -0.05004762,
    0.02201587, 0.07067800, 0.01539456,
    -0.05004762, 0.01539456, 0.07553061
  ),
  nrow = 3
)
LinSDECovEta(
  phi = phi,
  sigma = sigma
)
```

LinSDECovY

Steady-State Covariance Matrix for the Observed Variables in the Linear Stochastic Differential Equation Model

Description

The steady-state covariance matrix for the observed variables in the linear stochastic differential equation model $\text{Cov}(\mathbf{y})$ is given by

$$\text{Cov}(\mathbf{y}) = \mathbf{\Lambda} \text{Cov}(\boldsymbol{\eta}) \mathbf{\Lambda}' + \boldsymbol{\Theta}$$

where $\mathbf{\Lambda}$ is the matrix of factor loadings, $\boldsymbol{\Theta}$ is the covariance matrix of the measurement error, and $\text{Cov}(\boldsymbol{\eta})$ is the steady-state covariance matrix for the latent variables.

Usage

```
LinSDECovY(lambda, theta, cov_eta)
```

Arguments

lambda	Numeric matrix. Factor loading matrix linking the latent variables to the observed variables ($\mathbf{\Lambda}$).
theta	Numeric matrix. The covariance matrix of the measurement error ($\boldsymbol{\Theta}$).
cov_eta	Numeric matrix. The steady-state covariance matrix for the latent variables in the linear stochastic differential equation model

Author(s)

Ivan Jacob Agaloos Pesigan

See Also

Other Simulation of State Space Models Data Functions: [LinSDE2SSM\(\)](#), [LinSDECovEta\(\)](#), [LinSDEInterceptEta\(\)](#), [LinSDEInterceptY\(\)](#), [LinSDEMeanEta\(\)](#), [LinSDEMeanY\(\)](#), [ProjectToHurwitz\(\)](#), [ProjectToStability\(\)](#), [SSMCovEta\(\)](#), [SSMCovY\(\)](#), [SSMInterceptEta\(\)](#), [SSMInterceptY\(\)](#), [SSMMeanEta\(\)](#), [SSMMeanY\(\)](#), [SimAlphaN\(\)](#), [SimBetaN\(\)](#), [SimBetaN2\(\)](#), [SimBetaNCovariate\(\)](#), [SimCovDiagN\(\)](#), [SimCovN\(\)](#), [SimIotaN\(\)](#), [SimMVN\(\)](#), [SimMuN\(\)](#), [SimNuN\(\)](#), [SimPhiN\(\)](#), [SimPhiN2\(\)](#), [SimPhiNCovariate\(\)](#), [SimSSMFixed\(\)](#), [SimSSMIVary\(\)](#), [SimSSMLinGrowth\(\)](#), [SimSSMLinGrowthIVary\(\)](#), [SimSSMLinSDEFixed\(\)](#), [SimSSMLinSDEIVary\(\)](#), [SimSSMOUFixed\(\)](#), [SimSSMOUIVary\(\)](#), [SimSSMVARFixed\(\)](#), [SimSSMVARIVary\(\)](#), [SpectralRadius\(\)](#), [TestPhi\(\)](#), [TestPhiHurwitz\(\)](#), [TestStability\(\)](#), [TestStationarity\(\)](#)

Examples

```
phi <- matrix(
  data = c(
    -0.357, 0.771, -0.450,
    0.0, -0.511, 0.729,
    0.0, 0.0, -0.693
```

```

    ),
    nrow = 3
  )
  sigma <- matrix(
    data = c(
      0.24455556, 0.02201587, -0.05004762,
      0.02201587, 0.07067800, 0.01539456,
      -0.05004762, 0.01539456, 0.07553061
    ),
    nrow = 3
  )
  lambda <- diag(3)
  theta <- diag(3)
  cov_eta <- LinSDECovEta(
    phi = phi,
    sigma = sigma
  )
  LinSDECovY(
    lambda = lambda,
    theta = theta,
    cov_eta = cov_eta
  )

```

LinSDEInterceptEta	<i>Intercept from Steady-State Mean Vector for the Latent Variables in the Linear Stochastic Differential Equation Model</i>
--------------------	--

Description

The intercept vector for the latent variables in the linear stochastic differential equation model $\boldsymbol{\iota}$ is given by

$$\boldsymbol{\iota} = -\boldsymbol{\Phi} \text{Mean}(\boldsymbol{\eta})$$

where $\boldsymbol{\Phi}$ is the drift matrix and $\text{Mean}(\boldsymbol{\eta})$ is the steady-state mean vector for the latent variables.

Usage

```
LinSDEInterceptEta(phi, mean_eta)
```

Arguments

phi	Numeric matrix. The drift matrix which represents the rate of change of the solution in the absence of any random fluctuations ($\boldsymbol{\Phi}$).
mean_eta	Numeric vector. Steady-state mean vector of the latent variables $\text{Mean}(\boldsymbol{\eta})$.

Author(s)

Ivan Jacob Agaloos Pesigan

See Also

Other Simulation of State Space Models Data Functions: [LinSDE2SSM\(\)](#), [LinSDECovEta\(\)](#), [LinSDECovY\(\)](#), [LinSDEInterceptY\(\)](#), [LinSDEMeanEta\(\)](#), [LinSDEMeanY\(\)](#), [ProjectToHurwitz\(\)](#), [ProjectToStability\(\)](#), [SSMCovEta\(\)](#), [SSMCovY\(\)](#), [SSMInterceptEta\(\)](#), [SSMInterceptY\(\)](#), [SSMMeanEta\(\)](#), [SSMMeanY\(\)](#), [SimAlphaN\(\)](#), [SimBetaN\(\)](#), [SimBetaN2\(\)](#), [SimBetaNCovariate\(\)](#), [SimCovDiagN\(\)](#), [SimCovN\(\)](#), [SimIotaN\(\)](#), [SimMVN\(\)](#), [SimMuN\(\)](#), [SimNuN\(\)](#), [SimPhiN\(\)](#), [SimPhiN2\(\)](#), [SimPhiNCovariate\(\)](#), [SimSSMFixed\(\)](#), [SimSSMIVary\(\)](#), [SimSSMLinGrowth\(\)](#), [SimSSMLinGrowthIVary\(\)](#), [SimSSMLinSDEFixed\(\)](#), [SimSSMLinSDEIVary\(\)](#), [SimSSMOUFixed\(\)](#), [SimSSMOUIVary\(\)](#), [SimSSMVARFixed\(\)](#), [SimSSMVARIVary\(\)](#), [SpectralRadius\(\)](#), [TestPhi\(\)](#), [TestPhiHurwitz\(\)](#), [TestStability\(\)](#), [TestStationarity\(\)](#)

Examples

```
phi <- matrix(
  data = c(
    -0.357, 0.771, -0.450,
    0.0, -0.511, 0.729,
    0.0, 0.0, -0.693
  ),
  nrow = 3
)
iota <- rep(x = 1, times = 3)
lambda <- diag(3)
nu <- rep(x = 1, times = 3)
LinSDEMeanEta(
  phi = phi,
  iota = iota
)
```

LinSDEInterceptY

*Intercept from Steady-State Mean Vector for the Observed Variables
in the Linear Stochastic Differential Equation Model*

Description

The intercept vector for the observed variables in the linear stochastic differential equation model ν is given by

$$\nu = \text{Mean}(\mathbf{y}) - \mathbf{\Lambda} \text{Mean}(\boldsymbol{\eta})$$

where $\mathbf{\Lambda}$ is the matrix of factor loadings, $\text{Mean}(\mathbf{y})$ is the steady-state mean vector for the observed variables, and $\text{Mean}(\boldsymbol{\eta})$ is the steady-state mean vector for the latent variables.

Usage

```
LinSDEInterceptY(mean_y, mean_eta, lambda)
```

Arguments

mean_y	Numeric vector. Steady-state mean vector of the observed variables Mean (y).
mean_eta	Numeric vector. Steady-state mean vector of the latent variables Mean (η).
lambda	Numeric matrix. Factor loading matrix linking the latent variables to the observed variables (Λ).

Author(s)

Ivan Jacob Agaloos Pesigan

See Also

Other Simulation of State Space Models Data Functions: [LinSDE2SSM\(\)](#), [LinSDECovEta\(\)](#), [LinSDECovY\(\)](#), [LinSDEInterceptEta\(\)](#), [LinSDEMeanEta\(\)](#), [LinSDEMeanY\(\)](#), [ProjectToHurwitz\(\)](#), [ProjectToStability\(\)](#), [SSMCovEta\(\)](#), [SSMCovY\(\)](#), [SSMInterceptEta\(\)](#), [SSMInterceptY\(\)](#), [SSMMeanEta\(\)](#), [SSMMeanY\(\)](#), [SimAlphaN\(\)](#), [SimBetaN\(\)](#), [SimBetaN2\(\)](#), [SimBetaNCovariate\(\)](#), [SimCovDiagN\(\)](#), [SimCovN\(\)](#), [SimIotaN\(\)](#), [SimMVN\(\)](#), [SimMuN\(\)](#), [SimNuN\(\)](#), [SimPhiN\(\)](#), [SimPhiN2\(\)](#), [SimPhiNCovariate\(\)](#), [SimSSMFixed\(\)](#), [SimSSMIVary\(\)](#), [SimSSMLinGrowth\(\)](#), [SimSSMLinGrowthIVary\(\)](#), [SimSSMLinSDEFixed\(\)](#), [SimSSMLinSDEIVary\(\)](#), [SimSSMOUFixed\(\)](#), [SimSSMOUIVary\(\)](#), [SimSSMVARFixed\(\)](#), [SimSSMVARIVary\(\)](#), [SpectralRadius\(\)](#), [TestPhi\(\)](#), [TestPhiHurwitz\(\)](#), [TestStability\(\)](#), [TestStationarity\(\)](#)

Examples

```
phi <- matrix(
  data = c(
    -0.357, 0.771, -0.450,
    0.0, -0.511, 0.729,
    0.0, 0.0, -0.693
  ),
  nrow = 3
)
iota <- rep(x = 1, times = 3)
lambda <- diag(3)
nu <- rep(x = 1, times = 3)
mean_eta <- LinSDEMeanEta(
  phi = phi,
  iota = iota
)
mean_y <- LinSDEMeanY(
  nu = nu,
  lambda = lambda,
  mean_eta = mean_eta
)
LinSDEInterceptY(
  mean_y = mean_y,
  mean_eta = mean_eta,
  lambda = lambda
)
```

LinSDEMeanEta	<i>Steady-State Mean Vector for the Latent Variables in the Linear Stochastic Differential Equation Model</i>
---------------	---

Description

The steady-state mean vector for the latent variables in the linear stochastic differential equation model $\text{Mean}(\boldsymbol{\eta})$ is given by

$$\text{Mean}(\boldsymbol{\eta}) = -\boldsymbol{\Phi}^{-1}\boldsymbol{\iota}$$

where $\boldsymbol{\Phi}$ is the drift matrix, and $\boldsymbol{\iota}$ is an unobserved term that is constant over time.

Usage

```
LinSDEMeanEta(phi, iota)
```

Arguments

phi	Numeric matrix. The drift matrix which represents the rate of change of the solution in the absence of any random fluctuations ($\boldsymbol{\Phi}$).
iota	Numeric vector. An unobserved term that is constant over time ($\boldsymbol{\iota}$).

Author(s)

Ivan Jacob Agaloos Pesigan

See Also

Other Simulation of State Space Models Data Functions: [LinSDE2SSM\(\)](#), [LinSDECovEta\(\)](#), [LinSDECovY\(\)](#), [LinSDEInterceptEta\(\)](#), [LinSDEInterceptY\(\)](#), [LinSDEMeanY\(\)](#), [ProjectToHurwitz\(\)](#), [ProjectToStability\(\)](#), [SSMCovEta\(\)](#), [SSMCovY\(\)](#), [SSMInterceptEta\(\)](#), [SSMInterceptY\(\)](#), [SSMMeanEta\(\)](#), [SSMMeanY\(\)](#), [SimAlphaN\(\)](#), [SimBetaN\(\)](#), [SimBetaN2\(\)](#), [SimBetaNCovariate\(\)](#), [SimCovDiagN\(\)](#), [SimCovN\(\)](#), [SimIotaN\(\)](#), [SimMVN\(\)](#), [SimMuN\(\)](#), [SimNuN\(\)](#), [SimPhiN\(\)](#), [SimPhiN2\(\)](#), [SimPhiNCovariate\(\)](#), [SimSSMFixed\(\)](#), [SimSSMIVary\(\)](#), [SimSSMLinGrowth\(\)](#), [SimSSMLinGrowthIVary\(\)](#), [SimSSMLinSDEFixed\(\)](#), [SimSSMLinSDEIVary\(\)](#), [SimSSMOUFixed\(\)](#), [SimSSMOUIVary\(\)](#), [SimSSMVARFixed\(\)](#), [SimSSMVARIVary\(\)](#), [SpectralRadius\(\)](#), [TestPhi\(\)](#), [TestPhiHurwitz\(\)](#), [TestStability\(\)](#), [TestStationarity\(\)](#)

Examples

```
phi <- matrix(
  data = c(
    -0.357, 0.771, -0.450,
    0.0, -0.511, 0.729,
    0.0, 0.0, -0.693
  ),
  nrow = 3
)
iota <- rep(x = 1, times = 3)
LinSDEMeanEta(
```



```

    phi = phi,
    iota = iota
)

```

LinSDEMeanY

Steady-State Mean Vector for the Observed Variables in the Linear Stochastic Differential Equation Model

Description

The steady-state mean vector for the observed variables in the linear stochastic differential equation model $\text{Mean}(\mathbf{y})$ is given by

$$\text{Mean}(\mathbf{y}) = \boldsymbol{\nu} + \mathbf{\Lambda} \text{Mean}(\boldsymbol{\eta})$$

where $\boldsymbol{\nu}$ is the vector of intercept values for the measurement model, $\mathbf{\Lambda}$ is the matrix of factor loadings, and $\text{Mean}(\boldsymbol{\eta})$ is the steady-state mean vector for the latent variables.

Usage

```
LinSDEMeanY(nu, lambda, mean_eta)
```

Arguments

nu	Numeric vector. Vector of intercept values for the measurement model ($\boldsymbol{\nu}$).
lambda	Numeric matrix. Factor loading matrix linking the latent variables to the observed variables ($\mathbf{\Lambda}$).
mean_eta	Numeric vector. Steady-state mean vector of the latent variables $\text{Mean}(\boldsymbol{\eta})$.

Author(s)

Ivan Jacob Agaloos Pesigan

See Also

Other Simulation of State Space Models Data Functions: [LinSDE2SSM\(\)](#), [LinSDECovEta\(\)](#), [LinSDECovY\(\)](#), [LinSDEInterceptEta\(\)](#), [LinSDEInterceptY\(\)](#), [LinSDEMeanEta\(\)](#), [ProjectToHurwitz\(\)](#), [ProjectToStability\(\)](#), [SSMCovEta\(\)](#), [SSMCovY\(\)](#), [SSMInterceptEta\(\)](#), [SSMInterceptY\(\)](#), [SSMMeanEta\(\)](#), [SSMMeanY\(\)](#), [SimAlphaN\(\)](#), [SimBetaN\(\)](#), [SimBetaN2\(\)](#), [SimBetaNCovariate\(\)](#), [SimCovDiagN\(\)](#), [SimCovN\(\)](#), [SimIotaN\(\)](#), [SimMVN\(\)](#), [SimMuN\(\)](#), [SimNuN\(\)](#), [SimPhiN\(\)](#), [SimPhiN2\(\)](#), [SimPhiNCovariate\(\)](#), [SimSSMFixed\(\)](#), [SimSSMIVary\(\)](#), [SimSSMLinGrowth\(\)](#), [SimSSMLinGrowthIVary\(\)](#), [SimSSMLinSDEFixed\(\)](#), [SimSSMLinSDEIVary\(\)](#), [SimSSMOUFixed\(\)](#), [SimSSMOUIVary\(\)](#), [SimSSMVARFixed\(\)](#), [SimSSMVARIVary\(\)](#), [SpectralRadius\(\)](#), [TestPhi\(\)](#), [TestPhiHurwitz\(\)](#), [TestStability\(\)](#), [TestStationarity\(\)](#)

Examples

```

phi <- matrix(
  data = c(
    -0.357, 0.771, -0.450,
    0.0, -0.511, 0.729,
    0.0, 0.0, -0.693
  ),
  nrow = 3
)
iota <- rep(x = 1, times = 3)
lambda <- diag(3)
nu <- rep(x = 1, times = 3)
mean_eta <- LinSDEMeanEta(
  phi = phi,
  iota = iota
)
LinSDEMeanY(
  nu = nu,
  lambda = lambda,
  mean_eta = mean_eta
)

```

plot.simstatespace *Plot Method for an Object of Class simstatespace*

Description

Plot Method for an Object of Class simstatespace

Usage

```

## S3 method for class 'simstatespace'
plot(
  x,
  id = NULL,
  time = NULL,
  eta = FALSE,
  type = "b",
  burnin = 0,
  reset_time = TRUE,
  ...
)

```

Arguments

x Object of class simstatespace.

id	Numeric vector. Optional id numbers to plot. If id = NULL, plot all available data.
time	Numeric vector. Optional time points to plot. If time = NULL, plot all available data.
eta	Logical. If eta = TRUE, plot the latent variables. If eta = FALSE, plot the observed variables.
type	Character indicating the type of plotting; actually any of the types as in plot.default() .
burnin	Positive integer. Initial data points to discard. Default is zero.
reset_time	Logical. Reset the time index after burnin.
...	Additional arguments.

Author(s)

Ivan Jacob Agaloos Pesigan

Examples

```
# prepare parameters
set.seed(42)
## number of individuals
n <- 5
## time points
time <- 50
## dynamic structure
p <- 3
mu0 <- rep(x = 0, times = p)
sigma0 <- diag(p)
sigma0_l <- t(chol(sigma0))
alpha <- rep(x = 0, times = p)
beta <- 0.50 * diag(p)
psi <- diag(p)
psi_l <- t(chol(psi))
## measurement model
k <- 3
nu <- rep(x = 0, times = k)
lambda <- diag(k)
theta <- 0.50 * diag(k)
theta_l <- t(chol(theta))
## covariates
j <- 2
x <- lapply(
  X = seq_len(n),
  FUN = function(i) {
    matrix(
      data = stats::rnorm(n = time * j),
      nrow = j,
      ncol = time
    )
  }
)
```

```
gamma <- diag(x = 0.10, nrow = p, ncol = j)
kappa <- diag(x = 0.10, nrow = k, ncol = j)
```

```
# Type 0
ssm <- SimSSMFixed(
  n = n,
  time = time,
  mu0 = mu0,
  sigma0_l = sigma0_l,
  alpha = alpha,
  beta = beta,
  psi_l = psi_l,
  nu = nu,
  lambda = lambda,
  theta_l = theta_l,
  type = 0
)
```

```
plot(ssm)
plot(ssm, id = 1:3, time = 0:9)
```

```
# Type 1
ssm <- SimSSMFixed(
  n = n,
  time = time,
  mu0 = mu0,
  sigma0_l = sigma0_l,
  alpha = alpha,
  beta = beta,
  psi_l = psi_l,
  nu = nu,
  lambda = lambda,
  theta_l = theta_l,
  type = 1,
  x = x,
  gamma = gamma
)
```

```
plot(ssm)
plot(ssm, id = 1:3, time = 0:9)
```

```
# Type 2
ssm <- SimSSMFixed(
  n = n,
  time = time,
  mu0 = mu0,
  sigma0_l = sigma0_l,
  alpha = alpha,
  beta = beta,
  psi_l = psi_l,
  nu = nu,
  lambda = lambda,
  theta_l = theta_l,
)
```

```
    type = 2,  
    x = x,  
    gamma = gamma,  
    kappa = kappa  
  )  
  
plot(ssm)  
plot(ssm, id = 1:3, time = 0:9)
```

print.simstatespace *Print Method for an Object of Class simstatespace*

Description

Print Method for an Object of Class simstatespace

Usage

```
## S3 method for class 'simstatespace'  
print(x, ...)
```

Arguments

x	Object of Class simstatespace.
...	Additional arguments.

Value

Prints simulated data in long format.

Author(s)

Ivan Jacob Agaloos Pesigan

Examples

```
# prepare parameters  
set.seed(42)  
## number of individuals  
n <- 5  
## time points  
time <- 50  
## dynamic structure  
p <- 3  
mu0 <- rep(x = 0, times = p)  
sigma0 <- diag(p)  
sigma0_l <- t(chol(sigma0))  
alpha <- rep(x = 0, times = p)
```

```

beta <- 0.50 * diag(p)
psi <- diag(p)
psi_l <- t(chol(psi))
## measurement model
k <- 3
nu <- rep(x = 0, times = k)
lambda <- diag(k)
theta <- 0.50 * diag(k)
theta_l <- t(chol(theta))
## covariates
j <- 2
x <- lapply(
  X = seq_len(n),
  FUN = function(i) {
    matrix(
      data = stats::rnorm(n = time * j),
      nrow = j,
      ncol = time
    )
  }
)
gamma <- diag(x = 0.10, nrow = p, ncol = j)
kappa <- diag(x = 0.10, nrow = k, ncol = j)

# Type 0
ssm <- SimSSMFixed(
  n = n,
  time = time,
  mu0 = mu0,
  sigma0_l = sigma0_l,
  alpha = alpha,
  beta = beta,
  psi_l = psi_l,
  nu = nu,
  lambda = lambda,
  theta_l = theta_l,
  type = 0
)

print(ssm)

# Type 1
ssm <- SimSSMFixed(
  n = n,
  time = time,
  mu0 = mu0,
  sigma0_l = sigma0_l,
  alpha = alpha,
  beta = beta,
  psi_l = psi_l,
  nu = nu,
  lambda = lambda,
  theta_l = theta_l,

```

```

    type = 1,
    x = x,
    gamma = gamma
  )

  print(ssm)

# Type 2
ssm <- SimSSMFixed(
  n = n,
  time = time,
  mu0 = mu0,
  sigma0_l = sigma0_l,
  alpha = alpha,
  beta = beta,
  psi_l = psi_l,
  nu = nu,
  lambda = lambda,
  theta_l = theta_l,
  type = 2,
  x = x,
  gamma = gamma,
  kappa = kappa
)

print(ssm)

```

ProjectToHurwitz

Project Matrix to Hurwitz Stability

Description

Shifts a square matrix left on the real axis so that its spectral abscissa (maximum real part of the eigenvalues) is strictly less than $-\text{margin}$. This is useful for ensuring that continuous-time drift matrices (e.g. in linear SDEs/state-space models) are Hurwitz-stable. If the matrix already satisfies the margin, it is returned unchanged.

Usage

```
ProjectToHurwitz(x, margin = 0.001)
```

Arguments

<code>x</code>	Numeric square matrix.
<code>margin</code>	Positive numeric. Target buffer inside the Hurwitz region; the result satisfies $\max \Re\{\lambda_i(x^*)\} \leq -\text{margin}$ (default 1e-3).

Details

The projection is performed by subtracting a multiple of the identity:

$$x^* = x - (\alpha + \text{margin})I,$$

where $\alpha = \max \Re\{\lambda_i(x)\}$ is the spectral abscissa.

Value

A numeric matrix of the same dimensions as x, shifted if necessary to satisfy the Hurwitz stability constraint.

Author(s)

Ivan Jacob Agaloos Pesigan

See Also

Other Simulation of State Space Models Data Functions: [LinSDE2SSM\(\)](#), [LinSDECovEta\(\)](#), [LinSDECovY\(\)](#), [LinSDEInterceptEta\(\)](#), [LinSDEInterceptY\(\)](#), [LinSDEMeanEta\(\)](#), [LinSDEMeanY\(\)](#), [ProjectToStability\(\)](#), [SSMCovEta\(\)](#), [SSMCovY\(\)](#), [SSMInterceptEta\(\)](#), [SSMInterceptY\(\)](#), [SSMMeanEta\(\)](#), [SSMMeanY\(\)](#), [SimAlphaN\(\)](#), [SimBetaN\(\)](#), [SimBetaN2\(\)](#), [SimBetaNCovariate\(\)](#), [SimCovDiagN\(\)](#), [SimCovN\(\)](#), [SimIotaN\(\)](#), [SimMVN\(\)](#), [SimMuN\(\)](#), [SimNuN\(\)](#), [SimPhiN\(\)](#), [SimPhiN2\(\)](#), [SimPhiNCovariate\(\)](#), [SimSSMFixed\(\)](#), [SimSSMIVary\(\)](#), [SimSSMLinGrowth\(\)](#), [SimSSMLinGrowthIVary\(\)](#), [SimSSMLinSDEFixed\(\)](#), [SimSSMLinSDEIVary\(\)](#), [SimSSMOUFixed\(\)](#), [SimSSMOUIVary\(\)](#), [SimSSMVARFixed\(\)](#), [SimSSMVARIVary\(\)](#), [SpectralRadius\(\)](#), [TestPhi\(\)](#), [TestPhiHurwitz\(\)](#), [TestStability\(\)](#), [TestStationarity\(\)](#)

Examples

```
# Unstable (spectral abscissa >= 0):
x <- matrix(
  data = c(
    0.10, -0.40,
    0.50, 0.20
  ),
  nrow = 2
)
SpectralAbscissa(x = x) # >= 0
SpectralAbscissa(x = ProjectToHurwitz(x = x)) # <= -1e-3 (default margin)

# Already Hurwitz-stable is returned unchanged up to numerics:
x <- matrix(
  data = c(
    -0.50, -0.20,
    1.00, -0.30
  ),
  nrow = 2
)
SpectralAbscissa(x = x) # < 0
identical(ProjectToHurwitz(x = x), x)
```

ProjectToStability *Project Matrix to Stability*

Description

Scales a square matrix so that its spectral radius is strictly less than 1 by a specified stability margin. This is useful for ensuring that transition matrices in state space or vector autoregressive (VAR) models are stationary. If the matrix is already within the margin, it is returned unchanged.

Usage

```
ProjectToStability(x, margin = 0.98, tol = 1e-12)
```

Arguments

x	Numeric square matrix.
margin	Double in (0, 1). Target upper bound for the spectral radius (default = 0.98).
tol	Small positive double added to the denominator in the scaling factor to avoid division by zero (default 1e-12).

Details

The projection is performed by multiplying the matrix by a constant factor $c = \frac{\text{margin}}{\rho + \text{tol}}$, where ρ is the spectral radius and tol is a small positive number to prevent division by zero.

Value

A numeric matrix of the same dimensions as x, scaled if necessary to satisfy the stability constraint.

Author(s)

Ivan Jacob Agaloos Pesigan

See Also

Other Simulation of State Space Models Data Functions: [LinSDE2SSM\(\)](#), [LinSDECovEta\(\)](#), [LinSDECovY\(\)](#), [LinSDEInterceptEta\(\)](#), [LinSDEInterceptY\(\)](#), [LinSDEMeanEta\(\)](#), [LinSDEMeanY\(\)](#), [ProjectToHurwitz\(\)](#), [SSMCovEta\(\)](#), [SSMCovY\(\)](#), [SSMInterceptEta\(\)](#), [SSMInterceptY\(\)](#), [SSMMeanEta\(\)](#), [SSMMeanY\(\)](#), [SimAlphaN\(\)](#), [SimBetaN\(\)](#), [SimBetaN2\(\)](#), [SimBetaNCovariate\(\)](#), [SimCovDiagN\(\)](#), [SimCovN\(\)](#), [SimIotaN\(\)](#), [SimMVN\(\)](#), [SimMuN\(\)](#), [SimNuN\(\)](#), [SimPhiN\(\)](#), [SimPhiN2\(\)](#), [SimPhiNCovariate\(\)](#), [SimSSMFixed\(\)](#), [SimSSMIVary\(\)](#), [SimSSMLinGrowth\(\)](#), [SimSSMLinGrowthIVary\(\)](#), [SimSSMLinSDEFixed\(\)](#), [SimSSMLinSDEIVary\(\)](#), [SimSSMOUFixed\(\)](#), [SimSSMOUIVary\(\)](#), [SimSSMVARFixed\(\)](#), [SimSSMVARIVary\(\)](#), [SpectralRadius\(\)](#), [TestPhi\(\)](#), [TestPhiHurwitz\(\)](#), [TestStability\(\)](#), [TestStationarity\(\)](#)

Examples

```
# Matrix with eigenvalues greater than 1
x <- matrix(
  data = c(
    1.2, 0.3,
    0.4, 0.9
  ),
  nrow = 2
)
SpectralRadius(x = x) # > 1
SpectralRadius(x = ProjectToStability(x = x)) # < 1

# Matrix already stable is returned unchanged
x <- matrix(
  data = c(
    0.5, 0.3,
    0.2, 0.4
  ),
  nrow = 2
)
identical(ProjectToStability(x = x), x)
```

SimAlphaN

Simulate Intercept Vectors in a Discrete-Time Vector Autoregressive Model from the Multivariate Normal Distribution

Description

This function simulates random intercept vectors in a discrete-time vector autoregressive model from the multivariate normal distribution.

Usage

```
SimAlphaN(n, alpha, vcov_alpha_1)
```

Arguments

n	Positive integer. Number of replications.
alpha	Numeric vector. Intercept (α).
vcov_alpha_1	Numeric matrix. Cholesky factorization ($t(\text{chol}(\text{vcov_alpha}))$) of the sampling variance-covariance matrix of α .

Value

Returns a list of random intercept vectors.

Author(s)

Ivan Jacob Agaloos Pesigan

See Also

Other Simulation of State Space Models Data Functions: [LinSDE2SSM\(\)](#), [LinSDECovEta\(\)](#), [LinSDECovY\(\)](#), [LinSDEInterceptEta\(\)](#), [LinSDEInterceptY\(\)](#), [LinSDEMeanEta\(\)](#), [LinSDEMeanY\(\)](#), [ProjectToHurwitz\(\)](#), [ProjectToStability\(\)](#), [SSMCovEta\(\)](#), [SSMCovY\(\)](#), [SSMInterceptEta\(\)](#), [SSMInterceptY\(\)](#), [SSMMeanEta\(\)](#), [SSMMeanY\(\)](#), [SimBetaN\(\)](#), [SimBetaN2\(\)](#), [SimBetaNCovariate\(\)](#), [SimCovDiagN\(\)](#), [SimCovN\(\)](#), [SimIotaN\(\)](#), [SimMVN\(\)](#), [SimMuN\(\)](#), [SimNuN\(\)](#), [SimPhiN\(\)](#), [SimPhiN2\(\)](#), [SimPhiNCovariate\(\)](#), [SimSSMFixed\(\)](#), [SimSSMIVary\(\)](#), [SimSSMLinGrowth\(\)](#), [SimSSMLinGrowthIVary\(\)](#), [SimSSMLinSDEFixed\(\)](#), [SimSSMLinSDEIVary\(\)](#), [SimSSMOUFixed\(\)](#), [SimSSMOUIVary\(\)](#), [SimSSMVARFixed\(\)](#), [SimSSMVARIVary\(\)](#), [SpectralRadius\(\)](#), [TestPhi\(\)](#), [TestPhiHurwitz\(\)](#), [TestStability\(\)](#), [TestStationarity\(\)](#)

Examples

```
n <- 10
alpha <- c(0, 0, 0)
vcov_alpha_l <- t(chol(0.001 * diag(3)))
SimAlphaN(n = n, alpha = alpha, vcov_alpha_l = vcov_alpha_l)
```

SimBetaN

Simulate Transition Matrices from the Multivariate Normal Distribution

Description

This function simulates random transition matrices from the multivariate normal distribution. The function ensures that the generated transition matrices are stationary using [TestStationarity\(\)](#) with a rejection sampling approach.

Usage

```
SimBetaN(
  n,
  beta,
  vcov_beta_vec_l,
  margin = 1,
  beta_lbound = NULL,
  beta_ubound = NULL,
  bound = FALSE,
  max_iter = 100000L
)
```

Arguments

<code>n</code>	Positive integer. Number of replications.
<code>beta</code>	Numeric matrix. The transition matrix (β).
<code>vcov_beta_vec_1</code>	Numeric matrix. Cholesky factorization ($t(\text{chol}(\text{vcov_beta_vec}))$) of the sampling variance-covariance matrix of $\text{vec}(\beta)$.
<code>margin</code>	Numeric scalar specifying the stationarity threshold. Values less than 1 indicate stricter stationarity criteria.
<code>beta_lbound</code>	Optional numeric matrix of same dim as <code>beta</code> . Use NA for no lower bound.
<code>beta_ubound</code>	Optional numeric matrix of same dim as <code>beta</code> . Use NA for no upper bound.
<code>bound</code>	Logical; if TRUE, resample until all elements respect bounds (NA bounds ignored).
<code>max_iter</code>	Safety cap on resampling attempts per draw.

Value

Returns a list of random transition matrices.

Author(s)

Ivan Jacob Agaloos Pesigan

See Also

Other Simulation of State Space Models Data Functions: [LinSDE2SSM\(\)](#), [LinSDECovEta\(\)](#), [LinSDECovY\(\)](#), [LinSDEInterceptEta\(\)](#), [LinSDEInterceptY\(\)](#), [LinSDEMeanEta\(\)](#), [LinSDEMeanY\(\)](#), [ProjectToHurwitz\(\)](#), [ProjectToStability\(\)](#), [SSMCovEta\(\)](#), [SSMCovY\(\)](#), [SSMInterceptEta\(\)](#), [SSMInterceptY\(\)](#), [SSMMeanEta\(\)](#), [SSMMeanY\(\)](#), [SimAlphaN\(\)](#), [SimBetaN2\(\)](#), [SimBetaNCovariate\(\)](#), [SimCovDiagN\(\)](#), [SimCovN\(\)](#), [SimIotaN\(\)](#), [SimMVN\(\)](#), [SimMuN\(\)](#), [SimNuN\(\)](#), [SimPhiN\(\)](#), [SimPhiN2\(\)](#), [SimPhiNCovariate\(\)](#), [SimSSMFixed\(\)](#), [SimSSMIVary\(\)](#), [SimSSMLinGrowth\(\)](#), [SimSSMLinGrowthIVary\(\)](#), [SimSSMLinSDEFixed\(\)](#), [SimSSMLinSDEIVary\(\)](#), [SimSSMOUFixed\(\)](#), [SimSSMOUIVary\(\)](#), [SimSSMVARFixed\(\)](#), [SimSSMVARIVary\(\)](#), [SpectralRadius\(\)](#), [TestPhi\(\)](#), [TestPhiHurwitz\(\)](#), [TestStability\(\)](#), [TestStationarity\(\)](#)

Examples

```
n <- 10
beta <- matrix(
  data = c(
    0.7, 0.5, -0.1,
    0.0, 0.6, 0.4,
    0, 0, 0.5
  ),
  nrow = 3
)
vcov_beta_vec_1 <- t(chol(0.001 * diag(9)))
SimBetaN(n = n, beta = beta, vcov_beta_vec_1 = vcov_beta_vec_1)
```

SimBetaN2

Simulate Transition Matrices from the Multivariate Normal Distribution and Project to Stability

Description

This function simulates random transition matrices from the multivariate normal distribution then projects each draw to the stability region using [ProjectToStability\(\)](#).

Usage

```
SimBetaN2(n, beta, vcov_beta_vec_l, margin = 0.98, tol = 1e-12)
```

Arguments

n	Positive integer. Number of replications.
beta	Numeric matrix. The transition matrix (β).
vcov_beta_vec_l	Numeric matrix. Cholesky factorization ($t(chol(vcov_beta_vec)))$ of the sampling variance-covariance matrix of $vec(\beta)$.
margin	Double in $(0, 1)$. Target upper bound for the spectral radius (default = 0.98).
tol	Small positive double added to the denominator in the scaling factor to avoid division by zero (default = 1e-12).

Value

Returns a list of random transition matrices.

Author(s)

Ivan Jacob Agaloos Pesigan

See Also

Other Simulation of State Space Models Data Functions: [LinSDE2SSM\(\)](#), [LinSDECovEta\(\)](#), [LinSDECovY\(\)](#), [LinSDEInterceptEta\(\)](#), [LinSDEInterceptY\(\)](#), [LinSDEMeanEta\(\)](#), [LinSDEMeanY\(\)](#), [ProjectToHurwitz\(\)](#), [ProjectToStability\(\)](#), [SSMCovEta\(\)](#), [SSMCovY\(\)](#), [SSMInterceptEta\(\)](#), [SSMInterceptY\(\)](#), [SSMMeanEta\(\)](#), [SSMMeanY\(\)](#), [SimAlphaN\(\)](#), [SimBetaN\(\)](#), [SimBetaNCovariate\(\)](#), [SimCovDiagN\(\)](#), [SimCovN\(\)](#), [SimIotaN\(\)](#), [SimMVN\(\)](#), [SimMuN\(\)](#), [SimNuN\(\)](#), [SimPhiN\(\)](#), [SimPhiN2\(\)](#), [SimPhiNCovariate\(\)](#), [SimSSMFixed\(\)](#), [SimSSMIVary\(\)](#), [SimSSMLinGrowth\(\)](#), [SimSSMLinGrowthIVary\(\)](#), [SimSSMLinSDEFixed\(\)](#), [SimSSMLinSDEIVary\(\)](#), [SimSSMOUFixed\(\)](#), [SimSSMOUIVary\(\)](#), [SimSSMVARFixed\(\)](#), [SimSSMVARIVary\(\)](#), [SpectralRadius\(\)](#), [TestPhi\(\)](#), [TestPhiHurwitz\(\)](#), [TestStability\(\)](#), [TestStationarity\(\)](#)

Examples

```

n <- 10
beta <- matrix(
  data = c(
    0.7, 0.5, -0.1,
    0.0, 0.6, 0.4,
    0, 0, 0.5
  ),
  nrow = 3
)
vcov_beta_vec_l <- t(chol(0.001 * diag(9)))
SimBetaN2(n = n, beta = beta, vcov_beta_vec_l = vcov_beta_vec_l)

```

SimBetaNCovariate	<i>Simulate Transition Matrices with a Covariate from the Multivariate Normal Distribution</i>
-------------------	--

Description

This function simulates random transition matrices from a multivariate normal distribution, allowing the mean transition matrix to vary as a linear function of a covariate. The function ensures that the generated transition matrices are stationary using [TestStationarity\(\)](#) with a rejection sampling approach.

Usage

```

SimBetaNCovariate(
  n,
  beta0,
  vcov_beta_vec_l,
  beta1,
  x,
  margin = 1,
  beta_lbound = NULL,
  beta_ubound = NULL,
  bound = FALSE,
  max_iter = 100000L
)

```

Arguments

n	Positive integer. Number of replications.
beta0	Numeric matrix. Baseline transition matrix β_0 corresponding to $\mathbf{x} = \mathbf{0}$.
vcov_beta_vec_l	Numeric matrix. Cholesky factorization ($\mathbf{t}(\mathbf{chol}(\mathbf{vcov_beta_vec}))$) of the sampling variance-covariance matrix of $\text{vec}(\beta)$.

beta1	Numeric matrix. Matrix of covariate effects mapping $\mathbf{x}$ to $\text{vec}(\beta)$.
x	List of numeric vectors. Covariate values.
margin	Numeric scalar specifying the stationarity threshold. Values less than 1 indicate stricter stationarity criteria.
beta_lbound	Optional numeric matrix of same dim as beta. Use NA for no lower bound.
beta_ubound	Optional numeric matrix of same dim as beta. Use NA for no upper bound.
bound	Logical; if TRUE, resample until all elements respect bounds (NA bounds ignored).
max_iter	Safety cap on resampling attempts per draw.

Value

Returns a list of random transition matrices.

Author(s)

Ivan Jacob Agaloos Pesigan

See Also

Other Simulation of State Space Models Data Functions: [LinSDE2SSM\(\)](#), [LinSDECovEta\(\)](#), [LinSDECovY\(\)](#), [LinSDEInterceptEta\(\)](#), [LinSDEInterceptY\(\)](#), [LinSDEMeanEta\(\)](#), [LinSDEMeanY\(\)](#), [ProjectToHurwitz\(\)](#), [ProjectToStability\(\)](#), [SSMCovEta\(\)](#), [SSMCovY\(\)](#), [SSMInterceptEta\(\)](#), [SSMInterceptY\(\)](#), [SSMMeanEta\(\)](#), [SSMMeanY\(\)](#), [SimAlphaN\(\)](#), [SimBetaN\(\)](#), [SimBetaN2\(\)](#), [SimCovDiagN\(\)](#), [SimCovN\(\)](#), [SimIotaN\(\)](#), [SimMVN\(\)](#), [SimMuN\(\)](#), [SimNuN\(\)](#), [SimPhiN\(\)](#), [SimPhiN2\(\)](#), [SimPhiNCovariate\(\)](#), [SimSSMFixed\(\)](#), [SimSSMIVary\(\)](#), [SimSSMLinGrowth\(\)](#), [SimSSMLinGrowthIVary\(\)](#), [SimSSMLinSDEFixed\(\)](#), [SimSSMLinSDEIVary\(\)](#), [SimSSMOUFixed\(\)](#), [SimSSMOUIVary\(\)](#), [SimSSMVARFixed\(\)](#), [SimSSMVARIVary\(\)](#), [SpectralRadius\(\)](#), [TestPhi\(\)](#), [TestPhiHurwitz\(\)](#), [TestStability\(\)](#), [TestStationarity\(\)](#)

Examples

```
n <- 5
beta0 <- matrix(
  data = c(
    0.7, 0.5, -0.1,
    0.0, 0.6, 0.4,
    0, 0, 0.5
  ),
  nrow = 3
)
vcov_beta_vec_l <- t(chol(0.001 * diag(9)))
# One scalar covariate per replication
beta1 <- matrix(data = 0, nrow = 9, ncol = 1)
beta1[1, 1] <- 0.10 # x shifts beta[1,1]
x <- list(c(0), c(1), c(-1), c(0.5), c(2))

SimBetaNCovariate(
  n = n,
  beta0 = beta0,
```

```

vcov_beta_vec_l = vcov_beta_vec_l,
beta1 = beta1,
x = x
)

```

SimCovDiagN

Simulate Diagonal Covariance Matrices from the Multivariate Normal Distribution

Description

This function simulates random diagonal covariance matrices from the multivariate normal distribution. The function ensures that the generated covariance matrices are positive semi-definite.

Usage

```
SimCovDiagN(n, sigma_diag, vcov_sigma_diag_l)
```

Arguments

n Positive integer. Number of replications.

sigma_diag Numeric matrix. The covariance matrix (Σ).

vcov_sigma_diag_l Numeric matrix. Cholesky factorization ($t(\text{chol}(\text{vcov_sigma_vech}))$) of the sampling variance-covariance matrix of $\text{vech}(\Sigma)$.

Value

Returns a list of random diagonal covariance matrices.

Author(s)

Ivan Jacob Agaloos Pesigan

See Also

Other Simulation of State Space Models Data Functions: [LinSDE2SSM\(\)](#), [LinSDECovEta\(\)](#), [LinSDECovY\(\)](#), [LinSDEInterceptEta\(\)](#), [LinSDEInterceptY\(\)](#), [LinSDEMeanEta\(\)](#), [LinSDEMeanY\(\)](#), [ProjectToHurwitz\(\)](#), [ProjectToStability\(\)](#), [SSMCovEta\(\)](#), [SSMCovY\(\)](#), [SSMInterceptEta\(\)](#), [SSMInterceptY\(\)](#), [SSMMeanEta\(\)](#), [SSMMeanY\(\)](#), [SimAlphaN\(\)](#), [SimBetaN\(\)](#), [SimBetaN2\(\)](#), [SimBetaNCovariate\(\)](#), [SimCovN\(\)](#), [SimIotaN\(\)](#), [SimMVN\(\)](#), [SimMuN\(\)](#), [SimNuN\(\)](#), [SimPhiN\(\)](#), [SimPhiN2\(\)](#), [SimPhiNCovariate\(\)](#), [SimSSMFixed\(\)](#), [SimSSMIVary\(\)](#), [SimSSMLinGrowth\(\)](#), [SimSSMLinGrowthIVary\(\)](#), [SimSSMLinSDEFixed\(\)](#), [SimSSMLinSDEIVary\(\)](#), [SimSSMOUFixed\(\)](#), [SimSSMOUIVary\(\)](#), [SimSSMVARFixed\(\)](#), [SimSSMVARIVary\(\)](#), [SpectralRadius\(\)](#), [TestPhi\(\)](#), [TestPhiHurwitz\(\)](#), [TestStability\(\)](#), [TestStationarity\(\)](#)

Examples

```

n <- 10
sigma_diag <- c(1, 1, 1)
vcov_sigma_diag_l <- t(chol(0.001 * diag(3)))
SimCovDiagN(
  n = n,
  sigma_diag = sigma_diag,
  vcov_sigma_diag_l = vcov_sigma_diag_l
)

```

SimCovN

Simulate Covariance Matrices from the Multivariate Normal Distribution

Description

This function simulates random covariance matrices from the multivariate normal distribution. The function ensures that the generated covariance matrices are positive semi-definite.

Usage

```
SimCovN(n, sigma, vcov_sigma_vech_l)
```

Arguments

n	Positive integer. Number of replications.
sigma	Numeric matrix. The covariance matrix (Σ).
vcov_sigma_vech_l	Numeric matrix. Cholesky factorization ($t(chol(vcov_sigma_vech)))$ of the sampling variance-covariance matrix of $vech(\Sigma)$.

Value

Returns a list of random covariance matrices.

Author(s)

Ivan Jacob Agaloos Pesigan

See Also

Other Simulation of State Space Models Data Functions: [LinSDE2SSM\(\)](#), [LinSDECovEta\(\)](#), [LinSDECovY\(\)](#), [LinSDEInterceptEta\(\)](#), [LinSDEInterceptY\(\)](#), [LinSDEMeanEta\(\)](#), [LinSDEMeanY\(\)](#), [ProjectToHurwitz\(\)](#), [ProjectToStability\(\)](#), [SSMCovEta\(\)](#), [SSMCovY\(\)](#), [SSMInterceptEta\(\)](#), [SSMInterceptY\(\)](#), [SSMMeanEta\(\)](#), [SSMMeanY\(\)](#), [SimAlphaN\(\)](#), [SimBetaN\(\)](#), [SimBetaN2\(\)](#), [SimBetaNCovariate\(\)](#), [SimCovDiagN\(\)](#), [SimIotaN\(\)](#), [SimMVN\(\)](#), [SimMuN\(\)](#), [SimNuN\(\)](#), [SimPhiN\(\)](#), [SimPhiN2\(\)](#), [SimPhiNCovariate\(\)](#), [SimSSMFixed\(\)](#), [SimSSMIVary\(\)](#), [SimSSMLinGrowth\(\)](#), [SimSSMLinGrowthIVary\(\)](#), [SimSSMLinSDEFixed\(\)](#), [SimSSMLinSDEIVary\(\)](#), [SimSSMOUFixed\(\)](#), [SimSSMOUIVary\(\)](#), [SimSSMVARFixed\(\)](#), [SimSSMVARIVary\(\)](#), [SpectralRadius\(\)](#), [TestPhi\(\)](#), [TestPhiHurwitz\(\)](#), [TestStability\(\)](#), [TestStationarity\(\)](#)

Examples

```

n <- 10
sigma <- matrix(
  data = c(
    1.0, 0.5, 0.3,
    0.5, 1.0, 0.4,
    0.3, 0.4, 1.0
  ),
  nrow = 3
)
vcov_sigma_vech_1 <- t(
  chol(
    0.001 * diag(3 * (3 + 1) / 2)
  )
)
SimCovN(
  n = n,
  sigma = sigma,
  vcov_sigma_vech_1 = vcov_sigma_vech_1
)

```

SimIotaN

Simulate Intercept Vectors in a Continuous-Time Vector Autoregressive Model from the Multivariate Normal Distribution

Description

This function simulates random intercept vectors in a continuous-time vector autoregressive model from the multivariate normal distribution.

Usage

```
SimIotaN(n, iota, vcov_iota_1)
```

Arguments

<code>n</code>	Positive integer. Number of replications.
<code>iota</code>	Numeric vector. Intercept (ι).
<code>vcov_iota_1</code>	Numeric matrix. Cholesky factorization ($t(\text{chol}(\text{vcov_iota}))$) of the sampling variance-covariance matrix of ι .

Value

Returns a list of random intercept vectors.

Author(s)

Ivan Jacob Agaloos Pesigan

See Also

Other Simulation of State Space Models Data Functions: [LinSDE2SSM\(\)](#), [LinSDECovEta\(\)](#), [LinSDECovY\(\)](#), [LinSDEInterceptEta\(\)](#), [LinSDEInterceptY\(\)](#), [LinSDEMeanEta\(\)](#), [LinSDEMeanY\(\)](#), [ProjectToHurwitz\(\)](#), [ProjectToStability\(\)](#), [SSMCovEta\(\)](#), [SSMCovY\(\)](#), [SSMInterceptEta\(\)](#), [SSMInterceptY\(\)](#), [SSMMeanEta\(\)](#), [SSMMeanY\(\)](#), [SimAlphaN\(\)](#), [SimBetaN\(\)](#), [SimBetaN2\(\)](#), [SimBetaNCovariate\(\)](#), [SimCovDiagN\(\)](#), [SimCovN\(\)](#), [SimMVN\(\)](#), [SimMuN\(\)](#), [SimNuN\(\)](#), [SimPhiN\(\)](#), [SimPhiN2\(\)](#), [SimPhiNCovariate\(\)](#), [SimSSMFixed\(\)](#), [SimSSMIVary\(\)](#), [SimSSMLinGrowth\(\)](#), [SimSSMLinGrowthIVary\(\)](#), [SimSSMLinSDEFixed\(\)](#), [SimSSMLinSDEIVary\(\)](#), [SimSSMOUFixed\(\)](#), [SimSSMOUIVary\(\)](#), [SimSSMVARFixed\(\)](#), [SimSSMVARIVary\(\)](#), [SpectralRadius\(\)](#), [TestPhi\(\)](#), [TestPhiHurwitz\(\)](#), [TestStability\(\)](#), [TestStationarity\(\)](#)

Examples

```
n <- 10
iota <- c(0, 0, 0)
vcov_iota_l <- t(chol(0.001 * diag(3)))
SimIotaN(n = n, iota = iota, vcov_iota_l = vcov_iota_l)
```

SimMuN

Simulate Set Point Vectors from the Multivariate Normal Distribution

Description

This function simulates random set point vectors from the multivariate normal distribution.

Usage

```
SimMuN(n, mu, vcov_mu_l)
```

Arguments

n	Positive integer. Number of replications.
mu	Numeric vector. Set point (μ).
vcov_mu_l	Numeric matrix. Cholesky factorization ($t(chol(vcov_mu))$) of the sampling variance-covariance matrix of μ .

Value

Returns a list of random set point vectors.

Author(s)

Ivan Jacob Agaloos Pesigan

See Also

Other Simulation of State Space Models Data Functions: [LinSDE2SSM\(\)](#), [LinSDECovEta\(\)](#), [LinSDECovY\(\)](#), [LinSDEInterceptEta\(\)](#), [LinSDEInterceptY\(\)](#), [LinSDEMeanEta\(\)](#), [LinSDEMeanY\(\)](#), [ProjectToHurwitz\(\)](#), [ProjectToStability\(\)](#), [SSMCovEta\(\)](#), [SSMCovY\(\)](#), [SSMInterceptEta\(\)](#), [SSMInterceptY\(\)](#), [SSMMeanEta\(\)](#), [SSMMeanY\(\)](#), [SimAlphaN\(\)](#), [SimBetaN\(\)](#), [SimBetaN2\(\)](#), [SimBetaNCovariate\(\)](#), [SimCovDiagN\(\)](#), [SimCovN\(\)](#), [SimIotaN\(\)](#), [SimMVN\(\)](#), [SimNuN\(\)](#), [SimPhiN\(\)](#), [SimPhiN2\(\)](#), [SimPhiNCovariate\(\)](#), [SimSSMFixed\(\)](#), [SimSSMIVary\(\)](#), [SimSSMLinGrowth\(\)](#), [SimSSMLinGrowthIVary\(\)](#), [SimSSMLinSDEFixed\(\)](#), [SimSSMLinSDEIVary\(\)](#), [SimSSMOUFixed\(\)](#), [SimSSMOUIVary\(\)](#), [SimSSMVARFixed\(\)](#), [SimSSMVARIVary\(\)](#), [SpectralRadius\(\)](#), [TestPhi\(\)](#), [TestPhiHurwitz\(\)](#), [TestStability\(\)](#), [TestStationarity\(\)](#)

Examples

```
n <- 10
mu <- c(0, 0, 0)
vcov_mu_l <- t(chol(0.001 * diag(3)))
SimMuN(n = n, mu = mu, vcov_mu_l = vcov_mu_l)
```

SimMVN

Simulate Vectors from the Multivariate Normal Distribution

Description

This function simulates random vectors from the multivariate normal distribution.

Usage

```
SimMVN(n, mu, sigma_l)
```

Arguments

n	Positive integer. Number of replications.
mu	Numeric vector. Mean vector (μ).
sigma_l	Numeric matrix. Cholesky factorization ($t(chol(sigma))$) of the variance-covariance matrix Σ .

Value

Returns a list of random vectors.

Author(s)

Ivan Jacob Agaloos Pesigan

See Also

Other Simulation of State Space Models Data Functions: [LinSDE2SSM\(\)](#), [LinSDECovEta\(\)](#), [LinSDECovY\(\)](#), [LinSDEInterceptEta\(\)](#), [LinSDEInterceptY\(\)](#), [LinSDEMeanEta\(\)](#), [LinSDEMeanY\(\)](#), [ProjectToHurwitz\(\)](#), [ProjectToStability\(\)](#), [SSMCovEta\(\)](#), [SSMCovY\(\)](#), [SSMInterceptEta\(\)](#), [SSMInterceptY\(\)](#), [SSMMeanEta\(\)](#), [SSMMeanY\(\)](#), [SimAlphaN\(\)](#), [SimBetaN\(\)](#), [SimBetaN2\(\)](#), [SimBetaNCovariate\(\)](#), [SimCovDiagN\(\)](#), [SimCovN\(\)](#), [SimIotaN\(\)](#), [SimMuN\(\)](#), [SimNuN\(\)](#), [SimPhiN\(\)](#), [SimPhiN2\(\)](#), [SimPhiNCovariate\(\)](#), [SimSSMFixed\(\)](#), [SimSSMIVary\(\)](#), [SimSSMLinGrowth\(\)](#), [SimSSMLinGrowthIVary\(\)](#), [SimSSMLinSDEFixed\(\)](#), [SimSSMLinSDEIVary\(\)](#), [SimSSMOUFixed\(\)](#), [SimSSMOUIVary\(\)](#), [SimSSMVARFixed\(\)](#), [SimSSMVARIVary\(\)](#), [SpectralRadius\(\)](#), [TestPhi\(\)](#), [TestPhiHurwitz\(\)](#), [TestStability\(\)](#), [TestStationarity\(\)](#)

Examples

```
n <- 10
mu <- c(0, 0, 0)
sigma_l <- t(chol(0.001 * diag(3)))
SimMVN(n = n, mu = mu, sigma_l = sigma_l)
```

SimNuN

Simulate Intercept Vectors in a Discrete-Time Vector Autoregressive Model from the Multivariate Normal Distribution

Description

This function simulates random intercept vectors in a discrete-time vector autoregressive model from the multivariate normal distribution.

Usage

```
SimNuN(n, nu, vcov_nu_l)
```

Arguments

n	Positive integer. Number of replications.
nu	Numeric vector. Intercept (ν).
vcov_nu_l	Numeric matrix. Cholesky factorization ($t(chol(vcov_nu))$) of the sampling variance-covariance matrix of ν .

Value

Returns a list of random intercept vectors.

Author(s)

Ivan Jacob Agaloos Pesigan

See Also

Other Simulation of State Space Models Data Functions: [LinSDE2SSM\(\)](#), [LinSDECovEta\(\)](#), [LinSDECovY\(\)](#), [LinSDEInterceptEta\(\)](#), [LinSDEInterceptY\(\)](#), [LinSDEMeanEta\(\)](#), [LinSDEMeanY\(\)](#), [ProjectToHurwitz\(\)](#), [ProjectToStability\(\)](#), [SSMCovEta\(\)](#), [SSMCovY\(\)](#), [SSMInterceptEta\(\)](#), [SSMInterceptY\(\)](#), [SSMMeanEta\(\)](#), [SSMMeanY\(\)](#), [SimAlphaN\(\)](#), [SimBetaN\(\)](#), [SimBetaN2\(\)](#), [SimBetaNCovariate\(\)](#), [SimCovDiagN\(\)](#), [SimCovN\(\)](#), [SimIotaN\(\)](#), [SimMVN\(\)](#), [SimMuN\(\)](#), [SimPhiN\(\)](#), [SimPhiN2\(\)](#), [SimPhiNCovariate\(\)](#), [SimSSMFixed\(\)](#), [SimSSMIVary\(\)](#), [SimSSMLinGrowth\(\)](#), [SimSSMLinGrowthIVary\(\)](#), [SimSSMLinSDEFixed\(\)](#), [SimSSMLinSDEIVary\(\)](#), [SimSSMOUFixed\(\)](#), [SimSSMOUIVary\(\)](#), [SimSSMVARFixed\(\)](#), [SimSSMVARIVary\(\)](#), [SpectralRadius\(\)](#), [TestPhi\(\)](#), [TestPhiHurwitz\(\)](#), [TestStability\(\)](#), [TestStationarity\(\)](#)

Examples

```
n <- 10
nu <- c(0, 0, 0)
vcov_nu_l <- t(chol(0.001 * diag(3)))
SimNuN(n = n, nu = nu, vcov_nu_l = vcov_nu_l)
```

SimPhiN

Simulate Random Drift Matrices from the Multivariate Normal Distribution

Description

This function simulates random drift matrices from the multivariate normal distribution. The function ensures that the generated drift matrices are stable using [TestPhi\(\)](#).

Usage

```
SimPhiN(
  n,
  phi,
  vcov_phi_vec_l,
  margin = 0,
  auto_ubound = 0,
  phi_lbound = NULL,
  phi_ubound = NULL,
  bound = FALSE,
  max_iter = 100000L
)
```

Arguments

n	Positive integer. Number of replications.
phi	Numeric matrix. The drift matrix (Φ).
vcov_phi_vec_l	Numeric matrix. Cholesky factorization ($t(chol(vcov_phi_vec)))$ of the sampling variance-covariance matrix of $vec(\Phi)$.

margin	Numeric scalar specifying the stability threshold for the real part of the eigenvalues. The default 0.0 corresponds to the imaginary axis; values less than 0.0 enforce a stricter stability margin.
auto_ubound	Numeric scalar specifying the upper bound for the diagonal elements of Φ . Default is 0.0 , requiring all diagonal values to be ≤ 0 .
phi_lbound	Optional numeric matrix of same dim as phi. Use NA for no lower bound.
phi_ubound	Optional numeric matrix of same dim as phi. Use NA for no upper bound.
bound	Logical; if TRUE, resample until all elements respect bounds (NA bounds ignored).
max_iter	Safety cap on resampling attempts per draw.

Value

Returns a list of random drift matrices.

Author(s)

Ivan Jacob Agaloos Pesigan

See Also

Other Simulation of State Space Models Data Functions: [LinSDE2SSM\(\)](#), [LinSDECovEta\(\)](#), [LinSDECovY\(\)](#), [LinSDEInterceptEta\(\)](#), [LinSDEInterceptY\(\)](#), [LinSDEMeanEta\(\)](#), [LinSDEMeanY\(\)](#), [ProjectToHurwitz\(\)](#), [ProjectToStability\(\)](#), [SSMCovEta\(\)](#), [SSMCovY\(\)](#), [SSMInterceptEta\(\)](#), [SSMInterceptY\(\)](#), [SSMMeanEta\(\)](#), [SSMMeanY\(\)](#), [SimAlphaN\(\)](#), [SimBetaN\(\)](#), [SimBetaN2\(\)](#), [SimBetaNCovariate\(\)](#), [SimCovDiagN\(\)](#), [SimCovN\(\)](#), [SimIotaN\(\)](#), [SimMVN\(\)](#), [SimMuN\(\)](#), [SimNuN\(\)](#), [SimPhiN2\(\)](#), [SimPhiNCovariate\(\)](#), [SimSSMFixed\(\)](#), [SimSSMIVary\(\)](#), [SimSSMLinGrowth\(\)](#), [SimSSMLinGrowthIVary\(\)](#), [SimSSMLinSDEFixed\(\)](#), [SimSSMLinSDEIVary\(\)](#), [SimSSMOUFixed\(\)](#), [SimSSMOUIVary\(\)](#), [SimSSMVARFixed\(\)](#), [SimSSMVARIVary\(\)](#), [SpectralRadius\(\)](#), [TestPhi\(\)](#), [TestPhiHurwitz\(\)](#), [TestStability\(\)](#), [TestStationarity\(\)](#)

Examples

```
n <- 10
phi <- matrix(
  data = c(
    -0.357, 0.771, -0.450,
    0.0, -0.511, 0.729,
    0, 0, -0.693
  ),
  nrow = 3
)
vcov_phi_vec_l <- t(chol(0.001 * diag(9)))
SimPhiN(n = n, phi = phi, vcov_phi_vec_l = vcov_phi_vec_l)
```

SimPhiN2	<i>Simulate Random Drift Matrices from the Multivariate Normal Distribution and Project to Hurwitz</i>
----------	--

Description

This function simulates random drift matrices from the multivariate normal distribution then projects each draw to the Hurwitz-stable region using [ProjectToHurwitz\(\)](#).

Usage

```
SimPhiN2(n, phi, vcov_phi_vec_l, margin = 0.001)
```

Arguments

n	Positive integer. Number of replications.
phi	Numeric matrix. The drift matrix (Φ).
vcov_phi_vec_l	Numeric matrix. Cholesky factorization ($t(chol(vcov_phi_vec))$) of the sampling variance-covariance matrix of $vec(\Phi)$.
margin	Positive numeric. Target buffer so that the spectral abscissa is $\leq -margin$ (default $1e-3$).

Value

Returns a list of random drift matrices.

Author(s)

Ivan Jacob Agaloos Pesigan

See Also

Other Simulation of State Space Models Data Functions: [LinSDE2SSM\(\)](#), [LinSDECovEta\(\)](#), [LinSDECovY\(\)](#), [LinSDEInterceptEta\(\)](#), [LinSDEInterceptY\(\)](#), [LinSDEMeanEta\(\)](#), [LinSDEMeanY\(\)](#), [ProjectToHurwitz\(\)](#), [ProjectToStability\(\)](#), [SSMCovEta\(\)](#), [SSMCovY\(\)](#), [SSMInterceptEta\(\)](#), [SSMInterceptY\(\)](#), [SSMMeanEta\(\)](#), [SSMMeanY\(\)](#), [SimAlphaN\(\)](#), [SimBetaN\(\)](#), [SimBetaN2\(\)](#), [SimBetaNCovariate\(\)](#), [SimCovDiagN\(\)](#), [SimCovN\(\)](#), [SimIotaN\(\)](#), [SimMVN\(\)](#), [SimMuN\(\)](#), [SimNuN\(\)](#), [SimPhiN\(\)](#), [SimPhiNCovariate\(\)](#), [SimSSMFixed\(\)](#), [SimSSMIVary\(\)](#), [SimSSMLinGrowth\(\)](#), [SimSSMLinGrowthIVary\(\)](#), [SimSSMLinSDEFixed\(\)](#), [SimSSMLinSDEIVary\(\)](#), [SimSSMOUFixed\(\)](#), [SimSSMOUIVary\(\)](#), [SimSSMVARFixed\(\)](#), [SimSSMVARIVary\(\)](#), [SpectralRadius\(\)](#), [TestPhi\(\)](#), [TestPhiHurwitz\(\)](#), [TestStability\(\)](#), [TestStationarity\(\)](#)

Examples

```
n <- 10
phi <- matrix(
  data = c(
    -0.357, 0.771, -0.450,
    0.0, -0.511, 0.729,
    0, 0, -0.693
  ),
  nrow = 3
)
vcov_phi_vec_l <- t(chol(0.001 * diag(9)))
SimPhiN2(n = n, phi = phi, vcov_phi_vec_l = vcov_phi_vec_l)
```

SimPhiNCovariate

Simulate Random Drift Matrices with a Covariate from the Multivariate Normal Distribution

Description

This function simulates random drift matrices from the multivariate normal distribution, allowing the mean drift matrix to vary as a linear function of a covariate. The function ensures that the generated drift matrices are stable using [TestPhi\(\)](#).

Usage

```
SimPhiNCovariate(
  n,
  phi0,
  vcov_phi_vec_l,
  phi1,
  x,
  margin = 0,
  auto_ubound = 0,
  phi_lbound = NULL,
  phi_ubound = NULL,
  bound = FALSE,
  max_iter = 100000L
)
```

Arguments

n	Positive integer. Number of replications.
phi0	Numeric matrix. Baseline drift matrix (Φ_0).
vcov_phi_vec_l	Numeric matrix. Cholesky factorization ($t(chol(vcov_phi_vec))$) of the sampling variance-covariance matrix of $vec(\Phi)$.
phi1	Numeric matrix. Matrix of covariate effects mapping x to $vec(\Phi)$.

x	List of numeric vectors. Covariate values.
margin	Numeric scalar specifying the stability threshold for the real part of the eigenvalues. The default 0.0 corresponds to the imaginary axis; values less than 0.0 enforce a stricter stability margin.
auto_ubound	Numeric scalar specifying the upper bound for the diagonal elements of Φ . Default is 0.0, requiring all diagonal values to be ≤ 0 .
phi_lbound	Optional numeric matrix of same dim as phi. Use NA for no lower bound.
phi_ubound	Optional numeric matrix of same dim as phi. Use NA for no upper bound.
bound	Logical; if TRUE, resample until all elements respect bounds (NA bounds ignored).
max_iter	Safety cap on resampling attempts per draw.

Value

Returns a list of random drift matrices.

Author(s)

Ivan Jacob Agaloos Pesigan

See Also

Other Simulation of State Space Models Data Functions: [LinSDE2SSM\(\)](#), [LinSDECovEta\(\)](#), [LinSDECovY\(\)](#), [LinSDEInterceptEta\(\)](#), [LinSDEInterceptY\(\)](#), [LinSDEMeanEta\(\)](#), [LinSDEMeanY\(\)](#), [ProjectToHurwitz\(\)](#), [ProjectToStability\(\)](#), [SSMCovEta\(\)](#), [SSMCovY\(\)](#), [SSMInterceptEta\(\)](#), [SSMInterceptY\(\)](#), [SSMMeanEta\(\)](#), [SSMMeanY\(\)](#), [SimAlphaN\(\)](#), [SimBetaN\(\)](#), [SimBetaN2\(\)](#), [SimBetaNCovariate\(\)](#), [SimCovDiagN\(\)](#), [SimCovN\(\)](#), [SimIotaN\(\)](#), [SimMVN\(\)](#), [SimMuN\(\)](#), [SimNuN\(\)](#), [SimPhiN\(\)](#), [SimPhiN2\(\)](#), [SimSSMFixed\(\)](#), [SimSSMIVary\(\)](#), [SimSSMLinGrowth\(\)](#), [SimSSMLinGrowthIVary\(\)](#), [SimSSMLinSDEFixed\(\)](#), [SimSSMLinSDEIVary\(\)](#), [SimSSMOUFixed\(\)](#), [SimSSMOUIVary\(\)](#), [SimSSMVARFixed\(\)](#), [SimSSMVARIVary\(\)](#), [SpectralRadius\(\)](#), [TestPhi\(\)](#), [TestPhiHurwitz\(\)](#), [TestStability\(\)](#), [TestStationarity\(\)](#)

Examples

```
n <- 5
phi0 <- matrix(
  data = c(
    -0.357, 0.771, -0.450,
    0.0, -0.511, 0.729,
    0, 0, -0.693
  ),
  nrow = 3
)
vcov_phi_vec_l <- t(chol(0.001 * diag(9)))
# One scalar covariate per replication
phi1 <- matrix(data = 0, nrow = 9, ncol = 1)
phi1[1, 1] <- 0.10 # x shifts phi[1,1]
x <- list(c(0), c(1), c(-1), c(0.5), c(2))
SimPhiNCovariate(
  n = n,
```

```

    phi0 = phi0,
    vcov_phi_vec_l = vcov_phi_vec_l,
    phi1 = phi1,
    x = x
  )

```

SimSSMFixed

Simulate Data from a State Space Model (Fixed Parameters)

Description

This function simulates data using a state space model. It assumes that the parameters remain constant across individuals and over time.

Usage

```

SimSSMFixed(
  n,
  time,
  delta_t = 1,
  mu0,
  sigma0_l,
  alpha,
  beta,
  psi_l,
  nu,
  lambda,
  theta_l,
  type = 0,
  x = NULL,
  gamma = NULL,
  kappa = NULL
)

```

Arguments

n	Positive integer. Number of individuals.
time	Positive integer. Number of time points.
delta_t	Numeric. Time interval. The default value is 1.0 with an option to use a numeric value for the discretized state space model parameterization of the linear stochastic differential equation model.
mu0	Numeric vector. Mean of initial latent variable values ($\mu_{\eta 0}$).
sigma0_l	Numeric matrix. Cholesky factorization ($t(\text{chol}(\text{sigma0}))$) of the covariance matrix of initial latent variable values ($\Sigma_{\eta 0}$).
alpha	Numeric vector. Vector of constant values for the dynamic model (α).

beta	Numeric matrix. Transition matrix relating the values of the latent variables at the previous to the current time point (β).
psi_l	Numeric matrix. Cholesky factorization ($t(\text{chol}(\text{psi}))$) of the covariance matrix of the process noise (Ψ).
nu	Numeric vector. Vector of intercept values for the measurement model (ν).
lambda	Numeric matrix. Factor loading matrix linking the latent variables to the observed variables (Λ).
theta_l	Numeric matrix. Cholesky factorization ($t(\text{chol}(\text{theta}))$) of the covariance matrix of the measurement error (Θ).
type	Integer. State space model type. See Details for more information.
x	List. Each element of the list is a matrix of covariates for each individual i in n . The number of columns in each matrix should be equal to <code>time</code> .
gamma	Numeric matrix. Matrix linking the covariates to the latent variables at current time point (Γ).
kappa	Numeric matrix. Matrix linking the covariates to the observed variables at current time point (κ).

Details

Type 0:

The measurement model is given by

$$\mathbf{y}_{i,t} = \boldsymbol{\nu} + \boldsymbol{\Lambda}\boldsymbol{\eta}_{i,t} + \boldsymbol{\varepsilon}_{i,t}, \quad \text{with} \quad \boldsymbol{\varepsilon}_{i,t} \sim \mathcal{N}(\mathbf{0}, \boldsymbol{\Theta})$$

where $\mathbf{y}_{i,t}$, $\boldsymbol{\eta}_{i,t}$, and $\boldsymbol{\varepsilon}_{i,t}$ are random variables and $\boldsymbol{\nu}$, $\boldsymbol{\Lambda}$, and $\boldsymbol{\Theta}$ are model parameters. $\mathbf{y}_{i,t}$ represents a vector of observed random variables, $\boldsymbol{\eta}_{i,t}$ a vector of latent random variables, and $\boldsymbol{\varepsilon}_{i,t}$ a vector of random measurement errors, at time t and individual i . $\boldsymbol{\nu}$ denotes a vector of intercepts, $\boldsymbol{\Lambda}$ a matrix of factor loadings, and $\boldsymbol{\Theta}$ the covariance matrix of $\boldsymbol{\varepsilon}$.

An alternative representation of the measurement error is given by

$$\boldsymbol{\varepsilon}_{i,t} = \boldsymbol{\Theta}^{\frac{1}{2}} \mathbf{z}_{i,t}, \quad \text{with} \quad \mathbf{z}_{i,t} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$$

where $\mathbf{z}_{i,t}$ is a vector of independent standard normal random variables and $\left(\boldsymbol{\Theta}^{\frac{1}{2}}\right) \left(\boldsymbol{\Theta}^{\frac{1}{2}}\right)' = \boldsymbol{\Theta}$.

The dynamic structure is given by

$$\boldsymbol{\eta}_{i,t} = \boldsymbol{\alpha} + \boldsymbol{\beta}\boldsymbol{\eta}_{i,t-1} + \boldsymbol{\zeta}_{i,t}, \quad \text{with} \quad \boldsymbol{\zeta}_{i,t} \sim \mathcal{N}(\mathbf{0}, \boldsymbol{\Psi})$$

where $\boldsymbol{\eta}_{i,t}$, $\boldsymbol{\eta}_{i,t-1}$, and $\boldsymbol{\zeta}_{i,t}$ are random variables, and $\boldsymbol{\alpha}$, $\boldsymbol{\beta}$, and $\boldsymbol{\Psi}$ are model parameters. Here, $\boldsymbol{\eta}_{i,t}$ is a vector of latent variables at time t and individual i , $\boldsymbol{\eta}_{i,t-1}$ represents a vector of latent variables at time $t-1$ and individual i , and $\boldsymbol{\zeta}_{i,t}$ represents a vector of dynamic noise at time t and individual i . $\boldsymbol{\alpha}$ denotes a vector of intercepts, $\boldsymbol{\beta}$ a matrix of autoregression and cross regression coefficients, and $\boldsymbol{\Psi}$ the covariance matrix of $\boldsymbol{\zeta}_{i,t}$.

An alternative representation of the dynamic noise is given by

$$\boldsymbol{\zeta}_{i,t} = \boldsymbol{\Psi}^{\frac{1}{2}} \mathbf{z}_{i,t}, \quad \text{with} \quad \mathbf{z}_{i,t} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$$

where $\left(\boldsymbol{\Psi}^{\frac{1}{2}}\right) \left(\boldsymbol{\Psi}^{\frac{1}{2}}\right)' = \boldsymbol{\Psi}$.

Type 1:

The measurement model is given by

$$\mathbf{y}_{i,t} = \boldsymbol{\nu} + \boldsymbol{\Lambda}\boldsymbol{\eta}_{i,t} + \boldsymbol{\varepsilon}_{i,t}, \quad \text{with } \boldsymbol{\varepsilon}_{i,t} \sim \mathcal{N}(\mathbf{0}, \boldsymbol{\Theta}).$$

The dynamic structure is given by

$$\boldsymbol{\eta}_{i,t} = \boldsymbol{\alpha} + \beta\boldsymbol{\eta}_{i,t-1} + \boldsymbol{\Gamma}\mathbf{x}_{i,t} + \boldsymbol{\zeta}_{i,t}, \quad \text{with } \boldsymbol{\zeta}_{i,t} \sim \mathcal{N}(\mathbf{0}, \boldsymbol{\Psi})$$

where $\mathbf{x}_{i,t}$ represents a vector of covariates at time t and individual i , and $\boldsymbol{\Gamma}$ the coefficient matrix linking the covariates to the latent variables.

Type 2:

The measurement model is given by

$$\mathbf{y}_{i,t} = \boldsymbol{\nu} + \boldsymbol{\Lambda}\boldsymbol{\eta}_{i,t} + \boldsymbol{\kappa}\mathbf{x}_{i,t} + \boldsymbol{\varepsilon}_{i,t}, \quad \text{with } \boldsymbol{\varepsilon}_{i,t} \sim \mathcal{N}(\mathbf{0}, \boldsymbol{\Theta})$$

where $\boldsymbol{\kappa}$ represents the coefficient matrix linking the covariates to the observed variables.

The dynamic structure is given by

$$\boldsymbol{\eta}_{i,t} = \boldsymbol{\alpha} + \beta\boldsymbol{\eta}_{i,t-1} + \boldsymbol{\Gamma}\mathbf{x}_{i,t} + \boldsymbol{\zeta}_{i,t}, \quad \text{with } \boldsymbol{\zeta}_{i,t} \sim \mathcal{N}(\mathbf{0}, \boldsymbol{\Psi}).$$

Value

Returns an object of class `simstatespace` which is a list with the following elements:

- `call`: Function call.
- `args`: Function arguments.
- `data`: Generated data which is a list of length n . Each element of data is a list with the following elements:
 - `id`: A vector of ID numbers with length 1, where 1 is the value of the function argument time.
 - `time`: A vector time points of length 1.
 - `y`: A 1 by k matrix of values for the manifest variables.
 - `eta`: A 1 by p matrix of values for the latent variables.
 - `x`: A 1 by j matrix of values for the covariates (when covariates are included).
- `fun`: Function used.

Author(s)

Ivan Jacob Agaloos Pesigan

References

Chow, S.-M., Ho, M. R., Hamaker, E. L., & Dolan, C. V. (2010). Equivalence and differences between structural equation modeling and state-space modeling techniques. *Structural Equation Modeling: A Multidisciplinary Journal*, 17(2), 303-332. doi:10.1080/10705511003661553

See Also

Other Simulation of State Space Models Data Functions: [LinSDE2SSM\(\)](#), [LinSDECovEta\(\)](#), [LinSDECovY\(\)](#), [LinSDEInterceptEta\(\)](#), [LinSDEInterceptY\(\)](#), [LinSDEMeanEta\(\)](#), [LinSDEMeanY\(\)](#), [ProjectToHurwitz\(\)](#), [ProjectToStability\(\)](#), [SSMCovEta\(\)](#), [SSMCovY\(\)](#), [SSMInterceptEta\(\)](#), [SSMInterceptY\(\)](#), [SSMMeanEta\(\)](#), [SSMMeanY\(\)](#), [SimAlphaN\(\)](#), [SimBetaN\(\)](#), [SimBetaN2\(\)](#), [SimBetaNCovariate\(\)](#), [SimCovDiagN\(\)](#), [SimCovN\(\)](#), [SimIotaN\(\)](#), [SimMVN\(\)](#), [SimMuN\(\)](#), [SimNuN\(\)](#), [SimPhiN\(\)](#), [SimPhiN2\(\)](#), [SimPhiNCovariate\(\)](#), [SimSSMIVary\(\)](#), [SimSSMLinGrowth\(\)](#), [SimSSMLinGrowthIVary\(\)](#), [SimSSMLinSDEFixed\(\)](#), [SimSSMLinSDEIVary\(\)](#), [SimSSMOUFixed\(\)](#), [SimSSMOUIVary\(\)](#), [SimSSMVARFixed\(\)](#), [SimSSMVARIVary\(\)](#), [SpectralRadius\(\)](#), [TestPhi\(\)](#), [TestPhiHurwitz\(\)](#), [TestStability\(\)](#), [TestStationarity\(\)](#)

Examples

```
# prepare parameters
set.seed(42)
## number of individuals
n <- 5
## time points
time <- 50
## dynamic structure
p <- 3
mu0 <- rep(x = 0, times = p)
sigma0 <- 0.001 * diag(p)
sigma0_l <- t(chol(sigma0))
alpha <- rep(x = 0, times = p)
beta <- 0.50 * diag(p)
psi <- 0.001 * diag(p)
psi_l <- t(chol(psi))
## measurement model
k <- 3
nu <- rep(x = 0, times = k)
lambda <- diag(k)
theta <- 0.001 * diag(k)
theta_l <- t(chol(theta))
## covariates
j <- 2
x <- lapply(
  X = seq_len(n),
  FUN = function(i) {
    matrix(
      data = stats::rnorm(n = time * j),
      nrow = j,
      ncol = time
    )
  }
)
gamma <- diag(x = 0.10, nrow = p, ncol = j)
kappa <- diag(x = 0.10, nrow = k, ncol = j)

# Type 0
ssm <- SimSSMFixed(
  n = n,
```

```
    time = time,  
    mu0 = mu0,  
    sigma0_l = sigma0_l,  
    alpha = alpha,  
    beta = beta,  
    psi_l = psi_l,  
    nu = nu,  
    lambda = lambda,  
    theta_l = theta_l,  
    type = 0  
  )
```

```
plot(ssm)
```

```
# Type 1  
ssm <- SimSSMFixed(  
  n = n,  
  time = time,  
  mu0 = mu0,  
  sigma0_l = sigma0_l,  
  alpha = alpha,  
  beta = beta,  
  psi_l = psi_l,  
  nu = nu,  
  lambda = lambda,  
  theta_l = theta_l,  
  type = 1,  
  x = x,  
  gamma = gamma  
)
```

```
plot(ssm)
```

```
# Type 2  
ssm <- SimSSMFixed(  
  n = n,  
  time = time,  
  mu0 = mu0,  
  sigma0_l = sigma0_l,  
  alpha = alpha,  
  beta = beta,  
  psi_l = psi_l,  
  nu = nu,  
  lambda = lambda,  
  theta_l = theta_l,  
  type = 2,  
  x = x,  
  gamma = gamma,  
  kappa = kappa  
)
```

```
plot(ssm)
```

SimSSMIVary	<i>Simulate Data from a State Space Model (Individual-Varying Parameters)</i>
-------------	---

Description

This function simulates data using a state space model. It assumes that the parameters can vary across individuals.

Usage

```
SimSSMIVary(
  n,
  time,
  delta_t = 1,
  mu0,
  sigma0_l,
  alpha,
  beta,
  psi_l,
  nu,
  lambda,
  theta_l,
  type = 0,
  x = NULL,
  gamma = NULL,
  kappa = NULL
)
```

Arguments

n	Positive integer. Number of individuals.
time	Positive integer. Number of time points.
delta_t	Numeric. Time interval. The default value is 1.0 with an option to use a numeric value for the discretized state space model parameterization of the linear stochastic differential equation model.
mu0	List of numeric vectors. Each element of the list is the mean of initial latent variable values ($\mu_{\eta 0}$).
sigma0_l	List of numeric matrices. Each element of the list is the Cholesky factorization ($t(\text{chol}(\text{sigma0}))$) of the covariance matrix of initial latent variable values ($\Sigma_{\eta 0}$).
alpha	List of numeric vectors. Each element of the list is the vector of constant values for the dynamic model (α).
beta	List of numeric matrices. Each element of the list is the transition matrix relating the values of the latent variables at the previous to the current time point (β).

psi_1	List of numeric matrices. Each element of the list is the Cholesky factorization ($\mathbf{t}(\text{chol}(\boldsymbol{\Psi}))$) of the covariance matrix of the process noise ($\boldsymbol{\Psi}$).
nu	List of numeric vectors. Each element of the list is the vector of intercept values for the measurement model ($\boldsymbol{\nu}$).
lambda	List of numeric matrices. Each element of the list is the factor loading matrix linking the latent variables to the observed variables ($\boldsymbol{\Lambda}$).
theta_1	List of numeric matrices. Each element of the list is the Cholesky factorization ($\mathbf{t}(\text{chol}(\boldsymbol{\Theta}))$) of the covariance matrix of the measurement error ($\boldsymbol{\Theta}$).
type	Integer. State space model type. See Details in SimSSMFixed() for more information.
x	List. Each element of the list is a matrix of covariates for each individual i in n . The number of columns in each matrix should be equal to time.
gamma	List of numeric matrices. Each element of the list is the matrix linking the covariates to the latent variables at current time point ($\boldsymbol{\Gamma}$).
kappa	List of numeric matrices. Each element of the list is the matrix linking the covariates to the observed variables at current time point ($\boldsymbol{\kappa}$).

Details

Parameters can vary across individuals by providing a list of parameter values. If the length of any of the parameters (μ_0 , σ_0_1 , α , β , ψ_1 , ν , λ , θ_1 , γ , or κ) is less than n , the function will cycle through the available values.

Value

Returns an object of class `simstatespace` which is a list with the following elements:

- `call`: Function call.
- `args`: Function arguments.
- `data`: Generated data which is a list of length n . Each element of data is a list with the following elements:
 - `id`: A vector of ID numbers with length 1, where 1 is the value of the function argument time.
 - `time`: A vector time points of length 1.
 - `y`: A 1 by k matrix of values for the manifest variables.
 - `eta`: A 1 by p matrix of values for the latent variables.
 - `x`: A 1 by j matrix of values for the covariates (when covariates are included).
- `fun`: Function used.

Author(s)

Ivan Jacob Agaloos Pesigan

References

Chow, S.-M., Ho, M. R., Hamaker, E. L., & Dolan, C. V. (2010). Equivalence and differences between structural equation modeling and state-space modeling techniques. *Structural Equation Modeling: A Multidisciplinary Journal*, 17(2), 303-332. doi:10.1080/10705511003661553

See Also

Other Simulation of State Space Models Data Functions: [LinSDE2SSM\(\)](#), [LinSDECovEta\(\)](#), [LinSDECovY\(\)](#), [LinSDEInterceptEta\(\)](#), [LinSDEInterceptY\(\)](#), [LinSDEMeanEta\(\)](#), [LinSDEMeanY\(\)](#), [ProjectToHurwitz\(\)](#), [ProjectToStability\(\)](#), [SSMCovEta\(\)](#), [SSMCovY\(\)](#), [SSMInterceptEta\(\)](#), [SSMInterceptY\(\)](#), [SSMMeanEta\(\)](#), [SSMMeanY\(\)](#), [SimAlphaN\(\)](#), [SimBetaN\(\)](#), [SimBetaN2\(\)](#), [SimBetaNCovariate\(\)](#), [SimCovDiagN\(\)](#), [SimCovN\(\)](#), [SimIotaN\(\)](#), [SimMVN\(\)](#), [SimMuN\(\)](#), [SimNuN\(\)](#), [SimPhiN\(\)](#), [SimPhiN2\(\)](#), [SimPhiNCovariate\(\)](#), [SimSSMFixed\(\)](#), [SimSSMLinGrowth\(\)](#), [SimSSMLinGrowthIVary\(\)](#), [SimSSMLinSDEFixed\(\)](#), [SimSSMLinSDEIVary\(\)](#), [SimSSMOUFixed\(\)](#), [SimSSMOUIVary\(\)](#), [SimSSMVARFixed\(\)](#), [SimSSMVARIVary\(\)](#), [SpectralRadius\(\)](#), [TestPhi\(\)](#), [TestPhiHurwitz\(\)](#), [TestStability\(\)](#), [TestStationarity\(\)](#)

Examples

```
# prepare parameters
# In this example, beta varies across individuals.
set.seed(42)
## number of individuals
n <- 5
## time points
time <- 50
## dynamic structure
p <- 3
mu0 <- list(
  rep(x = 0, times = p)
)
sigma0 <- 0.001 * diag(p)
sigma0_l <- list(
  t(chol(sigma0))
)
alpha <- list(
  rep(x = 0, times = p)
)
beta <- list(
  0.1 * diag(p),
  0.2 * diag(p),
  0.3 * diag(p),
  0.4 * diag(p),
  0.5 * diag(p)
)
psi <- 0.001 * diag(p)
psi_l <- list(
  t(chol(psi))
)
## measurement model
k <- 3
nu <- list(
```

```

    rep(x = 0, times = k)
  )
  lambda <- list(
    diag(k)
  )
  theta <- 0.001 * diag(k)
  theta_l <- list(
    t(chol(theta))
  )
  ## covariates
  j <- 2
  x <- lapply(
    X = seq_len(n),
    FUN = function(i) {
      matrix(
        data = stats::rnorm(n = time * j),
        nrow = j,
        ncol = time
      )
    }
  )
  gamma <- list(
    diag(x = 0.10, nrow = p, ncol = j)
  )
  kappa <- list(
    diag(x = 0.10, nrow = k, ncol = j)
  )

# Type 0
ssm <- SimSSMIVary(
  n = n,
  time = time,
  mu0 = mu0,
  sigma0_l = sigma0_l,
  alpha = alpha,
  beta = beta,
  psi_l = psi_l,
  nu = nu,
  lambda = lambda,
  theta_l = theta_l,
  type = 0
)

plot(ssm)

# Type 1
ssm <- SimSSMIVary(
  n = n,
  time = time,
  mu0 = mu0,
  sigma0_l = sigma0_l,
  alpha = alpha,
  beta = beta,

```

```

    psi_l = psi_l,
    nu = nu,
    lambda = lambda,
    theta_l = theta_l,
    type = 1,
    x = x,
    gamma = gamma
)

plot(ssm)

# Type 2
ssm <- SimSSMIVary(
  n = n,
  time = time,
  mu0 = mu0,
  sigma0_l = sigma0_l,
  alpha = alpha,
  beta = beta,
  psi_l = psi_l,
  nu = nu,
  lambda = lambda,
  theta_l = theta_l,
  type = 2,
  x = x,
  gamma = gamma,
  kappa = kappa
)

plot(ssm)

```

SimSSMLinGrowth

Simulate Data from the Linear Growth Curve Model

Description

This function simulates data from the linear growth curve model.

Usage

```

SimSSMLinGrowth(
  n,
  time,
  mu0,
  sigma0_l,
  theta_l,
  type = 0,
  x = NULL,

```

```

    gamma = NULL,
    kappa = NULL
)

```

Arguments

n	Positive integer. Number of individuals.
time	Positive integer. Number of time points.
mu0	Numeric vector. A vector of length two. The first element is the mean of the intercept, and the second element is the mean of the slope.
sigma0_l	Numeric matrix. Cholesky factorization (<code>t(chol(sigma0))</code>) of the covariance matrix of the intercept and the slope.
theta_l	Numeric. Square root of the common measurement error variance.
type	Integer. State space model type. See Details for more information.
x	List. Each element of the list is a matrix of covariates for each individual i in n . The number of columns in each matrix should be equal to <code>time</code> .
gamma	Numeric matrix. Matrix linking the covariates to the latent variables at current time point (Γ).
kappa	Numeric matrix. Matrix linking the covariates to the observed variables at current time point (κ).

Details

Type 0:

The measurement model is given by

$$Y_{i,t} = \begin{pmatrix} 1 & 0 \end{pmatrix} \begin{pmatrix} \eta_{0i,t} \\ \eta_{1i,t} \end{pmatrix} + \varepsilon_{i,t}, \quad \text{with } \varepsilon_{i,t} \sim \mathcal{N}(0, \theta)$$

where $Y_{i,t}$, $\eta_{0i,t}$, $\eta_{1i,t}$, and $\varepsilon_{i,t}$ are random variables and θ is a model parameter. $Y_{i,t}$ is the observed random variable at time t and individual i , $\eta_{0i,t}$ (intercept) and $\eta_{1i,t}$ (slope) form a vector of latent random variables at time t and individual i , and $\varepsilon_{i,t}$ a vector of random measurement errors at time t and individual i . θ is the variance of ε .

The dynamic structure is given by

$$\begin{pmatrix} \eta_{0i,t} \\ \eta_{1i,t} \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix} \begin{pmatrix} \eta_{0i,t-1} \\ \eta_{1i,t-1} \end{pmatrix}.$$

The mean vector and covariance matrix of the intercept and slope are captured in the mean vector and covariance matrix of the initial condition given by

$$\mu_{\eta|0} = \begin{pmatrix} \mu_{\eta_0} \\ \mu_{\eta_1} \end{pmatrix} \quad \text{and,}$$

$$\Sigma_{\eta|0} = \begin{pmatrix} \sigma_{\eta_0}^2 & \sigma_{\eta_0, \eta_1} \\ \sigma_{\eta_1, \eta_0} & \sigma_{\eta_1}^2 \end{pmatrix}.$$

Type 1:

The measurement model is given by

$$Y_{i,t} = \begin{pmatrix} 1 & 0 \end{pmatrix} \begin{pmatrix} \eta_{0i,t} \\ \eta_{1i,t} \end{pmatrix} + \varepsilon_{i,t}, \quad \text{with } \varepsilon_{i,t} \sim \mathcal{N}(0, \theta).$$

The dynamic structure is given by

$$\begin{pmatrix} \eta_{0i,t} \\ \eta_{1i,t} \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix} \begin{pmatrix} \eta_{0i,t-1} \\ \eta_{1i,t-1} \end{pmatrix} + \mathbf{\Gamma} \mathbf{x}_{i,t}$$

where $\mathbf{x}_{i,t}$ represents a vector of covariates at time t and individual i , and $\mathbf{\Gamma}$ the coefficient matrix linking the covariates to the latent variables.

Type 2:

The measurement model is given by

$$Y_{i,t} = \begin{pmatrix} 1 & 0 \end{pmatrix} \begin{pmatrix} \eta_{0i,t} \\ \eta_{1i,t} \end{pmatrix} + \kappa \mathbf{x}_{i,t} + \varepsilon_{i,t}, \quad \text{with } \varepsilon_{i,t} \sim \mathcal{N}(0, \theta)$$

where κ represents the coefficient matrix linking the covariates to the observed variables.

The dynamic structure is given by

$$\begin{pmatrix} \eta_{0i,t} \\ \eta_{1i,t} \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix} \begin{pmatrix} \eta_{0i,t-1} \\ \eta_{1i,t-1} \end{pmatrix} + \mathbf{\Gamma} \mathbf{x}_{i,t}.$$

Value

Returns an object of class `simstatespace` which is a list with the following elements:

- `call`: Function call.
- `args`: Function arguments.
- `data`: Generated data which is a list of length n . Each element of data is a list with the following elements:
 - `id`: A vector of ID numbers with length 1, where 1 is the value of the function argument time.
 - `time`: A vector time points of length 1.
 - `y`: A 1 by k matrix of values for the manifest variables.
 - `eta`: A 1 by p matrix of values for the latent variables.
 - `x`: A 1 by j matrix of values for the covariates (when covariates are included).
- `fun`: Function used.

Author(s)

Ivan Jacob Agaloos Pesigan

References

Chow, S.-M., Ho, M. R., Hamaker, E. L., & Dolan, C. V. (2010). Equivalence and differences between structural equation modeling and state-space modeling techniques. *Structural Equation Modeling: A Multidisciplinary Journal*, 17(2), 303-332. doi:10.1080/10705511003661553

See Also

Other Simulation of State Space Models Data Functions: [LinSDE2SSM\(\)](#), [LinSDECovEta\(\)](#), [LinSDECovY\(\)](#), [LinSDEInterceptEta\(\)](#), [LinSDEInterceptY\(\)](#), [LinSDEMeanEta\(\)](#), [LinSDEMeanY\(\)](#), [ProjectToHurwitz\(\)](#), [ProjectToStability\(\)](#), [SSMCovEta\(\)](#), [SSMCovY\(\)](#), [SSMInterceptEta\(\)](#), [SSMInterceptY\(\)](#), [SSMMeanEta\(\)](#), [SSMMeanY\(\)](#), [SimAlphaN\(\)](#), [SimBetaN\(\)](#), [SimBetaN2\(\)](#), [SimBetaNCovariate\(\)](#), [SimCovDiagN\(\)](#), [SimCovN\(\)](#), [SimIotaN\(\)](#), [SimMVN\(\)](#), [SimMuN\(\)](#), [SimNuN\(\)](#), [SimPhiN\(\)](#), [SimPhiN2\(\)](#), [SimPhiNCovariate\(\)](#), [SimSSMFixed\(\)](#), [SimSSMIVary\(\)](#), [SimSSMLinGrowthIVary\(\)](#), [SimSSMLinSDEFixed\(\)](#), [SimSSMLinSDEIVary\(\)](#), [SimSSMOUFixed\(\)](#), [SimSSMOUIVary\(\)](#), [SimSSMVARFixed\(\)](#), [SimSSMVARIVary\(\)](#), [SpectralRadius\(\)](#), [TestPhi\(\)](#), [TestPhiHurwitz\(\)](#), [TestStability\(\)](#), [TestStationarity\(\)](#)

Examples

```
# prepare parameters
set.seed(42)
## number of individuals
n <- 5
## time points
time <- 5
## dynamic structure
p <- 2
mu0 <- c(0.615, 1.006)
sigma0 <- matrix(
  data = c(
    1.932,
    0.618,
    0.618,
    0.587
  ),
  nrow = p
)
sigma0_l <- t(chol(sigma0))
## measurement model
k <- 1
theta <- 0.50
theta_l <- sqrt(theta)
## covariates
j <- 2
x <- lapply(
  X = seq_len(n),
  FUN = function(i) {
    matrix(
      data = rnorm(n = j * time),
      nrow = j
    )
  }
)
gamma <- diag(x = 0.10, nrow = p, ncol = j)
kappa <- diag(x = 0.10, nrow = k, ncol = j)

# Type 0
ssm <- SimSSMLinGrowth(
```

```

    n = n,
    time = time,
    mu0 = mu0,
    sigma0_l = sigma0_l,
    theta_l = theta_l,
    type = 0
  )

plot(ssm)

# Type 1
ssm <- SimSSMLinGrowth(
  n = n,
  time = time,
  mu0 = mu0,
  sigma0_l = sigma0_l,
  theta_l = theta_l,
  type = 1,
  x = x,
  gamma = gamma
)

plot(ssm)

# Type 2
ssm <- SimSSMLinGrowth(
  n = n,
  time = time,
  mu0 = mu0,
  sigma0_l = sigma0_l,
  theta_l = theta_l,
  type = 2,
  x = x,
  gamma = gamma,
  kappa = kappa
)

plot(ssm)

```

SimSSMLinGrowthIVary	<i>Simulate Data from the Linear Growth Curve Model (Individual-Varying Parameters)</i>
----------------------	---

Description

This function simulates data from the linear growth curve model. It assumes that the parameters can vary across individuals.

Usage

```
SimSSMLinGrowthIVary(
  n,
  time,
  mu0,
  sigma0_l,
  theta_l,
  type = 0,
  x = NULL,
  gamma = NULL,
  kappa = NULL
)
```

Arguments

n	Positive integer. Number of individuals.
time	Positive integer. Number of time points.
mu0	A list of numeric vectors. Each element of the list is a vector of length two. The first element is the mean of the intercept, and the second element is the mean of the slope.
sigma0_l	A list of numeric matrices. Each element of the list is the Cholesky factorization ($t(\text{chol}(\text{sigma0}))$) of the covariance matrix of the intercept and the slope.
theta_l	A list numeric values. Each element of the list is the square root of the common measurement error variance.
type	Integer. State space model type. See Details in SimSSMLinGrowth() for more information.
x	List. Each element of the list is a matrix of covariates for each individual i in n . The number of columns in each matrix should be equal to <code>time</code> .
gamma	List of numeric matrices. Each element of the list is the matrix linking the covariates to the latent variables at current time point (Γ).
kappa	List of numeric matrices. Each element of the list is the matrix linking the covariates to the observed variables at current time point (κ).

Details

Parameters can vary across individuals by providing a list of parameter values. If the length of any of the parameters (`mu0`, `sigma0`, `mu`, `theta_l`, `gamma`, or `kappa`) is less than `n`, the function will cycle through the available values.

Value

Returns an object of class `simstatespace` which is a list with the following elements:

- `call`: Function call.
- `args`: Function arguments.

- data: Generated data which is a list of length n. Each element of data is a list with the following elements:
 - id: A vector of ID numbers with length 1, where 1 is the value of the function argument time.
 - time: A vector time points of length 1.
 - y: A 1 by k matrix of values for the manifest variables.
 - eta: A 1 by p matrix of values for the latent variables.
 - x: A 1 by j matrix of values for the covariates (when covariates are included).
- fun: Function used.

Author(s)

Ivan Jacob Agaloos Pesigan

References

Chow, S.-M., Ho, M. R., Hamaker, E. L., & Dolan, C. V. (2010). Equivalence and differences between structural equation modeling and state-space modeling techniques. *Structural Equation Modeling: A Multidisciplinary Journal*, 17(2), 303-332. doi:10.1080/10705511003661553

See Also

Other Simulation of State Space Models Data Functions: [LinSDE2SSM\(\)](#), [LinSDECovEta\(\)](#), [LinSDECovY\(\)](#), [LinSDEInterceptEta\(\)](#), [LinSDEInterceptY\(\)](#), [LinSDEMeanEta\(\)](#), [LinSDEMeanY\(\)](#), [ProjectToHurwitz\(\)](#), [ProjectToStability\(\)](#), [SSMCovEta\(\)](#), [SSMCovY\(\)](#), [SSMInterceptEta\(\)](#), [SSMInterceptY\(\)](#), [SSMMeanEta\(\)](#), [SSMMeanY\(\)](#), [SimAlphaN\(\)](#), [SimBetaN\(\)](#), [SimBetaN2\(\)](#), [SimBetaNCovariate\(\)](#), [SimCovDiagN\(\)](#), [SimCovN\(\)](#), [SimIotaN\(\)](#), [SimMVN\(\)](#), [SimMuN\(\)](#), [SimNuN\(\)](#), [SimPhiN\(\)](#), [SimPhiN2\(\)](#), [SimPhiNCovariate\(\)](#), [SimSSMFixed\(\)](#), [SimSSMIVary\(\)](#), [SimSSMLinGrowth\(\)](#), [SimSSMLinSDEFixed\(\)](#), [SimSSMLinSDEIVary\(\)](#), [SimSSMOUFixed\(\)](#), [SimSSMOUIVary\(\)](#), [SimSSMVARFixed\(\)](#), [SimSSMVARIVary\(\)](#), [SpectralRadius\(\)](#), [TestPhi\(\)](#), [TestPhiHurwitz\(\)](#), [TestStability\(\)](#), [TestStationarity\(\)](#)

Examples

```
# prepare parameters
# In this example, the mean vector of the intercept and slope vary.
# Specifically,
# there are two sets of values representing two latent classes.
set.seed(42)
## number of individuals
n <- 10
## time points
time <- 5
## dynamic structure
p <- 2
mu0_1 <- c(0.615, 1.006) # lower starting point, higher growth
mu0_2 <- c(1.000, 0.500) # higher starting point, lower growth
mu0 <- list(mu0_1, mu0_2)
sigma0 <- matrix(
  data = c(
    1.932,
```

```

      0.618,
      0.618,
      0.587
    ),
    nrow = p
  )
  sigma0_l <- list(t(chol(sigma0)))
  ## measurement model
  k <- 1
  theta <- 0.50
  theta_l <- list(sqrt(theta))
  ## covariates
  j <- 2
  x <- lapply(
    X = seq_len(n),
    FUN = function(i) {
      matrix(
        data = stats::rnorm(n = time * j),
        nrow = j,
        ncol = time
      )
    }
  )
  gamma <- list(
    diag(x = 0.10, nrow = p, ncol = j)
  )
  kappa <- list(
    diag(x = 0.10, nrow = k, ncol = j)
  )

  # Type 0
  ssm <- SimSSMLinGrowthIVary(
    n = n,
    time = time,
    mu0 = mu0,
    sigma0_l = sigma0_l,
    theta_l = theta_l,
    type = 0
  )

  plot(ssm)

  # Type 1
  ssm <- SimSSMLinGrowthIVary(
    n = n,
    time = time,
    mu0 = mu0,
    sigma0_l = sigma0_l,
    theta_l = theta_l,
    type = 1,
    x = x,
    gamma = gamma
  )

```

```

plot(ssm)

# Type 2
ssm <- SimSSMLinGrowthIVary(
  n = n,
  time = time,
  mu0 = mu0,
  sigma0_l = sigma0_l,
  theta_l = theta_l,
  type = 2,
  x = x,
  gamma = gamma,
  kappa = kappa
)

plot(ssm)

```

SimSSMLinSDEFixed

*Simulate Data from the Linear Stochastic Differential Equation Model
using a State Space Model Parameterization (Fixed Parameters)*

Description

This function simulates data from the linear stochastic differential equation model using a state space model parameterization. It assumes that the parameters remain constant across individuals and over time.

Usage

```

SimSSMLinSDEFixed(
  n,
  time,
  delta_t = 1,
  mu0,
  sigma0_l,
  iota,
  phi,
  sigma_l,
  nu,
  lambda,
  theta_l,
  type = 0,
  x = NULL,
  gamma = NULL,
  kappa = NULL
)

```

Arguments

n	Positive integer. Number of individuals.
time	Positive integer. Number of time points.
delta_t	Numeric. Time interval (Δ_t).
mu0	Numeric vector. Mean of initial latent variable values ($\mu_{\eta 0}$).
sigma0_l	Numeric matrix. Cholesky factorization ($t(\text{chol}(\text{sigma0}))$) of the covariance matrix of initial latent variable values ($\Sigma_{\eta 0}$).
iota	Numeric vector. An unobserved term that is constant over time (ι).
phi	Numeric matrix. The drift matrix which represents the rate of change of the solution in the absence of any random fluctuations (Φ).
sigma_l	Numeric matrix. Cholesky factorization ($t(\text{chol}(\text{sigma}))$) of the covariance matrix of volatility or randomness in the process (Σ).
nu	Numeric vector. Vector of intercept values for the measurement model (ν).
lambda	Numeric matrix. Factor loading matrix linking the latent variables to the observed variables (Λ).
theta_l	Numeric matrix. Cholesky factorization ($t(\text{chol}(\text{theta}))$) of the covariance matrix of the measurement error (Θ).
type	Integer. State space model type. See Details for more information.
x	List. Each element of the list is a matrix of covariates for each individual i in n . The number of columns in each matrix should be equal to <code>time</code> .
gamma	Numeric matrix. Matrix linking the covariates to the latent variables at current time point (Γ).
kappa	Numeric matrix. Matrix linking the covariates to the observed variables at current time point (κ).

Details**Type 0:**

The measurement model is given by

$$\mathbf{y}_{i,t} = \boldsymbol{\nu} + \boldsymbol{\Lambda}\boldsymbol{\eta}_{i,t} + \boldsymbol{\varepsilon}_{i,t}, \quad \text{with} \quad \boldsymbol{\varepsilon}_{i,t} \sim \mathcal{N}(\mathbf{0}, \boldsymbol{\Theta})$$

where $\mathbf{y}_{i,t}$, $\boldsymbol{\eta}_{i,t}$, and $\boldsymbol{\varepsilon}_{i,t}$ are random variables and $\boldsymbol{\nu}$, $\boldsymbol{\Lambda}$, and $\boldsymbol{\Theta}$ are model parameters. $\mathbf{y}_{i,t}$ represents a vector of observed random variables, $\boldsymbol{\eta}_{i,t}$ a vector of latent random variables, and $\boldsymbol{\varepsilon}_{i,t}$ a vector of random measurement errors, at time t and individual i . $\boldsymbol{\nu}$ denotes a vector of intercepts, $\boldsymbol{\Lambda}$ a matrix of factor loadings, and $\boldsymbol{\Theta}$ the covariance matrix of $\boldsymbol{\varepsilon}$.

An alternative representation of the measurement error is given by

$$\boldsymbol{\varepsilon}_{i,t} = \boldsymbol{\Theta}^{\frac{1}{2}} \mathbf{z}_{i,t}, \quad \text{with} \quad \mathbf{z}_{i,t} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$$

where $\mathbf{z}_{i,t}$ is a vector of independent standard normal random variables and $\left(\boldsymbol{\Theta}^{\frac{1}{2}}\right) \left(\boldsymbol{\Theta}^{\frac{1}{2}}\right)' = \boldsymbol{\Theta}$.

The dynamic structure is given by

$$d\boldsymbol{\eta}_{i,t} = (\boldsymbol{\iota} + \boldsymbol{\Phi}\boldsymbol{\eta}_{i,t}) dt + \boldsymbol{\Sigma}^{\frac{1}{2}} d\mathbf{W}_{i,t}$$

where $\boldsymbol{\nu}$ is a term which is unobserved and constant over time, $\boldsymbol{\Phi}$ is the drift matrix which represents the rate of change of the solution in the absence of any random fluctuations, $\boldsymbol{\Sigma}$ is the matrix of volatility or randomness in the process, and $d\mathbf{W}$ is a Wiener process or Brownian motion, which represents random fluctuations.

Type 1:

The measurement model is given by

$$\mathbf{y}_{i,t} = \boldsymbol{\nu} + \boldsymbol{\Lambda}\boldsymbol{\eta}_{i,t} + \boldsymbol{\varepsilon}_{i,t}, \quad \text{with} \quad \boldsymbol{\varepsilon}_{i,t} \sim \mathcal{N}(\mathbf{0}, \boldsymbol{\Theta}).$$

The dynamic structure is given by

$$d\boldsymbol{\eta}_{i,t} = (\boldsymbol{\nu} + \boldsymbol{\Phi}\boldsymbol{\eta}_{i,t}) dt + \boldsymbol{\Gamma}\mathbf{x}_{i,t} + \boldsymbol{\Sigma}^{\frac{1}{2}}d\mathbf{W}_{i,t}$$

where $\mathbf{x}_{i,t}$ represents a vector of covariates at time t and individual i , and $\boldsymbol{\Gamma}$ the coefficient matrix linking the covariates to the latent variables.

Type 2:

The measurement model is given by

$$\mathbf{y}_{i,t} = \boldsymbol{\nu} + \boldsymbol{\Lambda}\boldsymbol{\eta}_{i,t} + \boldsymbol{\kappa}\mathbf{x}_{i,t} + \boldsymbol{\varepsilon}_{i,t}, \quad \text{with} \quad \boldsymbol{\varepsilon}_{i,t} \sim \mathcal{N}(\mathbf{0}, \boldsymbol{\Theta})$$

where $\boldsymbol{\kappa}$ represents the coefficient matrix linking the covariates to the observed variables.

The dynamic structure is given by

$$d\boldsymbol{\eta}_{i,t} = (\boldsymbol{\nu} + \boldsymbol{\Phi}\boldsymbol{\eta}_{i,t}) dt + \boldsymbol{\Gamma}\mathbf{x}_{i,t} + \boldsymbol{\Sigma}^{\frac{1}{2}}d\mathbf{W}_{i,t}.$$

State Space Parameterization:

The state space parameters as a function of the linear stochastic differential equation model parameters are given by

$$\boldsymbol{\beta}_{\Delta t_{l_i}} = \exp(\Delta t \boldsymbol{\Phi})$$

$$\boldsymbol{\alpha}_{\Delta t_{l_i}} = \boldsymbol{\Phi}^{-1} (\boldsymbol{\beta} - \mathbf{I}_p) \boldsymbol{\nu}$$

$$\text{vec}(\boldsymbol{\Psi}_{\Delta t_{l_i}}) = [(\boldsymbol{\Phi} \otimes \mathbf{I}_p) + (\mathbf{I}_p \otimes \boldsymbol{\Phi})] [\exp([(\boldsymbol{\Phi} \otimes \mathbf{I}_p) + (\mathbf{I}_p \otimes \boldsymbol{\Phi})] \Delta t) - \mathbf{I}_{p \times p}] \text{vec}(\boldsymbol{\Sigma})$$

where p is the number of latent variables and Δt is the time interval.

Value

Returns an object of class `simstatespace` which is a list with the following elements:

- `call`: Function call.
- `args`: Function arguments.
- `data`: Generated data which is a list of length n . Each element of data is a list with the following elements:
 - `id`: A vector of ID numbers with length 1, where 1 is the value of the function argument time.

- time: A vector time points of length 1.
- y: A 1 by k matrix of values for the manifest variables.
- eta: A 1 by p matrix of values for the latent variables.
- x: A 1 by j matrix of values for the covariates (when covariates are included).
- fun: Function used.

Author(s)

Ivan Jacob Agaloos Pesigan

References

- Chow, S.-M., Ho, M. R., Hamaker, E. L., & Dolan, C. V. (2010). Equivalence and differences between structural equation modeling and state-space modeling techniques. *Structural Equation Modeling: A Multidisciplinary Journal*, 17(2), 303-332. doi:10.1080/10705511003661553
- Chow, S.-M., Losardo, D., Park, J., & Molenaar, P. C. M. (2023). Continuous-time dynamic models: Connections to structural equation models and other discrete-time models. In R. H. Hoyle (Ed.), *Handbook of structural equation modeling* (2nd ed.). The Guilford Press.
- Harvey, A. C. (1990). *Forecasting, structural time series models and the Kalman filter*. Cambridge University Press. doi:10.1017/cbo9781107049994

See Also

Other Simulation of State Space Models Data Functions: [LinSDE2SSM\(\)](#), [LinSDECovEta\(\)](#), [LinSDECovY\(\)](#), [LinSDEInterceptEta\(\)](#), [LinSDEInterceptY\(\)](#), [LinSDEMeanEta\(\)](#), [LinSDEMeanY\(\)](#), [ProjectToHurwitz\(\)](#), [ProjectToStability\(\)](#), [SSMCovEta\(\)](#), [SSMCovY\(\)](#), [SSMInterceptEta\(\)](#), [SSMInterceptY\(\)](#), [SSMMeanEta\(\)](#), [SSMMeanY\(\)](#), [SimAlphaN\(\)](#), [SimBetaN\(\)](#), [SimBetaN2\(\)](#), [SimBetaNCovariate\(\)](#), [SimCovDiagN\(\)](#), [SimCovN\(\)](#), [SimIotaN\(\)](#), [SimMVN\(\)](#), [SimMuN\(\)](#), [SimNuN\(\)](#), [SimPhiN\(\)](#), [SimPhiN2\(\)](#), [SimPhiNCovariate\(\)](#), [SimSSMFixed\(\)](#), [SimSSMIVary\(\)](#), [SimSSMLinGrowth\(\)](#), [SimSSMLinGrowthIVary\(\)](#), [SimSSMLinSDEIVary\(\)](#), [SimSSMOUFixed\(\)](#), [SimSSMOUIVary\(\)](#), [SimSSMVARFixed\(\)](#), [SimSSMVARIVary\(\)](#), [SpectralRadius\(\)](#), [TestPhi\(\)](#), [TestPhiHurwitz\(\)](#), [TestStability\(\)](#), [TestStationarity\(\)](#)

Examples

```
# prepare parameters
set.seed(42)
## number of individuals
n <- 5
## time points
time <- 50
delta_t <- 0.10
## dynamic structure
p <- 2
mu0 <- c(-3.0, 1.5)
sigma0 <- 0.001 * diag(p)
sigma0_l <- t(chol(sigma0))
iota <- c(0.317, 0.230)
phi <- matrix(
  data = c(
```

```

      -0.10,
      0.05,
      0.05,
      -0.10
    ),
    nrow = p
  )
  sigma <- matrix(
    data = c(
      2.79,
      0.06,
      0.06,
      3.27
    ),
    nrow = p
  )
  sigma_l <- t(chol(sigma))
  ## measurement model
  k <- 2
  nu <- rep(x = 0, times = k)
  lambda <- diag(k)
  theta <- 0.001 * diag(k)
  theta_l <- t(chol(theta))
  ## covariates
  j <- 2
  x <- lapply(
    X = seq_len(n),
    FUN = function(i) {
      matrix(
        data = stats::rnorm(n = time * j),
        nrow = j,
        ncol = time
      )
    }
  )
  gamma <- diag(x = 0.10, nrow = p, ncol = j)
  kappa <- diag(x = 0.10, nrow = k, ncol = j)

  # Type 0
  ssm <- SimSSMLinSDEFixed(
    n = n,
    time = time,
    delta_t = delta_t,
    mu0 = mu0,
    sigma0_l = sigma0_l,
    iota = iota,
    phi = phi,
    sigma_l = sigma_l,
    nu = nu,
    lambda = lambda,
    theta_l = theta_l,
    type = 0
  )

```



```

plot(ssm)

# Type 1
ssm <- SimSSMLinSDEFixed(
  n = n,
  time = time,
  delta_t = delta_t,
  mu0 = mu0,
  sigma0_l = sigma0_l,
  iota = iota,
  phi = phi,
  sigma_l = sigma_l,
  nu = nu,
  lambda = lambda,
  theta_l = theta_l,
  type = 1,
  x = x,
  gamma = gamma
)

plot(ssm)

# Type 2
ssm <- SimSSMLinSDEFixed(
  n = n,
  time = time,
  delta_t = delta_t,
  mu0 = mu0,
  sigma0_l = sigma0_l,
  iota = iota,
  phi = phi,
  sigma_l = sigma_l,
  nu = nu,
  lambda = lambda,
  theta_l = theta_l,
  type = 2,
  x = x,
  gamma = gamma,
  kappa = kappa
)

plot(ssm)

```

Description

This function simulates data from the linear stochastic differential equation model using a state space model parameterization. It assumes that the parameters can vary across individuals.

Usage

```
SimSSMLinSDEIVary(
  n,
  time,
  delta_t = 1,
  mu0,
  sigma0_l,
  iota,
  phi,
  sigma_l,
  nu,
  lambda,
  theta_l,
  type = 0,
  x = NULL,
  gamma = NULL,
  kappa = NULL
)
```

Arguments

n	Positive integer. Number of individuals.
time	Positive integer. Number of time points.
delta_t	Numeric. Time interval. The default value is 1.0 with an option to use a numeric value for the discretized state space model parameterization of the linear stochastic differential equation model.
mu0	List of numeric vectors. Each element of the list is the mean of initial latent variable values ($\mu_{\eta 0}$).
sigma0_l	List of numeric matrices. Each element of the list is the Cholesky factorization ($t(\text{chol}(\text{sigma0}))$) of the covariance matrix of initial latent variable values ($\Sigma_{\eta 0}$).
iota	List of numeric vectors. Each element of the list is an unobserved term that is constant over time (ι).
phi	List of numeric matrix. Each element of the list is the drift matrix which represents the rate of change of the solution in the absence of any random fluctuations (Φ).
sigma_l	List of numeric matrix. Each element of the list is the Cholesky factorization ($t(\text{chol}(\text{sigma}))$) of the covariance matrix of volatility or randomness in the process Σ .
nu	List of numeric vectors. Each element of the list is the vector of intercept values for the measurement model (ν).

lambda	List of numeric matrices. Each element of the list is the factor loading matrix linking the latent variables to the observed variables (Λ).
theta_1	List of numeric matrices. Each element of the list is the Cholesky factorization ($t(\text{chol}(\text{theta}))$) of the covariance matrix of the measurement error (Θ).
type	Integer. State space model type. See Details in SimSSMLinSDEFixed() for more information.
x	List. Each element of the list is a matrix of covariates for each individual i in n . The number of columns in each matrix should be equal to <code>time</code> .
gamma	List of numeric matrices. Each element of the list is the matrix linking the covariates to the latent variables at current time point (Γ).
kappa	List of numeric matrices. Each element of the list is the matrix linking the covariates to the observed variables at current time point (κ).

Details

Parameters can vary across individuals by providing a list of parameter values. If the length of any of the parameters (`mu0`, `sigma0_1`, `iota`, `phi`, `sigma_1`, `nu`, `lambda`, `theta_1`, `gamma`, or `kappa`) is less than `n`, the function will cycle through the available values.

Value

Returns an object of class `simstatespace` which is a list with the following elements:

- `call`: Function call.
- `args`: Function arguments.
- `data`: Generated data which is a list of length `n`. Each element of `data` is a list with the following elements:
 - `id`: A vector of ID numbers with length `1`, where `1` is the value of the function argument `time`.
 - `time`: A vector time points of length `1`.
 - `y`: A `1` by `k` matrix of values for the manifest variables.
 - `eta`: A `1` by `p` matrix of values for the latent variables.
 - `x`: A `1` by `j` matrix of values for the covariates (when covariates are included).
- `fun`: Function used.

Author(s)

Ivan Jacob Agaloos Pesigan

References

- Chow, S.-M., Ho, M. R., Hamaker, E. L., & Dolan, C. V. (2010). Equivalence and differences between structural equation modeling and state-space modeling techniques. *Structural Equation Modeling: A Multidisciplinary Journal*, 17(2), 303-332. doi:10.1080/10705511003661553
- Chow, S.-M., Losardo, D., Park, J., & Molenaar, P. C. M. (2023). Continuous-time dynamic models: Connections to structural equation models and other discrete-time models. In R. H. Hoyle (Ed.), *Handbook of structural equation modeling* (2nd ed.). The Guilford Press.

Harvey, A. C. (1990). Forecasting, structural time series models and the Kalman filter. Cambridge University Press. doi:10.1017/cbo9781107049994

See Also

Other Simulation of State Space Models Data Functions: `LinSDE2SSM()`, `LinSDECovEta()`, `LinSDECovY()`, `LinSDEInterceptEta()`, `LinSDEInterceptY()`, `LinSDEMeanEta()`, `LinSDEMeanY()`, `ProjectToHurwitz()`, `ProjectToStability()`, `SSMCovEta()`, `SSMCovY()`, `SSMInterceptEta()`, `SSMInterceptY()`, `SSMMeanEta()`, `SSMMeanY()`, `SimAlphaN()`, `SimBetaN()`, `SimBetaN2()`, `SimBetaNCovariate()`, `SimCovDiagN()`, `SimCovN()`, `SimIotaN()`, `SimMVN()`, `SimMuN()`, `SimNuN()`, `SimPhiN()`, `SimPhiN2()`, `SimPhiNCovariate()`, `SimSSMFixed()`, `SimSSMIVary()`, `SimSSMLinGrowth()`, `SimSSMLinGrowthIVary()`, `SimSSMLinSDEFixed()`, `SimSSMOUFixed()`, `SimSSMOUIVary()`, `SimSSMVARFixed()`, `SimSSMVARIVary()`, `SpectralRadius()`, `TestPhi()`, `TestPhiHurwitz()`, `TestStability()`, `TestStationarity()`

Examples

```
# prepare parameters
# In this example, phi varies across individuals.
set.seed(42)
## number of individuals
n <- 5
## time points
time <- 50
delta_t <- 0.10
## dynamic structure
p <- 2
mu0 <- list(
  c(-3.0, 1.5)
)
sigma0 <- 0.001 * diag(p)
sigma0_l <- list(
  t(chol(sigma0))
)
iota <- list(
  c(0.317, 0.230)
)
phi <- list(
  -0.1 * diag(p),
  -0.2 * diag(p),
  -0.3 * diag(p),
  -0.4 * diag(p),
  -0.5 * diag(p)
)
sigma <- matrix(
  data = c(
    2.79,
    0.06,
    0.06,
    3.27
  ),
  nrow = p
)
```

```

sigma_l <- list(
  t(chol(sigma))
)
## measurement model
k <- 2
nu <- list(
  rep(x = 0, times = k)
)
lambda <- list(
  diag(k)
)
theta <- 0.001 * diag(k)
theta_l <- list(
  t(chol(theta))
)
## covariates
j <- 2
x <- lapply(
  X = seq_len(n),
  FUN = function(i) {
    matrix(
      data = stats::rnorm(n = time * j),
      nrow = j,
      ncol = time
    )
  }
)
gamma <- list(
  diag(x = 0.10, nrow = p, ncol = j)
)
kappa <- list(
  diag(x = 0.10, nrow = k, ncol = j)
)

# Type 0
ssm <- SimSSMLinSDEIVary(
  n = n,
  time = time,
  delta_t = delta_t,
  mu0 = mu0,
  sigma0_l = sigma0_l,
  iota = iota,
  phi = phi,
  sigma_l = sigma_l,
  nu = nu,
  lambda = lambda,
  theta_l = theta_l,
  type = 0
)

plot(ssm)

# Type 1

```

```

ssm <- SimSSMLinSDEIVary(
  n = n,
  time = time,
  delta_t = delta_t,
  mu0 = mu0,
  sigma0_l = sigma0_l,
  iota = iota,
  phi = phi,
  sigma_l = sigma_l,
  nu = nu,
  lambda = lambda,
  theta_l = theta_l,
  type = 1,
  x = x,
  gamma = gamma
)

plot(ssm)

# Type 2
ssm <- SimSSMLinSDEIVary(
  n = n,
  time = time,
  delta_t = delta_t,
  mu0 = mu0,
  sigma0_l = sigma0_l,
  iota = iota,
  phi = phi,
  sigma_l = sigma_l,
  nu = nu,
  lambda = lambda,
  theta_l = theta_l,
  type = 2,
  x = x,
  gamma = gamma,
  kappa = kappa
)

plot(ssm)

```

SimSSMOUFixed

Simulate Data from the Ornstein-Uhlenbeck Model using a State Space Model Parameterization (Fixed Parameters)

Description

This function simulates data from the Ornstein-Uhlenbeck (OU) model using a state space model parameterization. It assumes that the parameters remain constant across individuals and over time.

Usage

```

SimSSMOUFixed(
  n,
  time,
  delta_t = 1,
  mu0,
  sigma0_l,
  mu,
  phi,
  sigma_l,
  nu,
  lambda,
  theta_l,
  type = 0,
  x = NULL,
  gamma = NULL,
  kappa = NULL
)

```

Arguments

n	Positive integer. Number of individuals.
time	Positive integer. Number of time points.
delta_t	Numeric. Time interval (Δ_t).
mu0	Numeric vector. Mean of initial latent variable values ($\mu_{\eta 0}$).
sigma0_l	Numeric matrix. Cholesky factorization ($t(\text{chol}(\text{sigma0}))$) of the covariance matrix of initial latent variable values ($\Sigma_{\eta 0}$).
mu	Numeric vector. The long-term mean or equilibrium level (μ).
phi	Numeric matrix. The drift matrix which represents the rate of change of the solution in the absence of any random fluctuations (Φ). It also represents the rate of mean reversion, determining how quickly the variable returns to its mean.
sigma_l	Numeric matrix. Cholesky factorization ($t(\text{chol}(\text{sigma}))$) of the covariance matrix of volatility or randomness in the process (Σ).
nu	Numeric vector. Vector of intercept values for the measurement model (ν).
lambda	Numeric matrix. Factor loading matrix linking the latent variables to the observed variables (Λ).
theta_l	Numeric matrix. Cholesky factorization ($t(\text{chol}(\text{theta}))$) of the covariance matrix of the measurement error (Θ).
type	Integer. State space model type. See Details for more information.
x	List. Each element of the list is a matrix of covariates for each individual i in n . The number of columns in each matrix should be equal to <code>time</code> .
gamma	Numeric matrix. Matrix linking the covariates to the latent variables at current time point (Γ).
kappa	Numeric matrix. Matrix linking the covariates to the observed variables at current time point (κ).

Details

Type 0:

The measurement model is given by

$$\mathbf{y}_{i,t} = \boldsymbol{\nu} + \boldsymbol{\Lambda}\boldsymbol{\eta}_{i,t} + \boldsymbol{\varepsilon}_{i,t}, \quad \text{with} \quad \boldsymbol{\varepsilon}_{i,t} \sim \mathcal{N}(\mathbf{0}, \boldsymbol{\Theta})$$

where $\mathbf{y}_{i,t}$, $\boldsymbol{\eta}_{i,t}$, and $\boldsymbol{\varepsilon}_{i,t}$ are random variables and $\boldsymbol{\nu}$, $\boldsymbol{\Lambda}$, and $\boldsymbol{\Theta}$ are model parameters. $\mathbf{y}_{i,t}$ represents a vector of observed random variables, $\boldsymbol{\eta}_{i,t}$ a vector of latent random variables, and $\boldsymbol{\varepsilon}_{i,t}$ a vector of random measurement errors, at time t and individual i . $\boldsymbol{\nu}$ denotes a vector of intercepts, $\boldsymbol{\Lambda}$ a matrix of factor loadings, and $\boldsymbol{\Theta}$ the covariance matrix of $\boldsymbol{\varepsilon}$.

An alternative representation of the measurement error is given by

$$\boldsymbol{\varepsilon}_{i,t} = \boldsymbol{\Theta}^{\frac{1}{2}} \mathbf{z}_{i,t}, \quad \text{with} \quad \mathbf{z}_{i,t} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$$

where $\mathbf{z}_{i,t}$ is a vector of independent standard normal random variables and $\left(\boldsymbol{\Theta}^{\frac{1}{2}}\right) \left(\boldsymbol{\Theta}^{\frac{1}{2}}\right)' = \boldsymbol{\Theta}$.

The dynamic structure is given by

$$d\boldsymbol{\eta}_{i,t} = \boldsymbol{\Phi}(\boldsymbol{\eta}_{i,t} - \boldsymbol{\mu}) dt + \boldsymbol{\Sigma}^{\frac{1}{2}} d\mathbf{W}_{i,t}$$

where $\boldsymbol{\mu}$ is the long-term mean or equilibrium level, $\boldsymbol{\Phi}$ is the rate of mean reversion, determining how quickly the variable returns to its mean, $\boldsymbol{\Sigma}$ is the matrix of volatility or randomness in the process, and $d\mathbf{W}$ is a Wiener process or Brownian motion, which represents random fluctuations.

Type 1:

The measurement model is given by

$$\mathbf{y}_{i,t} = \boldsymbol{\nu} + \boldsymbol{\Lambda}\boldsymbol{\eta}_{i,t} + \boldsymbol{\varepsilon}_{i,t}, \quad \text{with} \quad \boldsymbol{\varepsilon}_{i,t} \sim \mathcal{N}(\mathbf{0}, \boldsymbol{\Theta}).$$

The dynamic structure is given by

$$d\boldsymbol{\eta}_{i,t} = \boldsymbol{\Phi}(\boldsymbol{\eta}_{i,t} - \boldsymbol{\mu}) dt + \boldsymbol{\Gamma}\mathbf{x}_{i,t} + \boldsymbol{\Sigma}^{\frac{1}{2}} d\mathbf{W}_{i,t}$$

where $\mathbf{x}_{i,t}$ represents a vector of covariates at time t and individual i , and $\boldsymbol{\Gamma}$ the coefficient matrix linking the covariates to the latent variables.

Type 2:

The measurement model is given by

$$\mathbf{y}_{i,t} = \boldsymbol{\nu} + \boldsymbol{\Lambda}\boldsymbol{\eta}_{i,t} + \boldsymbol{\kappa}\mathbf{x}_{i,t} + \boldsymbol{\varepsilon}_{i,t}, \quad \text{with} \quad \boldsymbol{\varepsilon}_{i,t} \sim \mathcal{N}(\mathbf{0}, \boldsymbol{\Theta})$$

where $\boldsymbol{\kappa}$ represents the coefficient matrix linking the covariates to the observed variables.

The dynamic structure is given by

$$d\boldsymbol{\eta}_{i,t} = \boldsymbol{\Phi}(\boldsymbol{\eta}_{i,t} - \boldsymbol{\mu}) dt + \boldsymbol{\Gamma}\mathbf{x}_{i,t} + \boldsymbol{\Sigma}^{\frac{1}{2}} d\mathbf{W}_{i,t}.$$

The OU model as a linear stochastic differential equation model:

The OU model is a first-order linear stochastic differential equation model in the form of

$$d\boldsymbol{\eta}_{i,t} = (\boldsymbol{\iota} + \boldsymbol{\Phi}\boldsymbol{\eta}_{i,t}) dt + \boldsymbol{\Sigma}^{\frac{1}{2}} d\mathbf{W}_{i,t}$$

where $\boldsymbol{\mu} = -\boldsymbol{\Phi}^{-1}\boldsymbol{\iota}$ and, equivalently $\boldsymbol{\iota} = -\boldsymbol{\Phi}\boldsymbol{\mu}$.

Value

Returns an object of class `simstatespace` which is a list with the following elements:

- `call`: Function call.
- `args`: Function arguments.
- `data`: Generated data which is a list of length `n`. Each element of data is a list with the following elements:
 - `id`: A vector of ID numbers with length `l`, where `l` is the value of the function argument `time`.
 - `time`: A vector time points of length `l`.
 - `y`: A `l` by `k` matrix of values for the manifest variables.
 - `eta`: A `l` by `p` matrix of values for the latent variables.
 - `x`: A `l` by `j` matrix of values for the covariates (when covariates are included).
- `fun`: Function used.

Author(s)

Ivan Jacob Agaloos Pesigan

References

- Chow, S.-M., Ho, M. R., Hamaker, E. L., & Dolan, C. V. (2010). Equivalence and differences between structural equation modeling and state-space modeling techniques. *Structural Equation Modeling: A Multidisciplinary Journal*, 17(2), 303-332. doi:10.1080/10705511003661553
- Chow, S.-M., Losardo, D., Park, J., & Molenaar, P. C. M. (2023). Continuous-time dynamic models: Connections to structural equation models and other discrete-time models. In R. H. Hoyle (Ed.), *Handbook of structural equation modeling* (2nd ed.). The Guilford Press.
- Harvey, A. C. (1990). *Forecasting, structural time series models and the Kalman filter*. Cambridge University Press. doi:10.1017/cbo9781107049994
- Oravecz, Z., Tuerlinckx, F., & Vandekerckhove, J. (2011). A hierarchical latent stochastic differential equation model for affective dynamics. *Psychological Methods*, 16 (4), 468-490. doi:10.1037/a0024375
- Uhlenbeck, G. E., & Ornstein, L. S. (1930). On the theory of the brownian motion. *Physical Review*, 36 (5), 823-841. doi:10.1103/physrev.36.823

See Also

Other Simulation of State Space Models Data Functions: `LinSDE2SSM()`, `LinSDECovEta()`, `LinSDECovY()`, `LinSDEInterceptEta()`, `LinSDEInterceptY()`, `LinSDEMeanEta()`, `LinSDEMeanY()`, `ProjectToHurwitz()`, `ProjectToStability()`, `SSMCovEta()`, `SSMCovY()`, `SSMInterceptEta()`, `SSMInterceptY()`, `SSMMeanEta()`, `SSMMeanY()`, `SimAlphaN()`, `SimBetaN()`, `SimBetaN2()`, `SimBetaNCovariate()`, `SimCovDiagN()`, `SimCovN()`, `SimIotaN()`, `SimMVN()`, `SimMuN()`, `SimNuN()`, `SimPhiN()`, `SimPhiN2()`, `SimPhiNCovariate()`, `SimSSMFixed()`, `SimSSMIVary()`, `SimSSMLinGrowth()`, `SimSSMLinGrowthIVary()`, `SimSSMLinSDEFixed()`, `SimSSMLinSDEIVary()`, `SimSSMOUIVary()`, `SimSSMVARFixed()`, `SimSSMVARIVary()`, `SpectralRadius()`, `TestPhi()`, `TestPhiHurwitz()`, `TestStability()`, `TestStationarity()`

Examples

```

# prepare parameters
set.seed(42)
## number of individuals
n <- 5
## time points
time <- 50
delta_t <- 0.10
## dynamic structure
p <- 2
mu0 <- c(-3.0, 1.5)
sigma0 <- 0.001 * diag(p)
sigma0_l <- t(chol(sigma0))
mu <- c(5.76, 5.18)
phi <- matrix(
  data = c(
    -0.10,
    0.05,
    0.05,
    -0.10
  ),
  nrow = p
)
sigma <- matrix(
  data = c(
    2.79,
    0.06,
    0.06,
    3.27
  ),
  nrow = p
)
sigma_l <- t(chol(sigma))
## measurement model
k <- 2
nu <- rep(x = 0, times = k)
lambda <- diag(k)
theta <- 0.001 * diag(k)
theta_l <- t(chol(theta))
## covariates
j <- 2
x <- lapply(
  X = seq_len(n),
  FUN = function(i) {
    matrix(
      data = stats::rnorm(n = time * j),
      nrow = j,
      ncol = time
    )
  }
)
gamma <- diag(x = 0.10, nrow = p, ncol = j)

```

```
kappa <- diag(x = 0.10, nrow = k, ncol = j)
```

```
# Type 0
ssm <- SimSSMOUFixed(
  n = n,
  time = time,
  delta_t = delta_t,
  mu0 = mu0,
  sigma0_l = sigma0_l,
  mu = mu,
  phi = phi,
  sigma_l = sigma_l,
  nu = nu,
  lambda = lambda,
  theta_l = theta_l,
  type = 0
)
```

```
plot(ssm)
```

```
# Type 1
ssm <- SimSSMOUFixed(
  n = n,
  time = time,
  delta_t = delta_t,
  mu0 = mu0,
  sigma0_l = sigma0_l,
  mu = mu,
  phi = phi,
  sigma_l = sigma_l,
  nu = nu,
  lambda = lambda,
  theta_l = theta_l,
  type = 1,
  x = x,
  gamma = gamma
)
```

```
plot(ssm)
```

```
# Type 2
ssm <- SimSSMOUFixed(
  n = n,
  time = time,
  delta_t = delta_t,
  mu0 = mu0,
  sigma0_l = sigma0_l,
  mu = mu,
  phi = phi,
  sigma_l = sigma_l,
  nu = nu,
  lambda = lambda,
  theta_l = theta_l,
)
```

```

    type = 2,
    x = x,
    gamma = gamma,
    kappa = kappa
  )

plot(ssm)

```

SimSSMOUIVary

Simulate Data from the Ornstein-Uhlenbeck Model using a State Space Model Parameterization (Individual-Varying Parameters)

Description

This function simulates data from the Ornstein-Uhlenbeck model using a state space model parameterization. It assumes that the parameters can vary across individuals.

Usage

```

SimSSMOUIVary(
  n,
  time,
  delta_t = 1,
  mu0,
  sigma0_l,
  mu,
  phi,
  sigma_l,
  nu,
  lambda,
  theta_l,
  type = 0,
  x = NULL,
  gamma = NULL,
  kappa = NULL
)

```

Arguments

n	Positive integer. Number of individuals.
time	Positive integer. Number of time points.
delta_t	Numeric. Time interval. The default value is 1.0 with an option to use a numeric value for the discretized state space model parameterization of the linear stochastic differential equation model.
mu0	List of numeric vectors. Each element of the list is the mean of initial latent variable values ($\mu_{\eta 0}$).

sigma0_l	List of numeric matrices. Each element of the list is the Cholesky factorization ($t(\text{chol}(\text{sigma0}))$) of the covariance matrix of initial latent variable values ($\Sigma_{\eta 0}$).
mu	List of numeric vectors. Each element of the list is the long-term mean or equilibrium level (μ).
phi	List of numeric matrix. Each element of the list is the drift matrix which represents the rate of change of the solution in the absence of any random fluctuations (Φ). It also represents the rate of mean reversion, determining how quickly the variable returns to its mean.
sigma_l	List of numeric matrix. Each element of the list is the Cholesky factorization ($t(\text{chol}(\text{sigma}))$) of the covariance matrix of volatility or randomness in the process Σ .
nu	List of numeric vectors. Each element of the list is the vector of intercept values for the measurement model (ν).
lambda	List of numeric matrices. Each element of the list is the factor loading matrix linking the latent variables to the observed variables (Λ).
theta_l	List of numeric matrices. Each element of the list is the Cholesky factorization ($t(\text{chol}(\text{theta}))$) of the covariance matrix of the measurement error (Θ).
type	Integer. State space model type. See Details in SimSSMOUFixed() for more information.
x	List. Each element of the list is a matrix of covariates for each individual i in n . The number of columns in each matrix should be equal to time.
gamma	List of numeric matrices. Each element of the list is the matrix linking the covariates to the latent variables at current time point (Γ).
kappa	List of numeric matrices. Each element of the list is the matrix linking the covariates to the observed variables at current time point (κ).

Details

Parameters can vary across individuals by providing a list of parameter values. If the length of any of the parameters (`mu0`, `sigma0_l`, `mu`, `phi`, `sigma_l`, `nu`, `lambda`, `theta_l`, `gamma`, or `kappa`) is less than `n`, the function will cycle through the available values.

Value

Returns an object of class `simstatespace` which is a list with the following elements:

- `call`: Function call.
- `args`: Function arguments.
- `data`: Generated data which is a list of length `n`. Each element of data is a list with the following elements:
 - `id`: A vector of ID numbers with length `1`, where `1` is the value of the function argument `time`.
 - `time`: A vector time points of length `1`.
 - `y`: A `1` by `k` matrix of values for the manifest variables.

- eta: A 1 by p matrix of values for the latent variables.
- x: A 1 by j matrix of values for the covariates (when covariates are included).
- fun: Function used.

Author(s)

Ivan Jacob Agaloos Pesigan

References

- Chow, S.-M., Ho, M. R., Hamaker, E. L., & Dolan, C. V. (2010). Equivalence and differences between structural equation modeling and state-space modeling techniques. *Structural Equation Modeling: A Multidisciplinary Journal*, 17(2), 303-332. doi:10.1080/10705511003661553
- Chow, S.-M., Losardo, D., Park, J., & Molenaar, P. C. M. (2023). Continuous-time dynamic models: Connections to structural equation models and other discrete-time models. In R. H. Hoyle (Ed.), *Handbook of structural equation modeling* (2nd ed.). The Guilford Press.
- Harvey, A. C. (1990). *Forecasting, structural time series models and the Kalman filter*. Cambridge University Press. doi:10.1017/cbo9781107049994
- Oravecz, Z., Tuerlinckx, F., & Vandekerckhove, J. (2011). A hierarchical latent stochastic differential equation model for affective dynamics. *Psychological Methods*, 16 (4), 468-490. doi:10.1037/a0024375
- Uhlenbeck, G. E., & Ornstein, L. S. (1930). On the theory of the brownian motion. *Physical Review*, 36 (5), 823-841. doi:10.1103/physrev.36.823

See Also

Other Simulation of State Space Models Data Functions: `LinSDE2SSM()`, `LinSDECovEta()`, `LinSDECovY()`, `LinSDEInterceptEta()`, `LinSDEInterceptY()`, `LinSDEMeanEta()`, `LinSDEMeanY()`, `ProjectToHurwitz()`, `ProjectToStability()`, `SSMCovEta()`, `SSMCovY()`, `SSMInterceptEta()`, `SSMInterceptY()`, `SSMMeanEta()`, `SSMMeanY()`, `SimAlphaN()`, `SimBetaN()`, `SimBetaN2()`, `SimBetaNCovariate()`, `SimCovDiagN()`, `SimCovN()`, `SimIotaN()`, `SimMVN()`, `SimMuN()`, `SimNuN()`, `SimPhiN()`, `SimPhiN2()`, `SimPhiNCovariate()`, `SimSSMFixed()`, `SimSSMIVary()`, `SimSSMLinGrowth()`, `SimSSMLinGrowthIVary()`, `SimSSMLinSDEFixed()`, `SimSSMLinSDEIVary()`, `SimSSMOUFixed()`, `SimSSMVARFixed()`, `SimSSMVARIVary()`, `SpectralRadius()`, `TestPhi()`, `TestPhiHurwitz()`, `TestStability()`, `TestStationarity()`

Examples

```
# prepare parameters
# In this example, phi varies across individuals.
set.seed(42)
## number of individuals
n <- 5
## time points
time <- 50
delta_t <- 0.10
## dynamic structure
p <- 2
mu0 <- list(
  c(-3.0, 1.5)
```

```

)
sigma0 <- 0.001 * diag(p)
sigma0_l <- list(
  t(chol(sigma0))
)
mu <- list(
  c(5.76, 5.18)
)
phi <- list(
  -0.1 * diag(p),
  -0.2 * diag(p),
  -0.3 * diag(p),
  -0.4 * diag(p),
  -0.5 * diag(p)
)
sigma <- matrix(
  data = c(
    2.79,
    0.06,
    0.06,
    3.27
  ),
  nrow = p
)
sigma_l <- list(
  t(chol(sigma))
)
## measurement model
k <- 2
nu <- list(
  rep(x = 0, times = k)
)
lambda <- list(
  diag(k)
)
theta <- 0.001 * diag(k)
theta_l <- list(
  t(chol(theta))
)
## covariates
j <- 2
x <- lapply(
  X = seq_len(n),
  FUN = function(i) {
    matrix(
      data = stats::rnorm(n = time * j),
      nrow = j,
      ncol = time
    )
  }
)
gamma <- list(
  diag(x = 0.10, nrow = p, ncol = j)
)

```

```

)
kappa <- list(
  diag(x = 0.10, nrow = k, ncol = j)
)

# Type 0
ssm <- SimSSMOUIVary(
  n = n,
  time = time,
  delta_t = delta_t,
  mu0 = mu0,
  sigma0_l = sigma0_l,
  mu = mu,
  phi = phi,
  sigma_l = sigma_l,
  nu = nu,
  lambda = lambda,
  theta_l = theta_l,
  type = 0
)

plot(ssm)

# Type 1
ssm <- SimSSMOUIVary(
  n = n,
  time = time,
  delta_t = delta_t,
  mu0 = mu0,
  sigma0_l = sigma0_l,
  mu = mu,
  phi = phi,
  sigma_l = sigma_l,
  nu = nu,
  lambda = lambda,
  theta_l = theta_l,
  type = 1,
  x = x,
  gamma = gamma
)

plot(ssm)

# Type 2
ssm <- SimSSMOUIVary(
  n = n,
  time = time,
  delta_t = delta_t,
  mu0 = mu0,
  sigma0_l = sigma0_l,
  mu = mu,
  phi = phi,
  sigma_l = sigma_l,

```



```

    nu = nu,
    lambda = lambda,
    theta_l = theta_l,
    type = 2,
    x = x,
    gamma = gamma,
    kappa = kappa
)

plot(ssm)

```

SimSSMVARFixed	<i>Simulate Data from the Vector Autoregressive Model (Fixed Parameters)</i>
----------------	--

Description

This function simulates data from the vector autoregressive model using a state space model parameterization. It assumes that the parameters remain constant across individuals and over time.

Usage

```

SimSSMVARFixed(
  n,
  time,
  mu0,
  sigma0_l,
  alpha,
  beta,
  psi_l,
  type = 0,
  x = NULL,
  gamma = NULL
)

```

Arguments

n	Positive integer. Number of individuals.
time	Positive integer. Number of time points.
mu0	Numeric vector. Mean of initial latent variable values ($\mu_{\eta 0}$).
sigma0_l	Numeric matrix. Cholesky factorization ($t(\text{chol}(\text{sigma0}))$) of the covariance matrix of initial latent variable values ($\Sigma_{\eta 0}$).
alpha	Numeric vector. Vector of constant values for the dynamic model (α).
beta	Numeric matrix. Transition matrix relating the values of the latent variables at the previous to the current time point (β).

psi_l	Numeric matrix. Cholesky factorization ($\text{t}(\text{chol}(\text{psi}))$) of the covariance matrix of the process noise (Ψ).
type	Integer. State space model type. See Details for more information.
x	List. Each element of the list is a matrix of covariates for each individual i in n . The number of columns in each matrix should be equal to <code>time</code> .
gamma	Numeric matrix. Matrix linking the covariates to the latent variables at current time point (Γ).

Details

Type 0:

The measurement model is given by

$$\mathbf{y}_{i,t} = \boldsymbol{\eta}_{i,t}$$

where $\mathbf{y}_{i,t}$ represents a vector of observed variables and $\boldsymbol{\eta}_{i,t}$ a vector of latent variables for individual i and time t . Since the observed and latent variables are equal, we only generate data from the dynamic structure.

The dynamic structure is given by

$$\boldsymbol{\eta}_{i,t} = \boldsymbol{\alpha} + \beta \boldsymbol{\eta}_{i,t-1} + \boldsymbol{\zeta}_{i,t}, \quad \text{with} \quad \boldsymbol{\zeta}_{i,t} \sim \mathcal{N}(\mathbf{0}, \Psi)$$

where $\boldsymbol{\eta}_{i,t}$, $\boldsymbol{\eta}_{i,t-1}$, and $\boldsymbol{\zeta}_{i,t}$ are random variables, and $\boldsymbol{\alpha}$, β , and Ψ are model parameters. Here, $\boldsymbol{\eta}_{i,t}$ is a vector of latent variables at time t and individual i , $\boldsymbol{\eta}_{i,t-1}$ represents a vector of latent variables at time $t-1$ and individual i , and $\boldsymbol{\zeta}_{i,t}$ represents a vector of dynamic noise at time t and individual i . $\boldsymbol{\alpha}$ denotes a vector of intercepts, β a matrix of autoregression and cross regression coefficients, and Ψ the covariance matrix of $\boldsymbol{\zeta}_{i,t}$.

An alternative representation of the dynamic noise is given by

$$\boldsymbol{\zeta}_{i,t} = \Psi^{\frac{1}{2}} \mathbf{z}_{i,t}, \quad \text{with} \quad \mathbf{z}_{i,t} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$$

where $\left(\Psi^{\frac{1}{2}}\right) \left(\Psi^{\frac{1}{2}}\right)' = \Psi$.

Type 1:

The measurement model is given by

$$\mathbf{y}_{i,t} = \boldsymbol{\eta}_{i,t}$$

The dynamic structure is given by

$$\boldsymbol{\eta}_{i,t} = \boldsymbol{\alpha} + \beta \boldsymbol{\eta}_{i,t-1} + \Gamma \mathbf{x}_{i,t} + \boldsymbol{\zeta}_{i,t}, \quad \text{with} \quad \boldsymbol{\zeta}_{i,t} \sim \mathcal{N}(\mathbf{0}, \Psi)$$

where $\mathbf{x}_{i,t}$ represents a vector of covariates at time t and individual i , and Γ the coefficient matrix linking the covariates to the latent variables.

Value

Returns an object of class `simstatespace` which is a list with the following elements:

- `call`: Function call.
- `args`: Function arguments.

- data: Generated data which is a list of length n. Each element of data is a list with the following elements:
 - id: A vector of ID numbers with length 1, where 1 is the value of the function argument time.
 - time: A vector time points of length 1.
 - y: A 1 by k matrix of values for the manifest variables.
 - eta: A 1 by p matrix of values for the latent variables.
 - x: A 1 by j matrix of values for the covariates (when covariates are included).
- fun: Function used.

Author(s)

Ivan Jacob Agaloos Pesigan

References

Chow, S.-M., Ho, M. R., Hamaker, E. L., & Dolan, C. V. (2010). Equivalence and differences between structural equation modeling and state-space modeling techniques. *Structural Equation Modeling: A Multidisciplinary Journal*, 17(2), 303-332. doi:10.1080/10705511003661553

See Also

Other Simulation of State Space Models Data Functions: [LinSDE2SSM\(\)](#), [LinSDECovEta\(\)](#), [LinSDECovY\(\)](#), [LinSDEInterceptEta\(\)](#), [LinSDEInterceptY\(\)](#), [LinSDEMeanEta\(\)](#), [LinSDEMeanY\(\)](#), [ProjectToHurwitz\(\)](#), [ProjectToStability\(\)](#), [SSMCovEta\(\)](#), [SSMCovY\(\)](#), [SSMInterceptEta\(\)](#), [SSMInterceptY\(\)](#), [SSMMeanEta\(\)](#), [SSMMeanY\(\)](#), [SimAlphaN\(\)](#), [SimBetaN\(\)](#), [SimBetaN2\(\)](#), [SimBetaNCovariate\(\)](#), [SimCovDiagN\(\)](#), [SimCovN\(\)](#), [SimIotaN\(\)](#), [SimMVN\(\)](#), [SimMuN\(\)](#), [SimNuN\(\)](#), [SimPhiN\(\)](#), [SimPhiN2\(\)](#), [SimPhiNCovariate\(\)](#), [SimSSMFixed\(\)](#), [SimSSMIVary\(\)](#), [SimSSMLinGrowth\(\)](#), [SimSSMLinGrowthIVary\(\)](#), [SimSSMLinSDEFixed\(\)](#), [SimSSMLinSDEIVary\(\)](#), [SimSSMOUFixed\(\)](#), [SimSSMOUIVary\(\)](#), [SimSSMVARIVary\(\)](#), [SpectralRadius\(\)](#), [TestPhi\(\)](#), [TestPhiHurwitz\(\)](#), [TestStability\(\)](#), [TestStationarity\(\)](#)

Examples

```
# prepare parameters
set.seed(42)
## number of individuals
n <- 5
## time points
time <- 50
## dynamic structure
p <- 3
mu0 <- rep(x = 0, times = p)
sigma0 <- 0.001 * diag(p)
sigma0_l <- t(chol(sigma0))
alpha <- rep(x = 0, times = p)
beta <- 0.50 * diag(p)
psi <- 0.001 * diag(p)
psi_l <- t(chol(psi))
## covariates
j <- 2
```

```

x <- lapply(
  X = seq_len(n),
  FUN = function(i) {
    matrix(
      data = stats::rnorm(n = time * j),
      nrow = j,
      ncol = time
    )
  }
)
gamma <- diag(x = 0.10, nrow = p, ncol = j)

# Type 0
ssm <- SimSSMVARFixed(
  n = n,
  time = time,
  mu0 = mu0,
  sigma0_l = sigma0_l,
  alpha = alpha,
  beta = beta,
  psi_l = psi_l,
  type = 0
)

plot(ssm)

# Type 1
ssm <- SimSSMVARFixed(
  n = n,
  time = time,
  mu0 = mu0,
  sigma0_l = sigma0_l,
  alpha = alpha,
  beta = beta,
  psi_l = psi_l,
  type = 1,
  x = x,
  gamma = gamma
)

plot(ssm)

```

SimSSMVARIVary

Simulate Data from the Vector Autoregressive Model (Individual-Varying Parameters)

Description

This function simulates data from the vector autoregressive model using a state space model parameterization. It assumes that the parameters can vary across individuals.

Usage

```
SimSSMVARIVary(
  n,
  time,
  mu0,
  sigma0_l,
  alpha,
  beta,
  psi_l,
  type = 0,
  x = NULL,
  gamma = NULL
)
```

Arguments

n	Positive integer. Number of individuals.
time	Positive integer. Number of time points.
mu0	List of numeric vectors. Each element of the list is the mean of initial latent variable values ($\mu_{\eta 0}$).
sigma0_l	List of numeric matrices. Each element of the list is the Cholesky factorization ($t(\text{chol}(\text{sigma0}))$) of the covariance matrix of initial latent variable values ($\Sigma_{\eta 0}$).
alpha	List of numeric vectors. Each element of the list is the vector of constant values for the dynamic model (α).
beta	List of numeric matrices. Each element of the list is the transition matrix relating the values of the latent variables at the previous to the current time point (β).
psi_l	List of numeric matrices. Each element of the list is the Cholesky factorization ($t(\text{chol}(\text{psi}))$) of the covariance matrix of the process noise (Ψ).
type	Integer. State space model type. See Details in SimSSMVARFixed() for more information.
x	List. Each element of the list is a matrix of covariates for each individual i in n . The number of columns in each matrix should be equal to <code>time</code> .
gamma	List of numeric matrices. Each element of the list is the matrix linking the covariates to the latent variables at current time point (Γ).

Details

Parameters can vary across individuals by providing a list of parameter values. If the length of any of the parameters (`mu0`, `sigma0_l`, `alpha`, `beta`, `psi_l`, `gamma`, or `kappa`) is less than `n`, the function will cycle through the available values.

Value

Returns an object of class `simstatespace` which is a list with the following elements:

- call: Function call.
- args: Function arguments.
- data: Generated data which is a list of length n. Each element of data is a list with the following elements:
 - id: A vector of ID numbers with length 1, where 1 is the value of the function argument time.
 - time: A vector time points of length 1.
 - y: A 1 by k matrix of values for the manifest variables.
 - eta: A 1 by p matrix of values for the latent variables.
 - x: A 1 by j matrix of values for the covariates (when covariates are included).
- fun: Function used.

Author(s)

Ivan Jacob Agaloos Pesigan

References

Chow, S.-M., Ho, M. R., Hamaker, E. L., & Dolan, C. V. (2010). Equivalence and differences between structural equation modeling and state-space modeling techniques. *Structural Equation Modeling: A Multidisciplinary Journal*, 17(2), 303-332. doi:10.1080/10705511003661553

See Also

Other Simulation of State Space Models Data Functions: [LinSDE2SSM\(\)](#), [LinSDECovEta\(\)](#), [LinSDECovY\(\)](#), [LinSDEInterceptEta\(\)](#), [LinSDEInterceptY\(\)](#), [LinSDEMeanEta\(\)](#), [LinSDEMeanY\(\)](#), [ProjectToHurwitz\(\)](#), [ProjectToStability\(\)](#), [SSMCovEta\(\)](#), [SSMCovY\(\)](#), [SSMInterceptEta\(\)](#), [SSMInterceptY\(\)](#), [SSMMeanEta\(\)](#), [SSMMeanY\(\)](#), [SimAlphaN\(\)](#), [SimBetaN\(\)](#), [SimBetaN2\(\)](#), [SimBetaNCovariate\(\)](#), [SimCovDiagN\(\)](#), [SimCovN\(\)](#), [SimIotaN\(\)](#), [SimMVN\(\)](#), [SimMuN\(\)](#), [SimNuN\(\)](#), [SimPhiN\(\)](#), [SimPhiN2\(\)](#), [SimPhiNCovariate\(\)](#), [SimSSMFixed\(\)](#), [SimSSMIVary\(\)](#), [SimSSMLinGrowth\(\)](#), [SimSSMLinGrowthIVary\(\)](#), [SimSSMLinSDEFixed\(\)](#), [SimSSMLinSDEIVary\(\)](#), [SimSSMOUFixed\(\)](#), [SimSSMOUIVary\(\)](#), [SimSSMVARFixed\(\)](#), [SpectralRadius\(\)](#), [TestPhi\(\)](#), [TestPhiHurwitz\(\)](#), [TestStability\(\)](#), [TestStationarity\(\)](#)

Examples

```
# prepare parameters
# In this example, beta varies across individuals.
set.seed(42)
## number of individuals
n <- 5
## time points
time <- 50
## dynamic structure
p <- 3
mu0 <- list(
  rep(x = 0, times = p)
)
sigma0 <- 0.001 * diag(p)
sigma0_1 <- list(
```

```

      t(chol(sigma0))
    )
    alpha <- list(
      rep(x = 0, times = p)
    )
    beta <- list(
      0.1 * diag(p),
      0.2 * diag(p),
      0.3 * diag(p),
      0.4 * diag(p),
      0.5 * diag(p)
    )
    psi <- 0.001 * diag(p)
    psi_l <- list(
      t(chol(psi))
    )
    ## covariates
    j <- 2
    x <- lapply(
      X = seq_len(n),
      FUN = function(i) {
        matrix(
          data = stats::rnorm(n = time * j),
          nrow = j,
          ncol = time
        )
      }
    )
    gamma <- list(
      diag(x = 0.10, nrow = p, ncol = j)
    )

# Type 0
ssm <- SimSSMVARIVary(
  n = n,
  time = time,
  mu0 = mu0,
  sigma0_l = sigma0_l,
  alpha = alpha,
  beta = beta,
  psi_l = psi_l,
  type = 0
)

plot(ssm)

# Type 1
ssm <- SimSSMVARIVary(
  n = n,
  time = time,
  mu0 = mu0,
  sigma0_l = sigma0_l,
  alpha = alpha,

```

```

    beta = beta,
    psi_l = psi_l,
    type = 1,
    x = x,
    gamma = gamma
)

plot(ssm)

```

SpectralAbscissa	<i>Spectral Abscissa</i>
------------------	--------------------------

Description

Returns the maximum real part of the eigenvalues of a square matrix. For continuous-time stability (Hurwitz), a matrix is stable if the spectral abscissa is strictly less than 0.

Usage

```
SpectralAbscissa(x)
```

Arguments

x Numeric square matrix.

Value

Numeric value $\alpha(x) = \max \Re(\lambda_i(x))$.

Author(s)

Ivan Jacob Agaloos Pesigan

Examples

```

# Hurwitz-stable (spectral abscissa < 0):
x <- matrix(
  data = c(
    -0.5, -0.2,
    1.0, -0.3
  ),
  nrow = 2
)
SpectralAbscissa(x = x) # < 0

# Unstable (spectral abscissa > 0):
x <- matrix(
  data = c(
    0.10, 0.50,

```



```

        -0.40, 0.20
    ),
    nrow = 2
)
SpectralAbscissa(x = x) # > 0

```

SpectralRadius	<i>Spectral Radius</i>
----------------	------------------------

Description

Computes the spectral radius of a square matrix, defined as the maximum modulus (absolute value) of its eigenvalues. The spectral radius is often used to assess the stability of systems such as vector autoregressive (VAR) models: a system is considered stationary if the spectral radius of its transition matrix is strictly less than 1.

Usage

```
SpectralRadius(x)
```

Arguments

x Numeric square matrix.

Value

Numeric value representing the spectral radius of x.

Author(s)

Ivan Jacob Agaloos Pesigan

See Also

Other Simulation of State Space Models Data Functions: [LinSDE2SSM\(\)](#), [LinSDECovEta\(\)](#), [LinSDECovY\(\)](#), [LinSDEInterceptEta\(\)](#), [LinSDEInterceptY\(\)](#), [LinSDEMeanEta\(\)](#), [LinSDEMeanY\(\)](#), [ProjectToHurwitz\(\)](#), [ProjectToStability\(\)](#), [SSMCovEta\(\)](#), [SSMCovY\(\)](#), [SSMInterceptEta\(\)](#), [SSMInterceptY\(\)](#), [SSMMeanEta\(\)](#), [SSMMeanY\(\)](#), [SimAlphaN\(\)](#), [SimBetaN\(\)](#), [SimBetaN2\(\)](#), [SimBetaNCovariate\(\)](#), [SimCovDiagN\(\)](#), [SimCovN\(\)](#), [SimIotaN\(\)](#), [SimMVN\(\)](#), [SimMuN\(\)](#), [SimNuN\(\)](#), [SimPhiN\(\)](#), [SimPhiN2\(\)](#), [SimPhiNCovariate\(\)](#), [SimSSMFixed\(\)](#), [SimSSMIVary\(\)](#), [SimSSMLinGrowth\(\)](#), [SimSSMLinGrowthIVary\(\)](#), [SimSSMLinSDEFixed\(\)](#), [SimSSMLinSDEIVary\(\)](#), [SimSSMOUFixed\(\)](#), [SimSSMOUIVary\(\)](#), [SimSSMVARFixed\(\)](#), [SimSSMVARIVary\(\)](#), [TestPhi\(\)](#), [TestPhiHurwitz\(\)](#), [TestStability\(\)](#), [TestStationarity\(\)](#)

Examples

```
# Matrix with eigenvalues less than 1
x <- matrix(
  data = c(
    0.5, 0.3,
    0.2, 0.4
  ),
  nrow = 2
)
SpectralRadius(x)

# Matrix with eigenvalues greater than 1
y <- matrix(
  data = c(
    1.2, 0.3,
    0.4, 0.9
  ),
  nrow = 2
)
SpectralRadius(y)
```

SSMCovEta

Steady-State Covariance Matrix for the Latent Variables in the State Space Model

Description

The steady-state covariance matrix for the latent variables in the state space model $\text{Cov}(\boldsymbol{\eta})$ is given by

$$\text{vec}(\text{Cov}(\boldsymbol{\eta})) = (\mathbf{I} - \boldsymbol{\beta} \otimes \boldsymbol{\beta})^{-1} \text{vec}(\boldsymbol{\Psi})$$

where $\boldsymbol{\beta}$ is the transition matrix relating the values of the latent variables at the previous to the current time point and $\boldsymbol{\Psi}$ is the covariance matrix of volatility or randomness in the process.

Usage

```
SSMCovEta(beta, psi)
```

Arguments

beta	Numeric matrix. Transition matrix relating the values of the latent variables at the previous to the current time point ($\boldsymbol{\beta}$).
psi	Numeric matrix. The covariance matrix of the process noise ($\boldsymbol{\Psi}$).

Author(s)

Ivan Jacob Agaloos Pesigan

See Also

Other Simulation of State Space Models Data Functions: [LinSDE2SSM\(\)](#), [LinSDECovEta\(\)](#), [LinSDECovY\(\)](#), [LinSDEInterceptEta\(\)](#), [LinSDEInterceptY\(\)](#), [LinSDEMeanEta\(\)](#), [LinSDEMeanY\(\)](#), [ProjectToHurwitz\(\)](#), [ProjectToStability\(\)](#), [SSMCovY\(\)](#), [SSMInterceptEta\(\)](#), [SSMInterceptY\(\)](#), [SSMMeanEta\(\)](#), [SSMMeanY\(\)](#), [SimAlphaN\(\)](#), [SimBetaN\(\)](#), [SimBetaN2\(\)](#), [SimBetaNCovariate\(\)](#), [SimCovDiagN\(\)](#), [SimCovN\(\)](#), [SimIotaN\(\)](#), [SimMVN\(\)](#), [SimMuN\(\)](#), [SimNuN\(\)](#), [SimPhiN\(\)](#), [SimPhiN2\(\)](#), [SimPhiNCovariate\(\)](#), [SimSSMFixed\(\)](#), [SimSSMIVary\(\)](#), [SimSSMLinGrowth\(\)](#), [SimSSMLinGrowthIVary\(\)](#), [SimSSMLinSDEFixed\(\)](#), [SimSSMLinSDEIVary\(\)](#), [SimSSMOUFixed\(\)](#), [SimSSMOUIVary\(\)](#), [SimSSMVARFixed\(\)](#), [SimSSMVARIVary\(\)](#), [SpectralRadius\(\)](#), [TestPhi\(\)](#), [TestPhiHurwitz\(\)](#), [TestStability\(\)](#), [TestStationarity\(\)](#)

Examples

```
beta <- matrix(
  data = c(
    0.7, 0.5, -0.1,
    0.0, 0.6, 0.4,
    0.0, 0.0, 0.5
  ),
  nrow = 3
)
psi <- 0.1 * diag(3)
SSMCovEta(
  beta = beta,
  psi = psi
)
```

SSMCovY

Steady-State Covariance Matrix for the Observed Variables in the State Space Model

Description

The steady-state covariance matrix for the observed variables in the state space model $\text{Cov}(\mathbf{y})$ is given by

$$\text{Cov}(\mathbf{y}) = \mathbf{\Lambda} \text{Cov}(\boldsymbol{\eta}) \mathbf{\Lambda}' + \boldsymbol{\Theta}$$

where $\mathbf{\Lambda}$ is the matrix of factor loadings, $\boldsymbol{\Theta}$ is the covariance matrix of the measurement error, and $\text{Cov}(\boldsymbol{\eta})$ is the steady-state covariance matrix for the latent variables.

Usage

```
SSMCovY(lambda, theta, cov_eta)
```

Arguments

lambda	Numeric matrix. Factor loading matrix linking the latent variables to the observed variables (Λ).
theta	Numeric matrix. The covariance matrix of the measurement error (Θ).
cov_eta	Numeric matrix. The steady-state covariance matrix for the latent variables in the state space model

Author(s)

Ivan Jacob Agaloos Pesigan

See Also

Other Simulation of State Space Models Data Functions: [LinSDE2SSM\(\)](#), [LinSDECovEta\(\)](#), [LinSDECovY\(\)](#), [LinSDEInterceptEta\(\)](#), [LinSDEInterceptY\(\)](#), [LinSDEMeanEta\(\)](#), [LinSDEMeanY\(\)](#), [ProjectToHurwitz\(\)](#), [ProjectToStability\(\)](#), [SSMCovEta\(\)](#), [SSMInterceptEta\(\)](#), [SSMInterceptY\(\)](#), [SSMMeanEta\(\)](#), [SSMMeanY\(\)](#), [SimAlphaN\(\)](#), [SimBetaN\(\)](#), [SimBetaN2\(\)](#), [SimBetaNCovariate\(\)](#), [SimCovDiagN\(\)](#), [SimCovN\(\)](#), [SimIotaN\(\)](#), [SimMVN\(\)](#), [SimMuN\(\)](#), [SimNuN\(\)](#), [SimPhiN\(\)](#), [SimPhiN2\(\)](#), [SimPhiNCovariate\(\)](#), [SimSSMFixed\(\)](#), [SimSSMIVary\(\)](#), [SimSSMLinGrowth\(\)](#), [SimSSMLinGrowthIVary\(\)](#), [SimSSMLinSDEFixed\(\)](#), [SimSSMLinSDEIVary\(\)](#), [SimSSMOUFixed\(\)](#), [SimSSMOUIVary\(\)](#), [SimSSMVARFixed\(\)](#), [SimSSMVARIVary\(\)](#), [SpectralRadius\(\)](#), [TestPhi\(\)](#), [TestPhiHurwitz\(\)](#), [TestStability\(\)](#), [TestStationarity\(\)](#)

Examples

```

beta <- matrix(
  data = c(
    0.7, 0.5, -0.1,
    0.0, 0.6, 0.4,
    0.0, 0.0, 0.5
  ),
  nrow = 3
)
psi <- 0.1 * diag(3)
lambda <- diag(3)
theta <- diag(3)
cov_eta <- SSMCovEta(
  beta = beta,
  psi = psi
)
SSMCovY(
  lambda = lambda,
  theta = theta,
  cov_eta = cov_eta
)

```

SSMInterceptEta	<i>Intercept from Steady-State Mean Vector for the Latent Variables in the State Space Model</i>
-----------------	--

Description

The intercept vector for the latent variables in the state space model α is given by

$$\alpha = \text{Mean}(\eta) - \beta \text{Mean}(\eta)$$

where β is the transition matrix relating the values of the latent variables at the previous to the current time point and $\text{Mean}(\eta)$ is the steady-state mean vector for the latent variables.

Usage

```
SSMInterceptEta(beta, mean_eta)
```

Arguments

beta	Numeric matrix. Transition matrix relating the values of the latent variables at the previous to the current time point (β).
mean_eta	Numeric vector. Steady-state mean vector of the latent variables $\text{Mean}(\eta)$.

Author(s)

Ivan Jacob Agaloos Pesigan

See Also

Other Simulation of State Space Models Data Functions: [LinSDE2SSM\(\)](#), [LinSDECovEta\(\)](#), [LinSDECovY\(\)](#), [LinSDEInterceptEta\(\)](#), [LinSDEInterceptY\(\)](#), [LinSDEMeanEta\(\)](#), [LinSDEMeanY\(\)](#), [ProjectToHurwitz\(\)](#), [ProjectToStability\(\)](#), [SSMCovEta\(\)](#), [SSMCovY\(\)](#), [SSMInterceptY\(\)](#), [SSMMeanEta\(\)](#), [SSMMeanY\(\)](#), [SimAlphaN\(\)](#), [SimBetaN\(\)](#), [SimBetaN2\(\)](#), [SimBetaNCovariate\(\)](#), [SimCovDiagN\(\)](#), [SimCovN\(\)](#), [SimIotaN\(\)](#), [SimMVN\(\)](#), [SimMuN\(\)](#), [SimNuN\(\)](#), [SimPhiN\(\)](#), [SimPhiN2\(\)](#), [SimPhiNCovariate\(\)](#), [SimSSMFixed\(\)](#), [SimSSMIVary\(\)](#), [SimSSMLinGrowth\(\)](#), [SimSSMLinGrowthIVary\(\)](#), [SimSSMLinSDEFixed\(\)](#), [SimSSMLinSDEIVary\(\)](#), [SimSSMOUFixed\(\)](#), [SimSSMOUIVary\(\)](#), [SimSSMVARFixed\(\)](#), [SimSSMVARIVary\(\)](#), [SpectralRadius\(\)](#), [TestPhi\(\)](#), [TestPhiHurwitz\(\)](#), [TestStability\(\)](#), [TestStationarity\(\)](#)

Examples

```
beta <- matrix(
  data = c(
    0.7, 0.5, -0.1,
    0.0, 0.6, 0.4,
    0.0, 0.0, 0.5
  ),
  nrow = 3
)
```

```

alpha <- rep(x = 1, times = 3)
mean_eta <- SSMMeanEta(
  beta = beta,
  alpha = alpha
)
SSMInterceptEta(
  beta = beta,
  mean_eta = mean_eta
)

```

SSMInterceptY	<i>Intercept from Steady-State Mean Vector for the Observed Variables in the State Space Model</i>
---------------	--

Description

The intercept vector for the observed variables in the state space model ν is given by

$$\nu = \text{Mean}(\mathbf{y}) - \mathbf{\Lambda} \text{Mean}(\boldsymbol{\eta})$$

where $\mathbf{\Lambda}$ is the matrix of factor loadings, $\text{Mean}(\mathbf{y})$ is the steady-state mean vector for the observed variables, and $\text{Mean}(\boldsymbol{\eta})$ is the steady-state mean vector for the latent variables.

Usage

```
SSMInterceptY(mean_y, mean_eta, lambda)
```

Arguments

mean_y	Numeric vector. Steady-state mean vector of the observed variables $\text{Mean}(\mathbf{y})$.
mean_eta	Numeric vector. Steady-state mean vector of the latent variables $\text{Mean}(\boldsymbol{\eta})$.
lambda	Numeric matrix. Factor loading matrix linking the latent variables to the observed variables ($\mathbf{\Lambda}$).

Author(s)

Ivan Jacob Agaloos Pesigan

See Also

Other Simulation of State Space Models Data Functions: [LinSDE2SSM\(\)](#), [LinSDECovEta\(\)](#), [LinSDECovY\(\)](#), [LinSDEInterceptEta\(\)](#), [LinSDEInterceptY\(\)](#), [LinSDEMeanEta\(\)](#), [LinSDEMeanY\(\)](#), [ProjectToHurwitz\(\)](#), [ProjectToStability\(\)](#), [SSMCovEta\(\)](#), [SSMCovY\(\)](#), [SSMInterceptEta\(\)](#), [SSMMeanEta\(\)](#), [SSMMeanY\(\)](#), [SimAlphaN\(\)](#), [SimBetaN\(\)](#), [SimBetaN2\(\)](#), [SimBetaNCovariate\(\)](#), [SimCovDiagN\(\)](#), [SimCovN\(\)](#), [SimIotaN\(\)](#), [SimMVN\(\)](#), [SimMuN\(\)](#), [SimNuN\(\)](#), [SimPhiN\(\)](#), [SimPhiN2\(\)](#), [SimPhiNCovariate\(\)](#), [SimSSMFixed\(\)](#), [SimSSMIVary\(\)](#), [SimSSMLinGrowth\(\)](#), [SimSSMLinGrowthIVary\(\)](#), [SimSSMLinSDEFixed\(\)](#), [SimSSMLinSDEIVary\(\)](#), [SimSSMOUFixed\(\)](#), [SimSSMOUIVary\(\)](#), [SimSSMVARFixed\(\)](#), [SimSSMVARIVary\(\)](#), [SpectralRadius\(\)](#), [TestPhi\(\)](#), [TestPhiHurwitz\(\)](#), [TestStability\(\)](#), [TestStationarity\(\)](#)

Examples

```

beta <- matrix(
  data = c(
    0.7, 0.5, -0.1,
    0.0, 0.6, 0.4,
    0.0, 0.0, 0.5
  ),
  nrow = 3
)
alpha <- rep(x = 1, times = 3)
lambda <- diag(3)
nu <- rep(x = 1, times = 3)
mean_eta <- SSMMeanEta(
  beta = beta,
  alpha = alpha
)
mean_y <- SSMMeanY(
  nu = nu,
  lambda = lambda,
  mean_eta = mean_eta
)
SSMInterceptY(
  mean_y = mean_y,
  mean_eta = mean_eta,
  lambda = lambda
)

```

SSMMeanEta

Steady-State Mean Vector for the Latent Variables in the State Space Model

Description

The steady-state mean vector for the latent variables in the state space model $\text{Mean}(\boldsymbol{\eta})$ is given by

$$\text{Mean}(\boldsymbol{\eta}) = (\mathbf{I} - \boldsymbol{\beta})^{-1} \boldsymbol{\alpha}$$

where $\boldsymbol{\beta}$ is the transition matrix relating the values of the latent variables at the previous to the current time point, $\boldsymbol{\alpha}$ is a vector of constant values for the dynamic model, and $\mathbf{I}$ is an identity matrix.

Usage

```
SSMMeanEta(beta, alpha)
```

Arguments

beta	Numeric matrix. Transition matrix relating the values of the latent variables at the previous to the current time point ($\boldsymbol{\beta}$).
alpha	Numeric vector. Vector of constant values for the dynamic model ($\boldsymbol{\alpha}$).

Author(s)

Ivan Jacob Agaloos Pesigan

See Also

Other Simulation of State Space Models Data Functions: [LinSDE2SSM\(\)](#), [LinSDECovEta\(\)](#), [LinSDECovY\(\)](#), [LinSDEInterceptEta\(\)](#), [LinSDEInterceptY\(\)](#), [LinSDEMeanEta\(\)](#), [LinSDEMeanY\(\)](#), [ProjectToHurwitz\(\)](#), [ProjectToStability\(\)](#), [SSMCovEta\(\)](#), [SSMCovY\(\)](#), [SSMInterceptEta\(\)](#), [SSMInterceptY\(\)](#), [SSMMeanY\(\)](#), [SimAlphaN\(\)](#), [SimBetaN\(\)](#), [SimBetaN2\(\)](#), [SimBetaNCovariate\(\)](#), [SimCovDiagN\(\)](#), [SimCovN\(\)](#), [SimIotaN\(\)](#), [SimMVN\(\)](#), [SimMuN\(\)](#), [SimNuN\(\)](#), [SimPhiN\(\)](#), [SimPhiN2\(\)](#), [SimPhiNCovariate\(\)](#), [SimSSMFixed\(\)](#), [SimSSMIVary\(\)](#), [SimSSMLinGrowth\(\)](#), [SimSSMLinGrowthIVary\(\)](#), [SimSSMLinSDEFixed\(\)](#), [SimSSMLinSDEIVary\(\)](#), [SimSSMOUFixed\(\)](#), [SimSSMOUIVary\(\)](#), [SimSSMVARFixed\(\)](#), [SimSSMVARIVary\(\)](#), [SpectralRadius\(\)](#), [TestPhi\(\)](#), [TestPhiHurwitz\(\)](#), [TestStability\(\)](#), [TestStationarity\(\)](#)

Examples

```
beta <- matrix(
  data = c(
    0.7, 0.5, -0.1,
    0.0, 0.6, 0.4,
    0.0, 0.0, 0.5
  ),
  nrow = 3
)
alpha <- rep(x = 1, times = 3)
SSMMeanEta(
  beta = beta,
  alpha = alpha
)
```

SSMMeanY

Steady-State Mean Vector for the Observed Variables in the State Space Model

Description

The steady-state mean vector for the observed variables in the state space model $\text{Mean}(\mathbf{y})$ is given by

$$\text{Mean}(\mathbf{y}) = \boldsymbol{\nu} + \mathbf{\Lambda} \text{Mean}(\boldsymbol{\eta})$$

where $\boldsymbol{\nu}$ is the vector of intercept values for the measurement model, $\mathbf{\Lambda}$ is the matrix of factor loadings, and $\text{Mean}(\boldsymbol{\eta})$ is the steady-state mean vector for the latent variables.

Usage

```
SSMMeanY(nu, lambda, mean_eta)
```


Arguments

nu	Numeric vector. Vector of intercept values for the measurement model (ν).
lambda	Numeric matrix. Factor loading matrix linking the latent variables to the observed variables (Λ).
mean_eta	Numeric vector. Steady-state mean vector of the latent variables Mean (η).

Author(s)

Ivan Jacob Agaloos Pesigan

See Also

Other Simulation of State Space Models Data Functions: [LinSDE2SSM\(\)](#), [LinSDECovEta\(\)](#), [LinSDECovY\(\)](#), [LinSDEInterceptEta\(\)](#), [LinSDEInterceptY\(\)](#), [LinSDEMeanEta\(\)](#), [LinSDEMeanY\(\)](#), [ProjectToHurwitz\(\)](#), [ProjectToStability\(\)](#), [SSMCovEta\(\)](#), [SSMCovY\(\)](#), [SSMInterceptEta\(\)](#), [SSMInterceptY\(\)](#), [SSMMeanEta\(\)](#), [SimAlphaN\(\)](#), [SimBetaN\(\)](#), [SimBetaN2\(\)](#), [SimBetaNCovariate\(\)](#), [SimCovDiagN\(\)](#), [SimCovN\(\)](#), [SimIotaN\(\)](#), [SimMVN\(\)](#), [SimMuN\(\)](#), [SimNuN\(\)](#), [SimPhiN\(\)](#), [SimPhiN2\(\)](#), [SimPhiNCovariate\(\)](#), [SimSSMFixed\(\)](#), [SimSSMIVary\(\)](#), [SimSSMLinGrowth\(\)](#), [SimSSMLinGrowthIVary\(\)](#), [SimSSMLinSDEFixed\(\)](#), [SimSSMLinSDEIVary\(\)](#), [SimSSMOUFixed\(\)](#), [SimSSMOUIVary\(\)](#), [SimSSMVARFixed\(\)](#), [SimSSMVARIVary\(\)](#), [SpectralRadius\(\)](#), [TestPhi\(\)](#), [TestPhiHurwitz\(\)](#), [TestStability\(\)](#), [TestStationarity\(\)](#)

Examples

```

beta <- matrix(
  data = c(
    0.7, 0.5, -0.1,
    0.0, 0.6, 0.4,
    0.0, 0.0, 0.5
  ),
  nrow = 3
)
alpha <- rep(x = 1, times = 3)
lambda <- diag(3)
nu <- rep(x = 1, times = 3)
mean_eta <- SSMMeanEta(
  beta = beta,
  alpha = alpha
)
SSMMeanY(
  nu = nu,
  lambda = lambda,
  mean_eta = mean_eta
)

```

TestPhi	<i>Test the Drift Matrix</i>
---------	------------------------------

Description

Both have to be true for the function to return TRUE.

- Test that the real part of all eigenvalues of Φ are less than zero.
- Test that the diagonal values of Φ are between 0 to negative infinity.

Usage

```
TestPhi(phi, margin = 0, auto_ubound = 0)
```

Arguments

phi	Numeric matrix. The drift matrix (Φ).
margin	Numeric scalar specifying the stability threshold for the real part of the eigenvalues. The default 0.0 corresponds to the imaginary axis; values less than 0.0 enforce a stricter stability margin.
auto_ubound	Numeric scalar specifying the upper bound for the diagonal elements of Φ . Default is 0.0, requiring all diagonal values to be ≤ 0 .

Author(s)

Ivan Jacob Agaloos Pesigan

See Also

Other Simulation of State Space Models Data Functions: [LinSDE2SSM\(\)](#), [LinSDECovEta\(\)](#), [LinSDECovY\(\)](#), [LinSDEInterceptEta\(\)](#), [LinSDEInterceptY\(\)](#), [LinSDEMeanEta\(\)](#), [LinSDEMeanY\(\)](#), [ProjectToHurwitz\(\)](#), [ProjectToStability\(\)](#), [SSMCovEta\(\)](#), [SSMCovY\(\)](#), [SSMInterceptEta\(\)](#), [SSMInterceptY\(\)](#), [SSMMeanEta\(\)](#), [SSMMeanY\(\)](#), [SimAlphaN\(\)](#), [SimBetaN\(\)](#), [SimBetaN2\(\)](#), [SimBetaNCovariate\(\)](#), [SimCovDiagN\(\)](#), [SimCovN\(\)](#), [SimIotaN\(\)](#), [SimMVN\(\)](#), [SimMuN\(\)](#), [SimNuN\(\)](#), [SimPhiN\(\)](#), [SimPhiN2\(\)](#), [SimPhiNCovariate\(\)](#), [SimSSMFixed\(\)](#), [SimSSMIVary\(\)](#), [SimSSMLinGrowth\(\)](#), [SimSSMLinGrowthIVary\(\)](#), [SimSSMLinSDEFixed\(\)](#), [SimSSMLinSDEIVary\(\)](#), [SimSSMOUFixed\(\)](#), [SimSSMOUIVary\(\)](#), [SimSSMVARFixed\(\)](#), [SimSSMVARIVary\(\)](#), [SpectralRadius\(\)](#), [TestPhiHurwitz\(\)](#), [TestStability\(\)](#), [TestStationarity\(\)](#)

Examples

```
phi <- matrix(
  data = c(
    -0.357, 0.771, -0.450,
    0.0, -0.511, 0.729,
    0, 0, -0.693
  ),
  nrow = 3
)
```

```
TestPhi(phi = phi)
```

TestPhiHurwitz

Test Hurwitz Stability of a Drift Matrix

Description

Returns TRUE iff the drift matrix Φ is Hurwitz-stable, i.e., all eigenvalues have real parts strictly less than $-\text{eps}$. Setting $\text{eps} = 0$ enforces the usual strict condition $\max \Re\{\lambda_i(\Phi)\} < 0$. A small positive eps (e.g., $1\text{e-}12$) can be used to guard against floating-point round-off.

Usage

```
TestPhiHurwitz(phi, eps = 0)
```

Arguments

phi	Numeric matrix. The drift matrix (Φ).
eps	Nonnegative numeric tolerance (default 0.0). The test checks $\Re(\lambda_i) < -\text{eps}$ for all eigenvalues.

Author(s)

Ivan Jacob Agaloos Pesigan

See Also

Other Simulation of State Space Models Data Functions: [LinSDE2SSM\(\)](#), [LinSDECovEta\(\)](#), [LinSDECovY\(\)](#), [LinSDEInterceptEta\(\)](#), [LinSDEInterceptY\(\)](#), [LinSDEMeanEta\(\)](#), [LinSDEMeanY\(\)](#), [ProjectToHurwitz\(\)](#), [ProjectToStability\(\)](#), [SSMCovEta\(\)](#), [SSMCovY\(\)](#), [SSMInterceptEta\(\)](#), [SSMInterceptY\(\)](#), [SSMMeanEta\(\)](#), [SSMMeanY\(\)](#), [SimAlphaN\(\)](#), [SimBetaN\(\)](#), [SimBetaN2\(\)](#), [SimBetaNCovariate\(\)](#), [SimCovDiagN\(\)](#), [SimCovN\(\)](#), [SimIotaN\(\)](#), [SimMVN\(\)](#), [SimMuN\(\)](#), [SimNuN\(\)](#), [SimPhiN\(\)](#), [SimPhiN2\(\)](#), [SimPhiNCovariate\(\)](#), [SimSSMFixed\(\)](#), [SimSSMIVary\(\)](#), [SimSSMLinGrowth\(\)](#), [SimSSMLinGrowthIVary\(\)](#), [SimSSMLinSDEFixed\(\)](#), [SimSSMLinSDEIVary\(\)](#), [SimSSMOUFixed\(\)](#), [SimSSMOUIVary\(\)](#), [SimSSMVARFixed\(\)](#), [SimSSMVARIVary\(\)](#), [SpectralRadius\(\)](#), [TestPhi\(\)](#), [TestStability\(\)](#), [TestStationarity\(\)](#)

Examples

```
# Unstable example (spectral abscissa >= 0):
phi <- matrix(
  data = c(
    0.10, -0.40,
    0.50, 0.20
  ),
  nrow = 2
)
TestPhiHurwitz(phi = phi) # FALSE
```

```
# Stable example (all real parts < 0):
phi <- matrix(
  data = c(
    -0.50, -0.20,
    1.00, -0.30
  ),
  nrow = 2
)
TestPhiHurwitz(phi = phi) # TRUE
TestPhiHurwitz(phi = phi, eps = 1e-12) # also TRUE with tolerance
```

TestStability

Test Stability

Description

The function computes the eigenvalues of the input matrix x . It checks if the real part of all eigenvalues is negative. If all eigenvalues have negative real parts, the system is considered stable.

Usage

```
TestStability(x, margin = 0)
```

Arguments

x	Numeric matrix.
$margin$	Numeric scalar specifying the stability threshold for the real part of the eigenvalues. The default 0.0 corresponds to the imaginary axis; values less than 0.0 enforce a stricter stability margin.

Author(s)

Ivan Jacob Agaloos Pesigan

See Also

Other Simulation of State Space Models Data Functions: [LinSDE2SSM\(\)](#), [LinSDECovEta\(\)](#), [LinSDECovY\(\)](#), [LinSDEInterceptEta\(\)](#), [LinSDEInterceptY\(\)](#), [LinSDEMeanEta\(\)](#), [LinSDEMeanY\(\)](#), [ProjectToHurwitz\(\)](#), [ProjectToStability\(\)](#), [SSMCovEta\(\)](#), [SSMCovY\(\)](#), [SSMInterceptEta\(\)](#), [SSMInterceptY\(\)](#), [SSMMeanEta\(\)](#), [SSMMeanY\(\)](#), [SimAlphaN\(\)](#), [SimBetaN\(\)](#), [SimBetaN2\(\)](#), [SimBetaNCovariate\(\)](#), [SimCovDiagN\(\)](#), [SimCovN\(\)](#), [SimIotaN\(\)](#), [SimMVN\(\)](#), [SimMuN\(\)](#), [SimNuN\(\)](#), [SimPhiN\(\)](#), [SimPhiN2\(\)](#), [SimPhiNCovariate\(\)](#), [SimSSMFixed\(\)](#), [SimSSMIVary\(\)](#), [SimSSMLinGrowth\(\)](#), [SimSSMLinGrowthIVary\(\)](#), [SimSSMLinSDEFixed\(\)](#), [SimSSMLinSDEIVary\(\)](#), [SimSSMOUFixed\(\)](#), [SimSSMOUIVary\(\)](#), [SimSSMVARFixed\(\)](#), [SimSSMVARIVary\(\)](#), [SpectralRadius\(\)](#), [TestPhi\(\)](#), [TestPhiHurwitz\(\)](#), [TestStationarity\(\)](#)

Examples

```
x <- matrix(
  data = c(
    -0.357, 0.771, -0.450,
    0.0, -0.511, 0.729,
    0, 0, -0.693
  ),
  nrow = 3
)
TestStability(x)
```

TestStationarity	<i>Test Stationarity</i>
------------------	--------------------------

Description

The function computes the eigenvalues of the input matrix `x`. It checks if all eigenvalues have moduli less than 1. If all eigenvalues have moduli less than 1, the system is considered stationary.

Usage

```
TestStationarity(x, margin = 1)
```

Arguments

<code>x</code>	Numeric matrix.
<code>margin</code>	Numeric scalar specifying the stationarity threshold. Values less than 1 indicate stricter stationarity criteria.

Author(s)

Ivan Jacob Agaloos Pesigan

See Also

Other Simulation of State Space Models Data Functions: [LinSDE2SSM\(\)](#), [LinSDECovEta\(\)](#), [LinSDECovY\(\)](#), [LinSDEInterceptEta\(\)](#), [LinSDEInterceptY\(\)](#), [LinSDEMeanEta\(\)](#), [LinSDEMeanY\(\)](#), [ProjectToHurwitz\(\)](#), [ProjectToStability\(\)](#), [SSMCovEta\(\)](#), [SSMCovY\(\)](#), [SSMInterceptEta\(\)](#), [SSMInterceptY\(\)](#), [SSMMeanEta\(\)](#), [SSMMeanY\(\)](#), [SimAlphaN\(\)](#), [SimBetaN\(\)](#), [SimBetaN2\(\)](#), [SimBetaNCovariate\(\)](#), [SimCovDiagN\(\)](#), [SimCovN\(\)](#), [SimIotaN\(\)](#), [SimMVN\(\)](#), [SimMuN\(\)](#), [SimNuN\(\)](#), [SimPhiN\(\)](#), [SimPhiN2\(\)](#), [SimPhiNCovariate\(\)](#), [SimSSMFixed\(\)](#), [SimSSMIVary\(\)](#), [SimSSMLinGrowth\(\)](#), [SimSSMLinGrowthIVary\(\)](#), [SimSSMLinSDEFixed\(\)](#), [SimSSMLinSDEIVary\(\)](#), [SimSSMOUFixed\(\)](#), [SimSSMOUIVary\(\)](#), [SimSSMVARFixed\(\)](#), [SimSSMVARIVary\(\)](#), [SpectralRadius\(\)](#), [TestPhi\(\)](#), [TestPhiHurwitz\(\)](#), [TestStability\(\)](#)

Examples

```
x <- matrix(  
  data = c(0.5, 0.3, 0.2, 0.4),  
  nrow = 2  
)  
TestStationarity(x)
```

```
x <- matrix(  
  data = c(0.9, -0.5, 0.8, 0.7),  
  nrow = 2  
)  
TestStationarity(x)
```

Index

* Simulation of State Space Models Data

Functions

LinSDE2SSM, 8
LinSDECovEta, 10
LinSDECovY, 12
LinSDEInterceptEta, 13
LinSDEInterceptY, 14
LinSDEMeanEta, 16
LinSDEMeanY, 17
ProjectToHurwitz, 23
ProjectToStability, 25
SimAlphaN, 26
SimBetaN, 27
SimBetaN2, 29
SimBetaNCovariate, 30
SimCovDiagN, 32
SimCovN, 33
SimIotaN, 34
SimMuN, 35
SimMVN, 36
SimNuN, 37
SimPhiN, 38
SimPhiN2, 40
SimPhiNCovariate, 41
SimSSMFixed, 43
SimSSMIVary, 48
SimSSMLinGrowth, 52
SimSSMLinGrowthIVary, 56
SimSSMLinSDEFixed, 60
SimSSMLinSDEIVary, 65
SimSSMOUFixed, 70
SimSSMOUIVary, 76
SimSSMVARFixed, 81
SimSSMVARIVary, 84
SpectralRadius, 89
SSMCovEta, 90
SSMCovY, 91
SSMInterceptEta, 93
SSMInterceptY, 94

SSMMeanEta, 95

SSMMeanY, 96

TestPhi, 98

TestPhiHurwitz, 99

TestStability, 100

TestStationarity, 101

* growth

SimSSMLinGrowth, 52

SimSSMLinGrowthIVary, 56

* linsde

LinSDE2SSM, 8

LinSDECovEta, 10

LinSDECovY, 12

LinSDEInterceptEta, 13

LinSDEInterceptY, 14

LinSDEMeanEta, 16

LinSDEMeanY, 17

ProjectToHurwitz, 23

SimPhiN, 38

SimPhiN2, 40

SimPhiNCovariate, 41

SimSSMLinSDEFixed, 60

SimSSMLinSDEIVary, 65

SpectralAbscissa, 88

TestPhi, 98

TestPhiHurwitz, 99

TestStability, 100

* methods

as.data.frame.simstatespace, 3

as.matrix.simstatespace, 5

plot.simstatespace, 18

print.simstatespace, 21

* ou

SimSSMOUFixed, 70

SimSSMOUIVary, 76

* simStateSpace

LinSDE2SSM, 8

LinSDECovEta, 10

LinSDECovY, 12

- LinSDEInterceptEta, [13](#)
- LinSDEInterceptY, [14](#)
- LinSDEMeanEta, [16](#)
- LinSDEMeanY, [17](#)
- ProjectToHurwitz, [23](#)
- ProjectToStability, [25](#)
- SimAlphaN, [26](#)
- SimBetaN, [27](#)
- SimBetaN2, [29](#)
- SimBetaNCovariate, [30](#)
- SimCovDiagN, [32](#)
- SimCovN, [33](#)
- SimIotaN, [34](#)
- SimMuN, [35](#)
- SimMVN, [36](#)
- SimNuN, [37](#)
- SimPhiN, [38](#)
- SimPhiN2, [40](#)
- SimPhiNCovariate, [41](#)
- SimSSMFixed, [43](#)
- SimSSMIVary, [48](#)
- SimSSMLinGrowth, [52](#)
- SimSSMLinGrowthIVary, [56](#)
- SimSSMLinSDEFixed, [60](#)
- SimSSMLinSDEIVary, [65](#)
- SimSSMOUFixed, [70](#)
- SimSSMOUIVary, [76](#)
- SimSSMVARFixed, [81](#)
- SimSSMVARIVary, [84](#)
- SpectralAbscissa, [88](#)
- SpectralRadius, [89](#)
- SSMCovEta, [90](#)
- SSMCovY, [91](#)
- SSMInterceptEta, [93](#)
- SSMInterceptY, [94](#)
- SSMMeanEta, [95](#)
- SSMMeanY, [96](#)
- TestPhi, [98](#)
- TestPhiHurwitz, [99](#)
- TestStability, [100](#)
- TestStationarity, [101](#)
- * sim**
 - SimSSMFixed, [43](#)
 - SimSSMIVary, [48](#)
 - SimSSMLinGrowth, [52](#)
 - SimSSMLinGrowthIVary, [56](#)
 - SimSSMLinSDEFixed, [60](#)
 - SimSSMLinSDEIVary, [65](#)
 - SimSSMOUFixed, [70](#)
 - SimSSMOUIVary, [76](#)
 - SimSSMVARFixed, [81](#)
 - SimSSMVARIVary, [84](#)
- * ssm**
 - ProjectToStability, [25](#)
 - SimAlphaN, [26](#)
 - SimBetaN, [27](#)
 - SimBetaN2, [29](#)
 - SimBetaNCovariate, [30](#)
 - SimCovDiagN, [32](#)
 - SimCovN, [33](#)
 - SimIotaN, [34](#)
 - SimMuN, [35](#)
 - SimMVN, [36](#)
 - SimNuN, [37](#)
 - SimSSMFixed, [43](#)
 - SimSSMIVary, [48](#)
 - SpectralRadius, [89](#)
 - SSMCovEta, [90](#)
 - SSMCovY, [91](#)
 - SSMInterceptEta, [93](#)
 - SSMInterceptY, [94](#)
 - SSMMeanEta, [95](#)
 - SSMMeanY, [96](#)
 - TestStationarity, [101](#)
- * stability**
 - ProjectToHurwitz, [23](#)
 - ProjectToStability, [25](#)
 - SpectralAbscissa, [88](#)
 - SpectralRadius, [89](#)
- * test**
 - TestPhi, [98](#)
 - TestPhiHurwitz, [99](#)
 - TestStability, [100](#)
 - TestStationarity, [101](#)
- * transformation**
 - LinSDE2SSM, [8](#)
- * var**
 - SimSSMVARFixed, [81](#)
 - SimSSMVARIVary, [84](#)
- as.data.frame.simstatespace, [3](#)
- as.matrix.simstatespace, [5](#)
- LinSDE2SSM, [8](#)
- LinSDE2SSM(), [11](#), [12](#), [14–17](#), [24](#), [25](#), [27–29](#), [31–33](#), [35–40](#), [42](#), [46](#), [50](#), [55](#), [58](#), [63](#), [68](#), [73](#), [78](#), [83](#), [86](#), [89](#), [91–94](#), [96–101](#)

- LinSDECovEta, 10
- LinSDECovEta(), 9, 12, 14–17, 24, 25, 27–29, 31–33, 35–40, 42, 46, 50, 55, 58, 63, 68, 73, 78, 83, 86, 89, 91–94, 96–101
- LinSDECovY, 12
- LinSDECovY(), 9, 11, 14–17, 24, 25, 27–29, 31–33, 35–40, 42, 46, 50, 55, 58, 63, 68, 73, 78, 83, 86, 89, 91–94, 96–101
- LinSDEInterceptEta, 13
- LinSDEInterceptEta(), 9, 11, 12, 15–17, 24, 25, 27–29, 31–33, 35–40, 42, 46, 50, 55, 58, 63, 68, 73, 78, 83, 86, 89, 91–94, 96–101
- LinSDEInterceptY, 14
- LinSDEInterceptY(), 9, 11, 12, 14, 16, 17, 24, 25, 27–29, 31–33, 35–40, 42, 46, 50, 55, 58, 63, 68, 73, 78, 83, 86, 89, 91–94, 96–101
- LinSDEMeanEta, 16
- LinSDEMeanEta(), 9, 11, 12, 14, 15, 17, 24, 25, 27–29, 31–33, 35–40, 42, 46, 50, 55, 58, 63, 68, 73, 78, 83, 86, 89, 91–94, 96–101
- LinSDEMeanY, 17
- LinSDEMeanY(), 9, 11, 12, 14–16, 24, 25, 27–29, 31–33, 35–40, 42, 46, 50, 55, 58, 63, 68, 73, 78, 83, 86, 89, 91–94, 96–101
- plot.default(), 19
- plot.simstatespace, 18
- print.simstatespace, 21
- ProjectToHurwitz, 23
- ProjectToHurwitz(), 9, 11, 12, 14–17, 25, 27–29, 31–33, 35–40, 42, 46, 50, 55, 58, 63, 68, 73, 78, 83, 86, 89, 91–94, 96–101
- ProjectToStability, 25
- ProjectToStability(), 9, 11, 12, 14–17, 24, 27–29, 31–33, 35–40, 42, 46, 50, 55, 58, 63, 68, 73, 78, 83, 86, 89, 91–94, 96–101
- SimAlphaN, 26
- SimAlphaN(), 9, 11, 12, 14–17, 24, 25, 28, 29, 31–33, 35–40, 42, 46, 50, 55, 58, 63, 68, 73, 78, 83, 86, 89, 91–94, 96–101
- SimBetaN, 27
- SimBetaN(), 9, 11, 12, 14–17, 24, 25, 27, 29, 31–33, 35–40, 42, 46, 50, 55, 58, 63, 68, 73, 78, 83, 86, 89, 91–94, 96–101
- SimBetaN2, 29
- SimBetaN2(), 9, 11, 12, 14–17, 24, 25, 27, 28, 31–33, 35–40, 42, 46, 50, 55, 58, 63, 68, 73, 78, 83, 86, 89, 91–94, 96–101
- SimBetaNCovariate, 30
- SimBetaNCovariate(), 9, 11, 12, 14–17, 24, 25, 27–29, 32, 33, 35–40, 42, 46, 50, 55, 58, 63, 68, 73, 78, 83, 86, 89, 91–94, 96–101
- SimCovDiagN, 32
- SimCovDiagN(), 9, 11, 12, 14–17, 24, 25, 27–29, 31, 33, 35–40, 42, 46, 50, 55, 58, 63, 68, 73, 78, 83, 86, 89, 91–94, 96–101
- SimCovN, 33
- SimCovN(), 9, 11, 12, 14–17, 24, 25, 27–29, 31, 32, 35–40, 42, 46, 50, 55, 58, 63, 68, 73, 78, 83, 86, 89, 91–94, 96–101
- SimIotaN, 34
- SimIotaN(), 9, 11, 12, 14–17, 24, 25, 27–29, 31–33, 36–40, 42, 46, 50, 55, 58, 63, 68, 73, 78, 83, 86, 89, 91–94, 96–101
- SimMuN, 35
- SimMuN(), 9, 11, 12, 14–17, 24, 25, 27–29, 31–33, 35, 37–40, 42, 46, 50, 55, 58, 63, 68, 73, 78, 83, 86, 89, 91–94, 96–101
- SimMVN, 36
- SimMVN(), 9, 11, 12, 14–17, 24, 25, 27–29, 31–33, 35, 36, 38–40, 42, 46, 50, 55, 58, 63, 68, 73, 78, 83, 86, 89, 91–94, 96–101
- SimNuN, 37
- SimNuN(), 9, 11, 12, 14–17, 24, 25, 27–29, 31–33, 35–37, 39, 40, 42, 46, 50, 55, 58, 63, 68, 73, 78, 83, 86, 89, 91–94, 96–101
- SimPhiN, 38
- SimPhiN(), 9, 11, 12, 14–17, 24, 25, 27–29, 31–33, 35–38, 40, 42, 46, 50, 55, 58, 63, 68, 73, 78, 83, 86, 89, 91–94, 96–101
- SimPhiN2, 40
- SimPhiN2(), 9, 11, 12, 14–17, 24, 25, 27–29, 31–33, 35–39, 42, 46, 50, 55, 58, 63,

- 68, 73, 78, 83, 86, 89, 91–94, 96–101
- SimPhiNCovariate, 41
- SimPhiNCovariate(), 9, 11, 12, 14–17, 24, 25, 27–29, 31–33, 35–40, 46, 50, 55, 58, 63, 68, 73, 78, 83, 86, 89, 91–94, 96–101
- SimSSMFixed, 43
- SimSSMFixed(), 9, 11, 12, 14–17, 24, 25, 27–29, 31–33, 35–40, 42, 49, 50, 55, 58, 63, 68, 73, 78, 83, 86, 89, 91–94, 96–101
- SimSSMIVary, 48
- SimSSMIVary(), 9, 11, 12, 14–17, 24, 25, 27–29, 31–33, 35–40, 42, 46, 55, 58, 63, 68, 73, 78, 83, 86, 89, 91–94, 96–101
- SimSSMLinGrowth, 52
- SimSSMLinGrowth(), 9, 11, 12, 14–17, 24, 25, 27–29, 31–33, 35–40, 42, 46, 50, 57, 58, 63, 68, 73, 78, 83, 86, 89, 91–94, 96–101
- SimSSMLinGrowthIVary, 56
- SimSSMLinGrowthIVary(), 9, 11, 12, 14–17, 24, 25, 27–29, 31–33, 35–40, 42, 46, 50, 55, 63, 68, 73, 78, 83, 86, 89, 91–94, 96–101
- SimSSMLinSDEFixed, 60
- SimSSMLinSDEFixed(), 9, 11, 12, 14–17, 24, 25, 27–29, 31–33, 35–40, 42, 46, 50, 55, 58, 67, 68, 73, 78, 83, 86, 89, 91–94, 96–101
- SimSSMLinSDEIVary, 65
- SimSSMLinSDEIVary(), 9, 11, 12, 14–17, 24, 25, 27–29, 31–33, 35–40, 42, 46, 50, 55, 58, 63, 73, 78, 83, 86, 89, 91–94, 96–101
- SimSSMOUFixed, 70
- SimSSMOUFixed(), 9, 11, 12, 14–17, 24, 25, 27–29, 31–33, 35–40, 42, 46, 50, 55, 58, 63, 68, 77, 78, 83, 86, 89, 91–94, 96–101
- SimSSMOUIVary, 76
- SimSSMOUIVary(), 9, 11, 12, 14–17, 24, 25, 27–29, 31–33, 35–40, 42, 46, 50, 55, 58, 63, 68, 73, 83, 86, 89, 91–94, 96–101
- SimSSMVARFixed, 81
- SimSSMVARFixed(), 9, 11, 12, 14–17, 24, 25, 27–29, 31–33, 35–40, 42, 46, 50, 55, 58, 63, 68, 73, 78, 85, 86, 89, 91–94, 96–101
- SimSSMVARIVary, 84
- SimSSMVARIVary(), 9, 11, 12, 14–17, 24, 25, 27–29, 31–33, 35–40, 42, 46, 50, 55, 58, 63, 68, 73, 78, 83, 89, 91–94, 96–101
- SpectralAbscissa, 88
- SpectralRadius, 89
- SpectralRadius(), 9, 11, 12, 14–17, 24, 25, 27–29, 31–33, 35–40, 42, 46, 50, 55, 58, 63, 68, 73, 78, 83, 86, 91–94, 96–101
- SSMCovEta, 90
- SSMCovEta(), 9, 11, 12, 14–17, 24, 25, 27–29, 31–33, 35–40, 42, 46, 50, 55, 58, 63, 68, 73, 78, 83, 86, 89, 92–94, 96–101
- SSMCovY, 91
- SSMCovY(), 9, 11, 12, 14–17, 24, 25, 27–29, 31–33, 35–40, 42, 46, 50, 55, 58, 63, 68, 73, 78, 83, 86, 89, 91, 93, 94, 96–101
- SSMInterceptEta, 93
- SSMInterceptEta(), 9, 11, 12, 14–17, 24, 25, 27–29, 31–33, 35–40, 42, 46, 50, 55, 58, 63, 68, 73, 78, 83, 86, 89, 91, 92, 94, 96–101
- SSMInterceptY, 94
- SSMInterceptY(), 9, 11, 12, 14–17, 24, 25, 27–29, 31–33, 35–40, 42, 46, 50, 55, 58, 63, 68, 73, 78, 83, 86, 89, 91–93, 96–101
- SSMMeanEta, 95
- SSMMeanEta(), 9, 11, 12, 14–17, 24, 25, 27–29, 31–33, 35–40, 42, 46, 50, 55, 58, 63, 68, 73, 78, 83, 86, 89, 91–94, 97–101
- SSMMeanY, 96
- SSMMeanY(), 9, 11, 12, 14–17, 24, 25, 27–29, 31–33, 35–40, 42, 46, 50, 55, 58, 63, 68, 73, 78, 83, 86, 89, 91–94, 96, 98–101
- TestPhi, 98
- TestPhi(), 9, 11, 12, 14–17, 24, 25, 27–29, 31–33, 35–42, 46, 50, 55, 58, 63, 68, 73, 78, 83, 86, 89, 91–94, 96, 97, 99–101

TestPhiHurwitz, 99
TestPhiHurwitz(), 9, 11, 12, 14–17, 24, 25,
27–29, 31–33, 35–40, 42, 46, 50, 55,
58, 63, 68, 73, 78, 83, 86, 89, 91–94,
96–98, 100, 101
TestStability, 100
TestStability(), 9, 11, 12, 14–17, 24, 25,
27–29, 31–33, 35–40, 42, 46, 50, 55,
58, 63, 68, 73, 78, 83, 86, 89, 91–94,
96–99, 101
TestStationarity, 101
TestStationarity(), 9, 11, 12, 14–17, 24,
25, 27–33, 35–40, 42, 46, 50, 55, 58,
63, 68, 73, 78, 83, 86, 89, 91–94,
96–100